

고급머신러닝 보고서

GAN & Transformer

Generative Adversarial Networks
Transformer

전자공학부 전자공학과
2020144005 김윤성
2024. 11. 26 제출

목차 INDEX

(눌러서 해당 목차로 이동)

01 GAN

- 생성자와 판별자
- 목적 함수
- JSD
- DCGAN
- CGAN
- LSGAN
- WGAN

02 실습

- VGAN
- DCGAN
- CGAN
- LSGAN
- WGAN
- 각 모델 비교

03 Transformer

- Attention
- Attention Is All You Need 논문 분석
- 실습 #1)

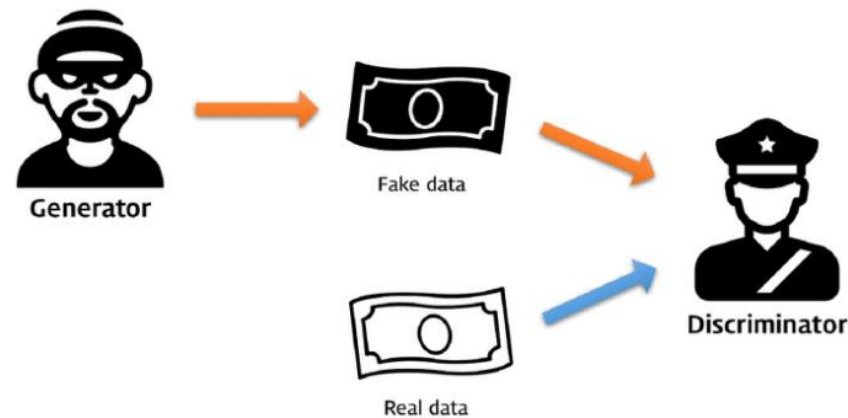
01 GAN

Generative Adversarial Networks

01 GAN Generative Adversarial Networks

GAN Generative Adversarial Networks

- 두 개의 신경망인 생성자(Generator)와 판별자(Discriminator)로 구성됨
- 생성자와 판별자는 서로 경쟁하며 학습하는 구조
- 생성자는 실제 데이터와 유사한 가짜 데이터를 생성하고, 판별자는 입력된 데이터가 실제인지 가짜인지 구별
- 이러한 경쟁적 학습 과정을 통해 생성자는 실제와 유사한 데이터 생성 가능



GAN의 구조

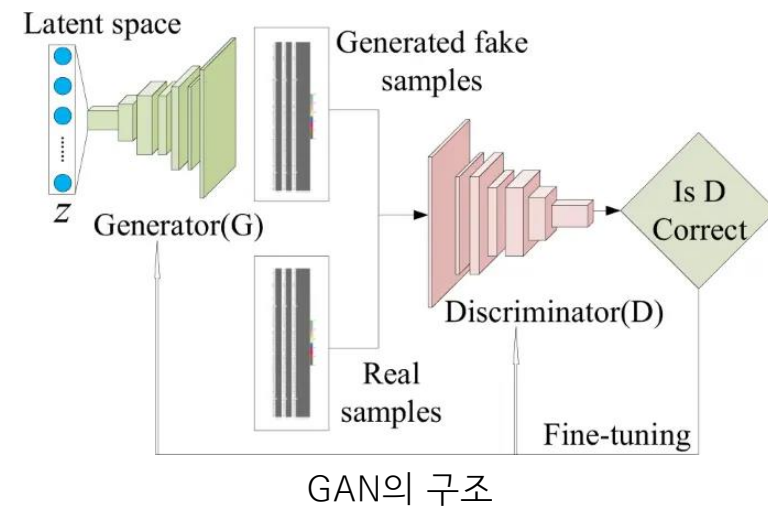
생성자 · 판별자 Generator · Discriminator

Generator

- 정규분포를 따르는 랜덤 노이즈 벡터를 입력받음
- 입력된 노이즈를 변환하여 실제 데이터와 유사한 가짜 데이터를 출력
- 일반적으로 MLP 또는 CNN으로 구성됨

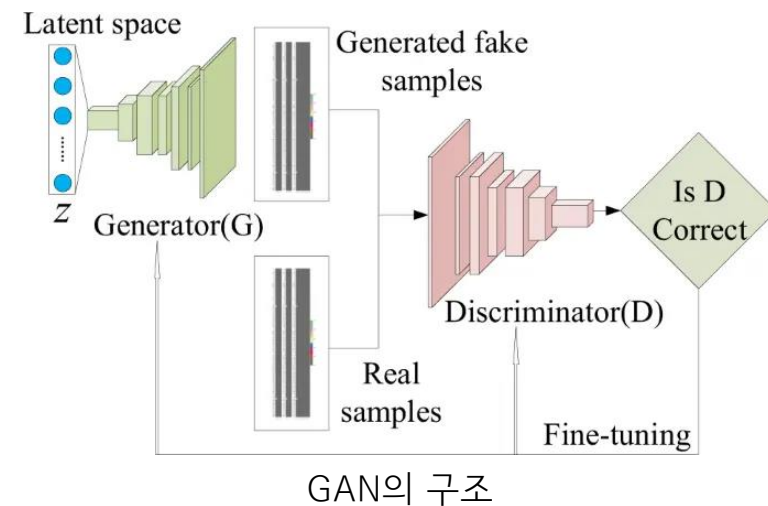
Discriminator

- 실제 데이터와 생성자가 만든 가짜 데이터 모두를 입력받음
- 입력 데이터가 실제일 확률을 나타내는 값
($D(G(z))$: 생성자가 만든 데이터를 입력받을 때, $D(x)$: 실제 데이터를 입력받을 때)을 출력
- 일반적으로 MLP 또는 CNN으로 구성됨



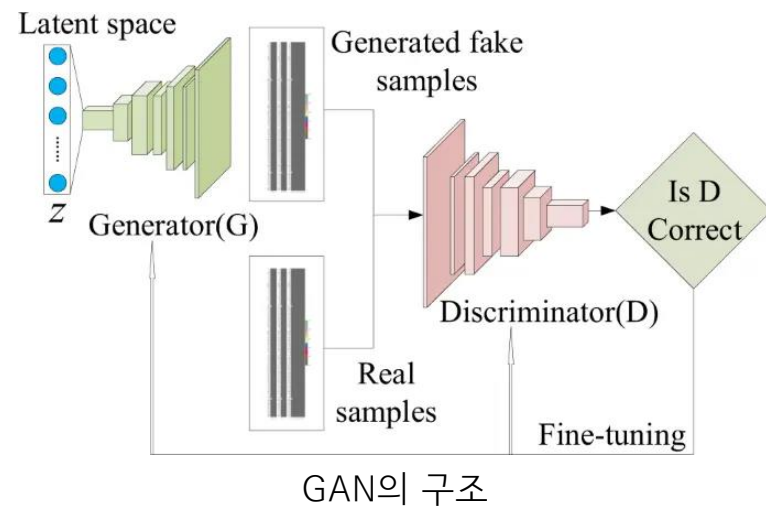
목적 함수 Objective Function

- Loss Function의 목표: 값이 최소화되는 지점을 찾기
GAN: 진짜 이미지에 대해서는 진짜일 확률이 높아지고,
가짜 이미지에 대해서는 진짜일 확률이 낮아져야 함 => Objective Function
- $G(z)$: Latent Vector Z 로 생성된 데이터 sample
 $D(x)$: 판별자가 판단했을 때, 실제 데이터일 확률
- $\min \max V(D, G) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_Z(z)} [\log(1 - D(G(Z)))]$ 로 정의되며,
G에 대해서 min, D에 대해서 max로 학습
- $E_{x \sim p_{data}(x)} [\log D(x)]$: 실제 데이터를 넣었을 때 판별자의 판단
 $E_{z \sim p_Z(z)} [\log(1 - D(G(Z)))]$: 가짜 데이터를 넣었을 때 판별자의 판단



목적 함수 (상세) Objective Function

- $\min \max V(D, G) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log(1 - D(G(z)))]$
- $p_{data}(x)$: 실제 데이터의 분포
 $p_z(z)$: Noise(Latent) Vector z 의 분포
 E : Expectation, 확률 변수의 평균값
- $E_{x \sim p_{data}(x)} [\log D(x)]$: 실제 데이터의 분포 $p_{data}(x)$ 에서 샘플링한 데이터 x 에 대해 $D(x)$ 에 로그를 취한 후, 기댓값 계산
=> 판별자가 실제 데이터를 진짜로 인식할 확률에 로그를 취한 값의 평균
- $E_{z \sim p_z(z)} [\log(1 - D(G(z)))]$: Latent Vector의 분포 $p_z(z)$ 에서 샘플링한 노이즈 z 을 생성자 G 에 입력하여 생성된 데이터 $G(z)$ 에 대해 $1 - D(G(z))$ 에 로그를 취한 후, 기댓값 계산
=> 판별자가 생성된 데이터를 가짜로 인식할 확률에 로그를 취한 값의 평균
- 생성자가 생성한 데이터와 실제 데이터 간의 유사성은 Jensen-Shannon Divergence로 측정



GAN의 학습 Back Propagation of GAN

Generator

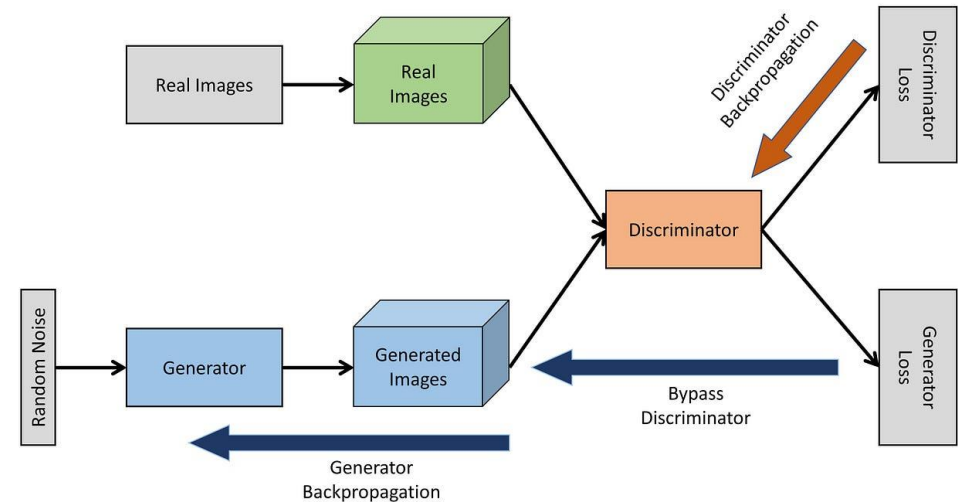
- 생성된 데이터 $G(z)$ 에 대해 $D(G(z))$ 를 최대화하여 판별자가 가짜 데이터를 진짜로 인식하도록 학습

Discriminator

- 실제 데이터 x 에 대해 $D(x)$ 를 최대화하여 판별자가 실제 데이터를 진짜로 인식하도록 학습
- 생성된 데이터 $G(z)$ 에 대해 $1 - D(G(z))$ 를 최대화하여 판별자가 가짜 데이터를 가짜로 인식하도록 학습

➤ 생성자와 판별자의 학습 횟수가 차이나는 이유

- 두 네트워크 간의 학습 균형을 맞추기 위함
- 판별자가 생성자보다 학습을 더 많이 수행한다고 해서 반드시 판별자가 우수해지는 것은 아님
- 오히려 판별자가 지나치게 강력해지면 생성자가 판별자를 속이기 어려워져 학습이 불안정해짐



GAN의 구조

KL Divergence

Kullback-Leibler Divergence

- 두 확률 분포 간의 차이를 측정하는 비대칭적인 척도
- 한 분포를 기준으로 다른 분포가 얼마나 다른지를 정량적으로 나타냄
- Relative Entropy라고도 함
- 이산적 KL Divergence에서,
 $\mathbb{KL}(p||q) = \sum_{k=1}^K p_k \log p_k - \sum_{k=1}^K p_k \log q_k$
 $\Rightarrow \mathbb{KL}(p||q) = (-\text{entropy of } p) - (\text{cross entropy of } p \& q)$ 이다.
- KLD는 비대칭: $\mathbb{KL}(p||q) \neq \mathbb{KL}(q||p)$
 $\Rightarrow$ 비대칭이기 때문에 유사도를 말할 때 거리라고 표현하지 않음
 $\Rightarrow$ 이를 대칭으로 바꾸고, 거리 개념을 도입: JSD

$$\mathbb{KL}(p||q) \triangleq \sum_{k=1}^K p_k \log \frac{p_k}{q_k}$$

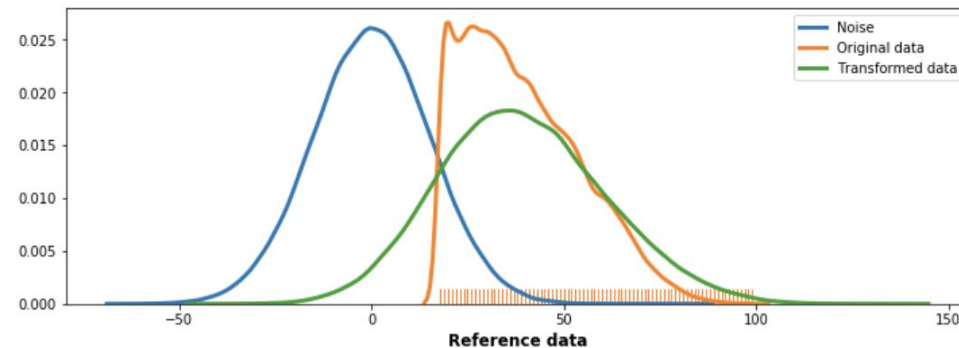
확률 분포가 이산적일때의
KL Divergence

$$\mathbb{KL}(p||q) \triangleq \int dx p(x) \log \frac{p(x)}{q(x)}$$

확률 분포가 연속적일때의
KL Divergence

JL Divergence Jensen-Shannon Divergence

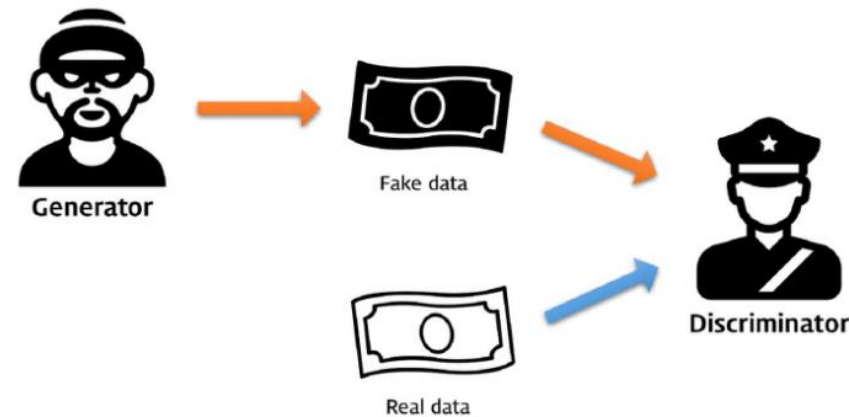
- 두 확률 분포 간의 차이를 측정하는 대칭적인 척도
KLD의 비대칭성 보완
- 두 분포의 평균 분포를 계산한 후, 해당 평균 분포와
각 분포와의 KLD를 계산하고, 그 평균을 구하는 방식으로 정의
- $$\mathbb{JS}(p, q) = \frac{1}{2}\mathbb{KL}(p||m) + \frac{1}{2}\mathbb{KL}(q||m), \quad m = \frac{1}{2}(p + q)$$
- JSD는 항상 유한한 값을 가지며, 두 분포가 동일할 경우 JSD는 0이 됨
- GAN에서, 생성자가 생성한 데이터의 분포와 실제 데이터의 분포 간의 차이를 JSD로 정의
JSD는 두 분포 사이의 “거리”를 나타내는 지표로 활용됨



JSD의 평균 분포(초록색)

GAN의 문제점 Problems of GAN

- 생성자가 다양한 데이터 분포를 학습하지 못하고, 특정 패턴이나 모드에만 집중하여 유사한 샘플을 반복적으로 생성하는 Mode Collapse 현상
- 생성자와 판별자가 경쟁적으로 학습하는 과정에서 학습이 불안정해지거나 수렴하지 않을 수 있음
- 판별자가 너무 강력해져서 생성자가 학습할 수 있는 유의미한 기울기를 얻지 못할 수 있음
- 생성된 샘플의 품질을 객관적으로 평가하기 위한 지표가 부족하여, 모델의 성능을 정량적으로 비교하거나 측정하는 데 어려움이 따름
- MLP 구조를 기반으로 설계되어, 이미지와 같은 고차원 데이터의 공간적 특성을 효과적으로 처리하는 데 한계가 있음



GAN의 구조

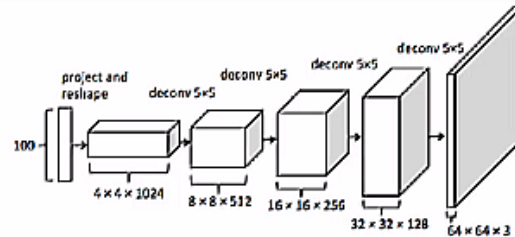
01 GAN Generative Adversarial Networks

DCGAN Deep Convolutional Generative Adversarial Networks

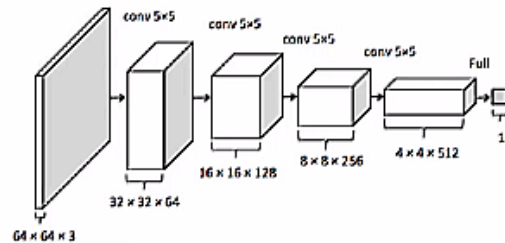
- 생성자와 판별자 모두에 Convolutional Layer 적용
- 생성자와 판별자의 각 층에 Batch Normalization 적용
- Pooling 대신 strided convolution이나 fractional-strided convolution을 사용하여 Feature map의 크기 조절
- 판별자의 활성화 함수로 Leaky ReLU 사용
- 기존 GAN 대비 Mode Collapse 현상 완화
- 그러나 기존 GAN과 유사한 문제점들 가짐

DCGAN Overall

Generator



Discriminator

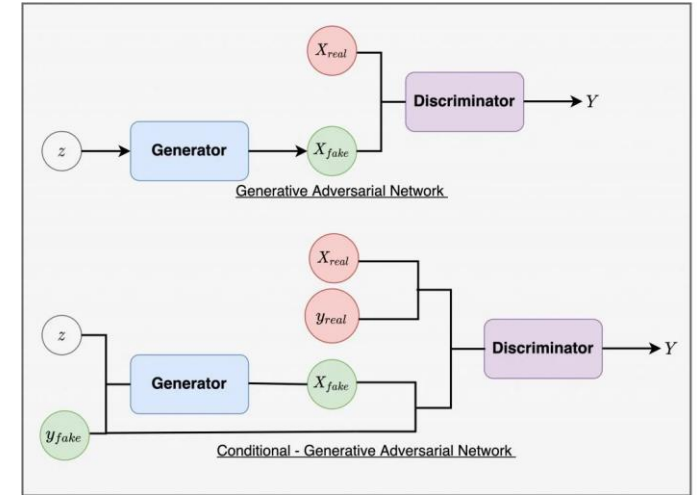


DCGAN의 구조

01 GAN Generative Adversarial Networks

CGAN Conditional Generative Adversarial Networks

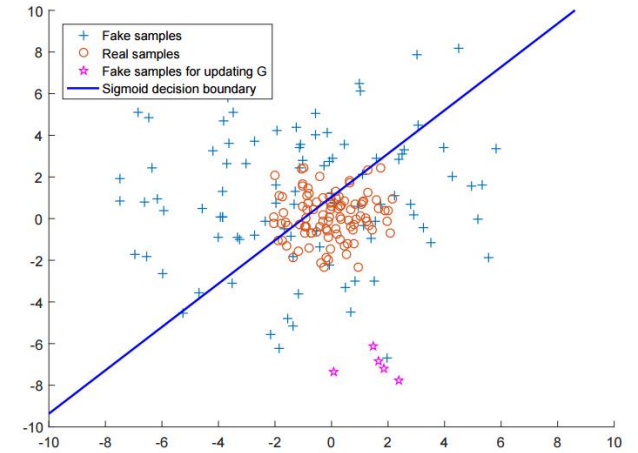
- 조건 정보를 추가하여 원하는 특성을 가진 데이터를 생성할 수 있게 함
- 생성자와 판별자 모두 조건 정보 y 를 추가로 받음
- $V(D, G) = E_{x \sim p_{data}(x)}[\log D(x|y)] + E_{z \sim p_z(z)}[\log(1 - D(G(z|y)|y))]$ 로 정의됨
($D(x|y)$ 는 조건 y 에 대해 데이터 x 가 실제일 확률)
- 기존 GAN의 단점에 더불어, 조건 정보 y 의 품질과 정확도에 의존됨



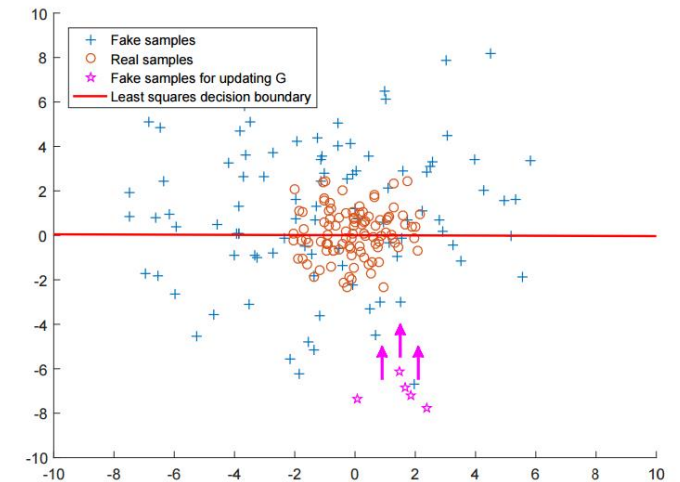
CGAN의 구조 (밑)

LSGAN Least Squares Generative Adversarial Networks

- 기존의 GAN은 판별자의 손실 함수로 CEE를 사용하나, 이는 학습 중 Gradient Vanishing 문제를 일으킬 수 있음 (생성자가 판별자를 속이는 단계가 되면, 딱히 더 학습할 거리가 없으므로)
- 생성자가 생성한 데이터가 실제 데이터와 얼마나 가까운지 정량적으로 평가 => 판별자가 생성된 데이터에 대해 더 강한 피드백 제공 가능
- Discriminator: $L_D = \frac{1}{2} E_{x \sim p_{data}(x)} [(D(x) - b)^2] + \frac{1}{2} E_{z \sim p_z(z)} [(D(G(z)) - a)^2]$
a, b: 가짜 데이터와 실제 데이터에 대한 레이블 값
Generator: $L_G = \frac{1}{2} E_{z \sim p_z(z)} [(D(G(z)) - c)^2]$
c: 생성자가 판별자를 속이기 위해 목표로 하는 값
- 기존 GAN에 비해 더 안정적인 학습



별표 샘플들이 진짜 샘플들과 멀리 떨어진 모습



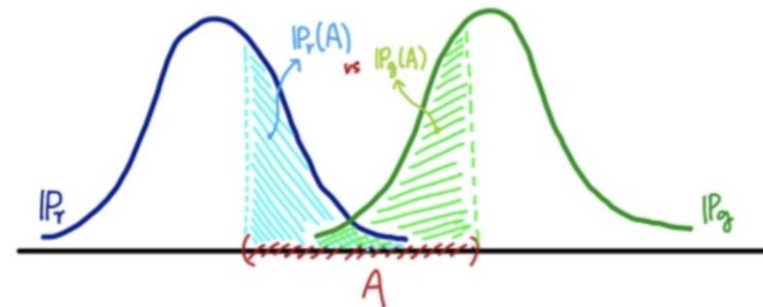
멀리 떨어진 샘플들에 페널티를 부과하여 더 진짜에 가까운 샘플 생성

거리의 종류 Distances

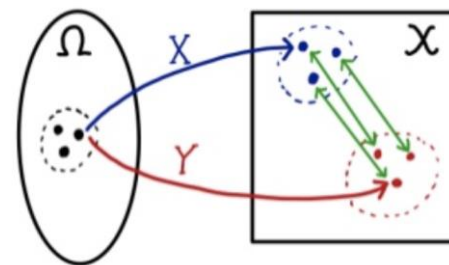
- Total Variation(TV): 두 확률 분포의 측정값이 벌어질 수 있는 가장 큰 값
- Kullback-Leibler Divergence(KLD): 한 분포를 기준으로 다른 분포가 얼마나 다른지 정량적으로 나타냄
- Jensen-Shannon Divergence(JSD): KLD 기반으로, 평균 분포와 두 분포가 얼마나 다른지 정량적으로 나타냄
- Earth-Mover Distance(EMD): 두 확률분포의 결합확률분포로 정의된 분포에서, 두 분포 사이의 길이의 기댓값 중 가장 작은 값

$W(\mathbb{P}_r, \mathbb{P}_g) = \inf_{\gamma \in \Pi(\mathbb{P}_r, \mathbb{P}_g)} \mathbb{E}_{(x,y) \sim \gamma} [\|x - y\|]$ 로 정의됨

$$\delta(\mathbb{P}_r, \mathbb{P}_g) = \sup_{A \in \Sigma} |\mathbb{P}_r(A) - \mathbb{P}_g(A)|$$



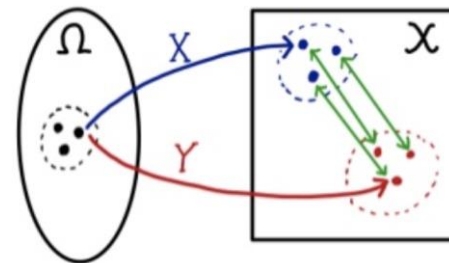
<Total Variation>
구간 A 내의 값들을 대입했을 때,
 $\mathbb{P}_r(A)$ 와 $\mathbb{P}_g(A)$ 의 차이 중 가장 큰 값



<Earth-Mover Distance>
빨간 분포와 파란 분포의 결합확률분포
 $\Pi(\mathbb{P}_r, \mathbb{P}_g)$ 에서 각 분포간 거리의 기댓값의 최소값

EMD Earth-Mover Distance / Wasserstein Distance

- $W(\mathbb{P}_r, \mathbb{P}_g) = \inf_{\gamma \in \Pi(\mathbb{P}_r, \mathbb{P}_g)} \mathbb{E}_{(x,y) \sim \gamma} [\|x - y\|]$
- $\Pi(\mathbb{P}_r, \mathbb{P}_g)$: $\mathbb{P}_r, \mathbb{P}_g$ 라는 분포들을 주변 분포로 가지는 특정한 결합 분포들의 집합
=> $\mathbb{P}_r, \mathbb{P}_g$ 를 한 쌍으로 연결하는 모든 가능한 방법
 $\gamma \in \Pi(\mathbb{P}_r, \mathbb{P}_g)$: 위의 모든 가능한 분포 중 특정한 결합 분포 γ 를 나타냄
 $\mathbb{E}_{(x,y) \sim \gamma} [\|x - y\|]$: 결합 분포 γ 에 따라 샘플링 된 쌍 (x, y) 에 대해, x 와 y 사이의 거리 $\|x - y\|$ 의 기댓값을 나타냄
 $\inf$: 어떤 집합의 하한, 즉 집합의 모든 원소보다 작거나 같은 값 중 가장 큰 값
- $\mathbb{P}_r$ 이라는 분포에서 $\mathbb{P}_g$ 라는 분포로 분포를 이동시키는 데 필요한 최소한의 비용 의미
(비용은 x 와 y 사이의 거리로 정의됨)
- 해당 비용을 최소화하는 결합 분포 γ 를 찾는 과정
- $W(\mathbb{P}_r, \mathbb{P}_g) = \inf_{\gamma \in \Pi(\mathbb{P}_r, \mathbb{P}_g)} \mathbb{E}_{(x,y) \sim \gamma} [\|x - y\|]$ 이러한 Wasserstein 거리는
실제 데이터 분포 $\mathbb{P}_r$ 와 생성된 데이터 분포 $\mathbb{P}_g$ 의 결합 분포를 알기 어렵기 때문에 직접 계산이 어려움



<Earth-Mover Distance>
빨간 분포와 파란 분포의 결합확률분포
 $\Pi(\mathbb{P}_r, \mathbb{P}_g)$ 에서 각 분포간 거리의 기댓값의 최소값

EMD Earth-Mover Distance / Wasserstein Distance

Example 1 (Learning parallel lines). Let $Z \sim U[0, 1]$ the uniform distribution on the unit interval. Let $\mathbb{P}_0$ be the distribution of $(0, Z) \in \mathbb{R}^2$ (a 0 on the x-axis and the random variable Z on the y-axis), uniform on a straight vertical line passing through the origin. Now let $g_\theta(z) = (\theta, z)$ with θ a single real parameter. It is easy to see that in this case,

- $W(\mathbb{P}_0, \mathbb{P}_\theta) = |\theta|,$
- $JS(\mathbb{P}_0, \mathbb{P}_\theta) = \begin{cases} \log 2 & \text{if } \theta \neq 0, \\ 0 & \text{if } \theta = 0, \end{cases}$
- $KL(\mathbb{P}_\theta \| \mathbb{P}_0) = KL(\mathbb{P}_0 \| \mathbb{P}_\theta) = \begin{cases} +\infty & \text{if } \theta \neq 0, \\ 0 & \text{if } \theta = 0, \end{cases}$
- and $\delta(\mathbb{P}_0, \mathbb{P}_\theta) = \begin{cases} 1 & \text{if } \theta \neq 0, \\ 0 & \text{if } \theta = 0. \end{cases}$

When $\theta_t \rightarrow 0$, the sequence $(\mathbb{P}_{\theta_t})_{t \in \mathbb{N}}$ converges to $\mathbb{P}_0$ under the EM distance, but does not converge at all under either the JS, KL, reverse KL, or TV divergences. Figure 1 illustrates this for the case of the EM and JS distances.

θ 는 생성자(Generator)의 파라미터를 나타낸다. 생성자는 이 파라미터 θ 를 조정하여 데이터 분포 P_θ 를 학습한다.

실제 데이터 분포 P_0 와 생성자의 파라미터 θ 에 의해 생성된 데이터 분포 P_θ 에 대해 TV, KLD, SJD는 두 분포가 서로 겹칠때는 0, 겹치지 않을 때에는 무한대 또는 상수로 나타난다. 이는 학습 과정에서 Gradient Vanishing 문제를 야기할 수 있다.

그러나 EMD에서는 거리가 분포가 겹치던 겹치지 않던 그대로 파라미터의 크기로 나오므로, 학습 과정 중 분포를 학습할 때 더 효과적인 피드백을 제공한다.

Lipschitz

- 두 거리 공간 (X, d_X) 와 (Y, d_Y) 사이의 함수 $f: X \rightarrow Y$ 가 아래를 만족하면, f 를 Lipschitz 함수라고 한다.
 $d_Y(f(x_1), f(x_2)) \leq K \times d_X(x_1, x_2), \forall x_1, x_2 \in X$
 K 는 Lipschitz 상수로, 함수의 변화율을 제한하는 역할을 한다.
- 즉, 입력 값의 변화에 따른 출력 값의 변화가 K 배를 넘지 않도록 한다.
- 만약 $K=1$ 인 경우, 이러한 함수를 1-Lipschitz Continuous Function이라 하며, 이런 함수들의 집합을 1-Lipschitz Continuous Function Space라 한다.
해당 공간은 함수의 변화율이 1로 제한되어 있으며, 함수의 안정성과 연속성을 보장한다.

Kantorovich-Rubinstein Duality

- 만약 Wasserstein 거리를 직접 계산하기 어려운 경우, 계산을 용이하게 하는 정리이며, Wasserstein 거리를 1-Lipschitz Continuous Function Space 에서의 문제로 변환한다.
- 두 확률 분포 $\mathbb{P}_r, \mathbb{P}_g$ 에 대해 1-Wasserstein 거리는 아래와 같다.

$$W(\mathbb{P}_r, \mathbb{P}_g) = \inf_{\gamma \in \Pi(\mathbb{P}_r, \mathbb{P}_g)} \mathbb{E}_{(x,y) \sim \gamma} [|x - y|]$$

그런데 $\mathbb{P}_r$ 은 우리가 알고자 하는 대상이므로 식을 계산할 수 없다.

따라서 Kantorovich-Rubinstein Duality를 사용하면 1-Wasserstein 거리는 아래와 같이 변환된다.

$$W(\mathbb{P}_r, \mathbb{P}_g) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{x \sim \mathbb{P}_r} [f(x)] - \mathbb{E}_{x \sim \mathbb{P}_g} [f(x)]$$

이 때, $\sup_{\|f\|_L \leq 1}$ 은 f 가 1-Lipschitz 조건을 만족한다는 뜻이다.

01 GAN Generative Adversarial Networks

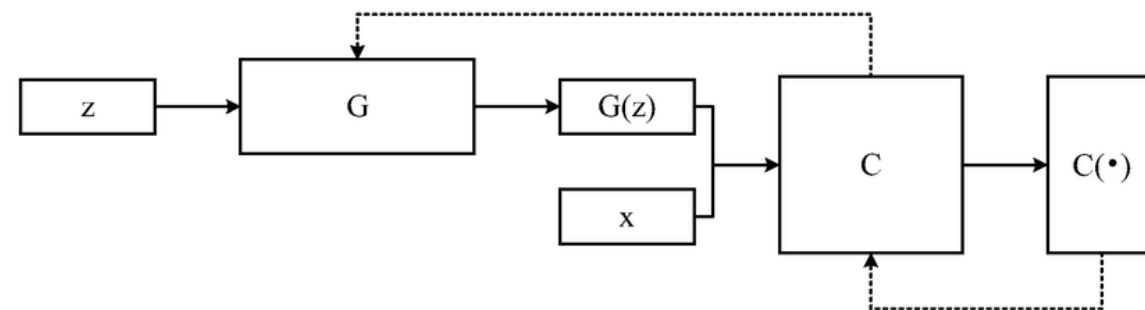
WGAN Wasserstein Generative Adversarial Networks

$$- W(\mathbb{P}_r, \mathbb{P}_g) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{x \sim \mathbb{P}_r} [f(x)] - \mathbb{E}_{x \sim \mathbb{P}_g} [f(x)]$$

해당 식을 WGAN의 학습을 위한 Loss로 사용하기 위해 Parameter가 추가된 f로 바꾸면 아래와 같다.

$$\max_{\omega \in W} \mathbb{E}_{x \sim \mathbb{P}_r} [f_{\omega}(x)] - \mathbb{E}_{z \sim p(z)} [f_{\omega}(g_{\theta}(z))]$$

아직 구해야 하는 목표값인 $\mathbb{P}_r$ 이 있지만, Gradient Update를 위해 θ 에 대해 편미분을 수행하면 $\mathbb{P}_r$ 이 사라지기 때문에 학습에는 문제가 없다.



WGAN의 구조

	Discriminator/Critic	Generator
GAN	$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m [\log D(x^{(i)}) + \log (1 - D(G(z^{(i)})))]$	$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (D(G(z^{(i)})))$
WGAN	$\nabla_w \frac{1}{m} \sum_{i=1}^m [f(x^{(i)}) - f(G(z^{(i)}))]$	$\nabla_{\theta} \frac{1}{m} \sum_{i=1}^m f(G(z^{(i)}))$

WGAN에서는 보통 Discriminator가 아닌 Critic으로 부름

WGAN Wasserstein Generative Adversarial Networks

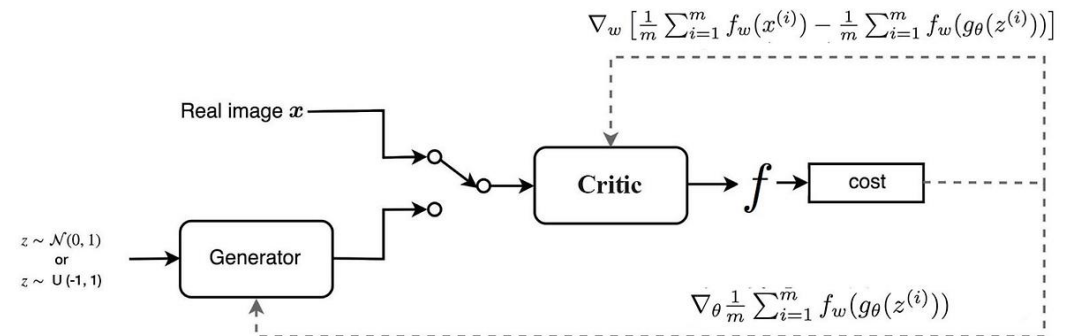
Algorithm 1 WGAN, our proposed algorithm. All experiments in the paper used the default values $\alpha = 0.00005$, $c = 0.01$, $m = 64$, $n_{\text{critic}} = 5$.

Require: : α , the learning rate. c , the clipping parameter. m , the batch size. n_{critic} , the number of iterations of the critic per generator iteration.

Require: : w_0 , initial critic parameters. θ_0 , initial generator's parameters.

```
1: while  $\theta$  has not converged do
2:   for  $t = 0, \dots, n_{\text{critic}}$  do
3:     Sample  $\{x^{(i)}\}_{i=1}^m \sim \mathbb{P}_r$  a batch from the real data.
4:     Sample  $\{z^{(i)}\}_{i=1}^m \sim p(z)$  a batch of prior samples.
5:      $g_w \leftarrow \nabla_w [\frac{1}{m} \sum_{i=1}^m f_w(x^{(i)}) - \frac{1}{m} \sum_{i=1}^m f_w(g_\theta(z^{(i)}))]$ 
6:      $w \leftarrow w + \alpha \cdot \text{RMSPProp}(w, g_w)$ 
7:      $w \leftarrow \text{clip}(w, -c, c)$ 
8:   end for
9:   Sample  $\{z^{(i)}\}_{i=1}^m \sim p(z)$  a batch of prior samples.
10:   $g_\theta \leftarrow -\nabla_\theta \frac{1}{m} \sum_{i=1}^m f_w(g_\theta(z^{(i)}))$ 
11:   $\theta \leftarrow \theta - \alpha \cdot \text{RMSPProp}(\theta, g_\theta)$ 
12: end while
```

- WGAN의 최종적인 학습 알고리즘



<WGAN의 학습>

WGAN의 학습

Critic

- 실제 데이터 분포 $\mathbb{P}_r$ 와 생성된 데이터의 분포 $\mathbb{P}_g$ 각각 미니배치 샘플링
- Critic의 손실 함수 $L_D = \mathbb{E}_{x \sim \mathbb{P}_r} [D(x)] - \mathbb{E}_{z \sim p(z)} [D(G(z))]$
(D는 판별자(Critic), z 는 잠재 변수를 의미)
- 판별자의 파라미터(w)를 RMSPProp을 이용하여 업데이트

Generator

- Generator의 손실 함수 $L_G = -\mathbb{E}_{z \sim p(z)} [D(G(z))]$
- 생성자의 파라미터(θ)를 RMSPProp을 이용하여 업데이트

위의 과정을 반복하여 생성자와 판별자를 교대로 학습

01 GAN Generative Adversarial Networks

WGAN Wasserstein Generative Adversarial Networks

- Gradient Clipping?

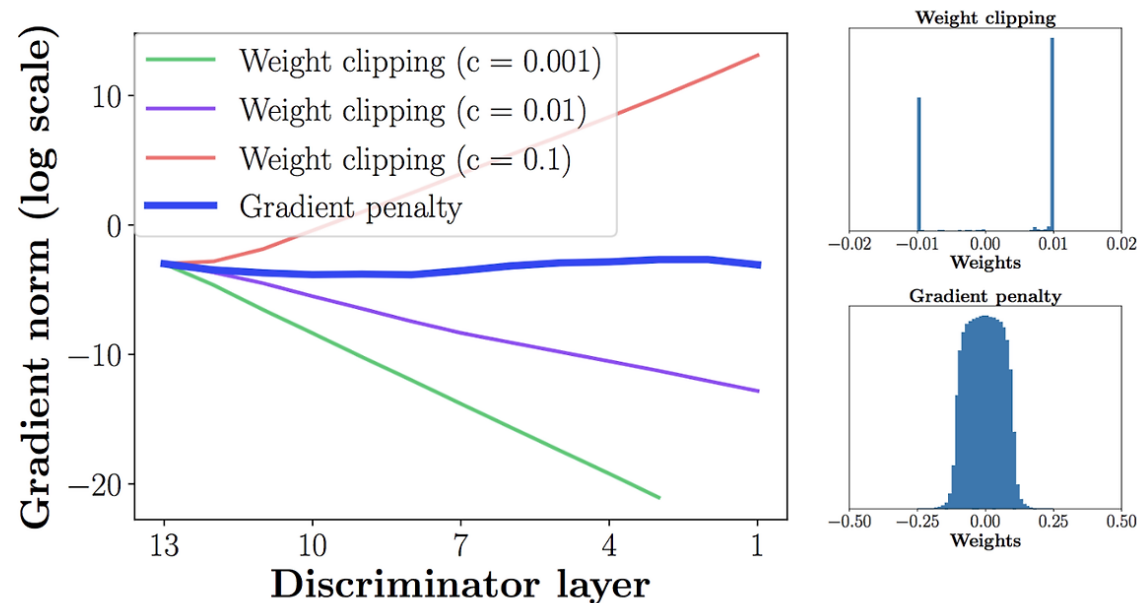
Gradient Explosion을 방지하기 위해 기울기의 크기를 제한
Critic이 1-Lipschitz Continuity를 만족하기 위해 사용됨
만약 c 값이 0.01이면 가중치를 $[-0.01, 0.01]$ 로 제한함

- Adam 대신 RMSProp을 사용하는 이유

adam과 같은 Momentum 기반의 Optimizer는
Critic의 Lipschitz 조건을 만족하는 데 어려움을 겪을 수 있기 때문에,
WGAN에서는 Momentum을 사용하지 않는 RMSProp 등의 Optimizer를 권장

- Critic과 Generator의 학습 횟수가 다른 이유

WGAN에서는 Critic이 Wasserstein 거리를 정확하게 추정하기 위해 충분히 학습되어야 하므로,
Critic을 여러 번 학습한 후 Generator를 한 번 학습
=> Critic이 Generator의 업데이트에 비해 더 정확한 피드백을 제공하도록 하여 학습의 안정성을 높임



Weight Clipping Parameter c 에 따른
Gradient의 변화

02

실습

GAN with MNIST

실습 #1) GAN을 이용한 MNIST 데이터 생성

- Functional API를 사용하여 설계
- MNIST 데이터에 대해 GAN 학습 진행
- GAN이 해결해야 될 문제 분석

(일반적인 GAN 모델은 편의상 Vanilla GAN, VGAN이라고 부르겠습니다.)

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras import layers, Input, Model

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
x_train = x_train / 127.5 - 1
x_test = x_test / 127.5 - 1
x_train = x_train.reshape(len(x_train), 784)
x_test = x_test.reshape(len(x_test), 784)

noise_dimension = 10
batch_size = 64
iterations = 10000
```

필요한 라이브러리를 임포트한다.

MNIST 데이터를 불러오고 정규화하는데, 이 때 -1~1의 값으로 정규화한다.

MNIST는 28by28의 크기이므로, 이를 784차원의 벡터로 변환한다.

Generator의 입력으로 사용될 노이즈 벡터는 10차원으로 한다.

```
def build_generator():
    inputs = Input(shape=(noise_dimension,), name='generator_input')
    x = layers.Dense(256, activation='relu', name='gen_dense_1')(inputs)
    x = layers.Dense(512, activation='relu', name='gen_dense_2')(x)
    x = layers.Dense(1024, activation='relu', name='gen_dense_3')(x)
    outputs = layers.Dense(784, activation='tanh', name='generator_output')(x)
    model = Model(inputs, outputs, name='generator')
    return model
```

Generator의 정의이다.

출력은 -1~1의 범위에 있어야 하기 때문에, 활성화함수로 hyperbolic tangent를 사용했다.

```
def build_discriminator():
    inputs = Input(shape=(784,), name='discriminator_input')
    x = layers.Dense(1024, activation='relu', name='dis_dense_1')(inputs)
    x = layers.Dropout(0.3)(x)
    x = layers.Dense(512, activation='relu', name='dis_dense_2')(x)
    x = layers.Dropout(0.3)(x)
    x = layers.Dense(256, activation='relu', name='dis_dense_3')(x)
    x = layers.Dropout(0.3)(x)
    outputs = layers.Dense(1, activation='sigmoid', name='discriminator_output')(x)
    model = Model(inputs, outputs, name='discriminator')
    return model
```

Discriminator의 정의이다.

중간중간의 드롭아웃을 이용하여 Overfitting을 방지하고, 출력층에 sigmoid를 사용하여 출력의 범위를 0~1로 제한한다.

```
generator = build_generator()
generator.summary()
discriminator = build_discriminator()
discriminator.summary()

discriminator.compile(loss='binary_crossentropy', optimizer='adam')
discriminator.trainable = False
gan_input = Input(shape=(noise_dimension,))
fake_image = generator(gan_input)
gan_output = discriminator(fake_image)
gan = Model(gan_input, gan_output, name='gan')
gan.compile(loss='binary_crossentropy', optimizer='adam')
```

Generator와 Discriminator를 생성하고, 각 모델의 구조를 출력한다.

Discriminator의 학습 가능성을 False로 설정하는데, 나중에 함수를 출력할때만 일시적으로 True로 바꿔주기 때문에 이렇게 해도 무방하다.

생성된 Generator와 Discriminator를 연결하여 GAN을 구성하고, binary CEE, adam으로 컴파일한다.

Loss로 일반적인 CEE여도 학습이 잘 되는 이유

- MNIST는 0부터 9까지의 손글씨 숫자 이미지로 구성된 비교적 단순한 데이터셋이다. 이러한 단순한 데이터셋에서는 기본적인 손실 함수와 네트워크 구조로도 효과적인 학습이 가능하기 때문이다.
- 생성자와 판별자 모두 다층 퍼셉트론(MLP) 구조를 사용하고 있다. 이러한 구조는 MNIST와 같은 단순한 데이터셋을 처리하는 데 충분하기 때문이다.
- 입력 데이터를 $[-1, 1]$ 범위로 정규화하여 신경망이 효율적으로 학습할 수 있도록 하였기 때문이다.

```
def train_discriminator(x_train):
    discriminator.trainable = True
    # Real 학습
    c = np.random.randint(0, x_train.shape[0], batch_size)
    real = x_train[c]
    d_loss_real = discriminator.train_on_batch(real, np.ones((batch_size, 1)))
    # Fake 학습
    p = np.random.normal(0, 1, (batch_size, noise_dimension))
    fake = generator.predict(p)
    d_loss_fake = discriminator.train_on_batch(fake, np.zeros((batch_size, 1)))
    return d_loss_real, d_loss_fake
```

Discriminator를 학습시키는 함수이다.

강의자료와 약간 다르게 개조하였는데, 실제 이미지를 학습할 때와 가짜 이미지를 학습할 때의 손실을 반환하여 학습의 추적을 더 쉽게 하도록 하였다. 기본적으로 discriminator.trainable는 False값인데, 이 함수를 실행할때만 임시로 True로 하여 학습이 가능하도록 하였다.

```
def train_generator():
    discriminator.trainable = False
    p = np.random.normal(0, 1, (batch_size, noise_dimension))
    g_loss = gan.train_on_batch(p, np.ones((batch_size, 1)))
    return g_loss
```

Generator를 학습시키는 함수이다.

위 함수와 동일하게 손실을 추적할 수 있게 개조하였다.

```
save_path = "path\\Vanilla GAN Architecture.png"
from tensorflow.keras.utils import plot_model
plot_model(gan, to_file=save_path, show_shapes=True, show_layer_names=True)
```

생성된 GAN 모델의 전체적인 구조를 시각화하는 부분이다.

```
d_losses = []
g_losses = []
d_losses_real = []
d_losses_fake = []

for i in range(iterations + 1):
    d_loss_real, d_loss_fake = train_discriminator(x_train)
    d_loss = 0.5 * np.add(d_loss_real, d_loss_fake)
    g_loss = train_generator()
    print(f"Iteration {i} [D loss: {d_loss:.4f}] [G loss: {g_loss:.4f}] (Real: {d_loss_real:4f}, Fake: {d_loss_fake:4f})")
    d_losses.append(d_loss)
    g_losses.append(g_loss)
    d_losses_real.append(d_loss_real)
    d_losses_fake.append(d_loss_fake)

    if i % 1000 == 0:
        plt.figure(figsize=(20, 4))
        plt.suptitle(f'Iteration {i}')
        for k in range(20):
            plt.subplot(2, 10, k + 1)
            image = generator.predict(np.random.normal(0, 1, (1, noise_dimension)))
            plt.imshow(image[0].reshape(28, 28), cmap='gray')
            plt.axis('off')
            plt.xticks([])
            plt.yticks([])
        plt.show()

plt.figure(figsize=(10, 5))
plt.plot(d_losses, label='Discriminator Loss')
plt.plot(g_losses, label='Generator Loss')
plt.plot(d_losses_real, label='Discriminator Real Loss', linestyle='dotted')
plt.plot(d_losses_fake, label='Discriminator Fake Loss', linestyle='dotted')
plt.title('Training Losses')
plt.xlabel('Iteration')
plt.ylabel('Loss')
plt.yscale('log')
plt.grid()
plt.legend()
plt.show()
```

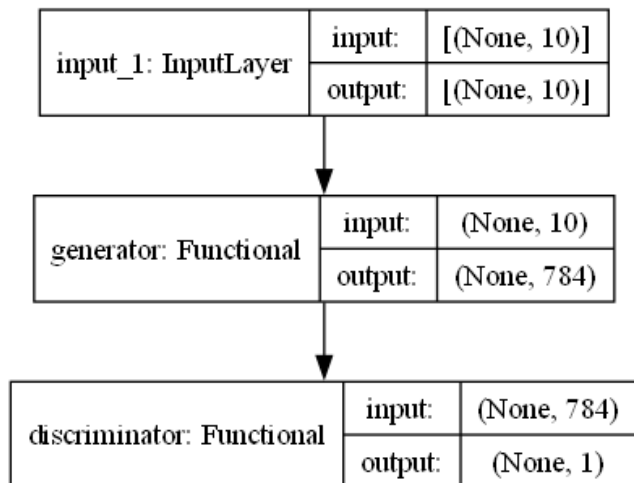
최종적으로 학습 및 시각화를 진행하는 코드이다.

작아서 잘 안보일 수 있는데, GAN 모델을 학습하고 loss를 출력 후, 1000번의 iteration마다 생성된 이미지를 출력한다.

이후, 생성자의 loss와 판별자의 loss를 그래프로 나타낸다.

Model: "generator"

Layer (type)	Output Shape	Param #
generator_input (InputLayer)	[(None, 10)]	0
gen_dense_1 (Dense)	(None, 256)	2816
gen_dense_2 (Dense)	(None, 512)	131584
gen_dense_3 (Dense)	(None, 1024)	525312
generator_output (Dense)	(None, 784)	803600
Total params: 1,463,312		
Trainable params: 1,463,312		
Non-trainable params: 0		

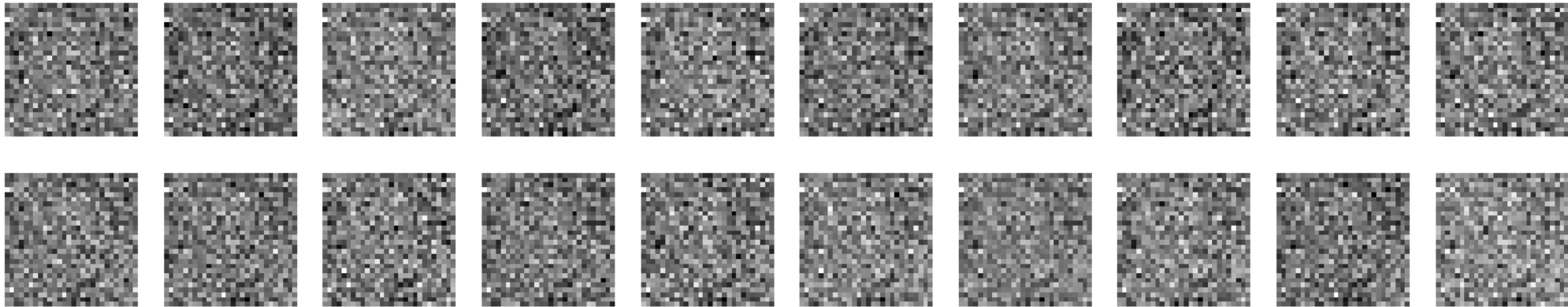


Model Architecture

Model: "discriminator"

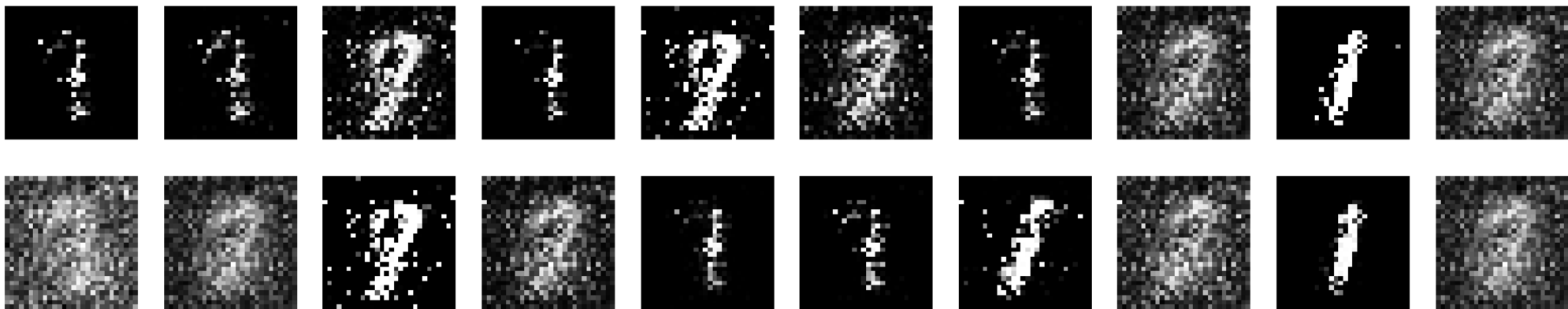
Layer (type)	Output Shape	Param #
discriminator_input (InputLa	[(None, 784)]	0
dis_dense_1 (Dense)	(None, 1024)	803840
dropout (Dropout)	(None, 1024)	0
dis_dense_2 (Dense)	(None, 512)	524800
dropout_1 (Dropout)	(None, 512)	0
dis_dense_3 (Dense)	(None, 256)	131328
dropout_2 (Dropout)	(None, 256)	0
discriminator_output (Dense)	(None, 1)	257
Total params: 1,460,225		
Trainable params: 1,460,225		
Non-trainable params: 0		

Iteration 0



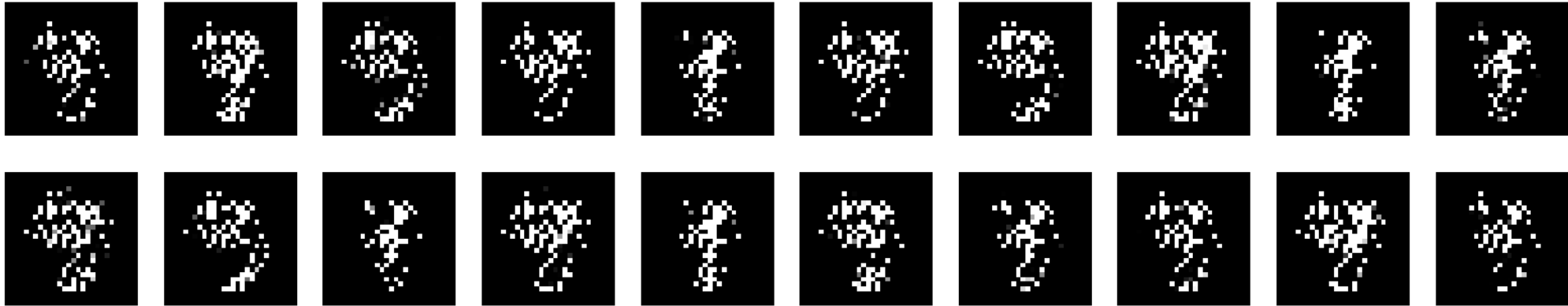
GAN의 학습 전, 무작위 노이즈가 입력된 모습

Iteration 1000



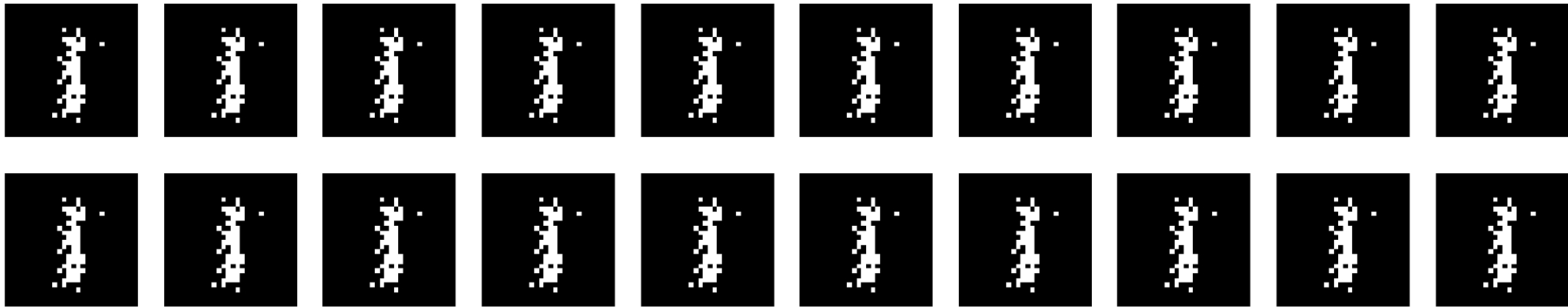
Iteration이 1000번 진행된 후이다.
뭔가 MNIST의 숫자 모습을 잡아가고 있는걸 볼 수 있다.

Iteration 2000



조금 더 학습하자, 배경에 노이즈가 사라져 완전 검은색이 되었으며, 숫자의 형상을 잡아가고 있다.

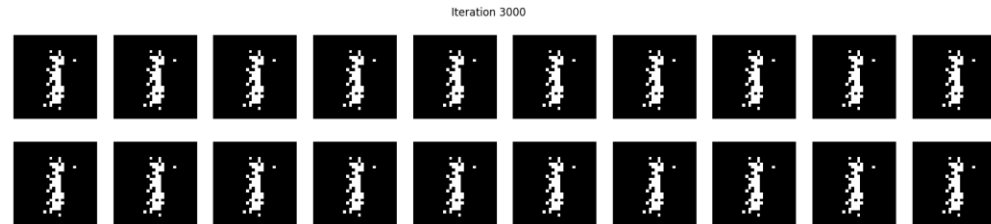
Iteration 3000



1에 대한 이미지만 생성하고 있다.
앞서 살펴보았던 Mode Collapse가 일어났기 때문이라고 추측된다.

모드 붕괴 Mode Collapse

- 생성자가 다양한 데이터 분포를 학습하지 못하고, 특정 패턴이나 모드에만 집중하여 유사한 샘플을 반복적으로 생성하는 현상
- 생성자와 판별자가 균형 있게 학습되지 않는 경우, 생성자가 판별자를 속이기 위해 특정한 패턴의 샘플(자기가 가장 잘 속일 수 있는 패턴)만 생성하게 되어 다양성 감소
- 코드에서 사용한 binary CEE로는 생성자의 출력 다양성을 충분히 보장하지 못할 수 있음



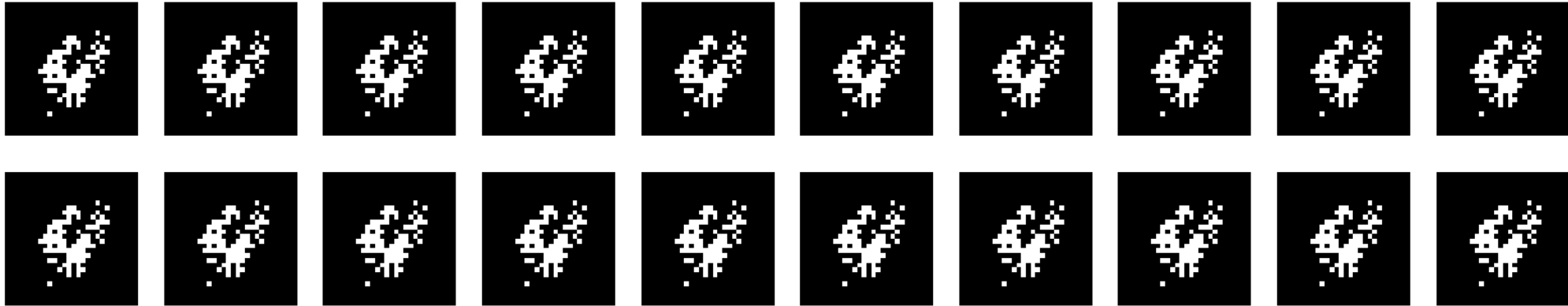
Iteration = 3000에서
Mode Collapse가 일어난 모습

➤ Wasserstein 거리를 활용하여 손실 함수를 개선함으로써
모드 붕괴를 줄이고 학습의 안정성을 높임

해결책

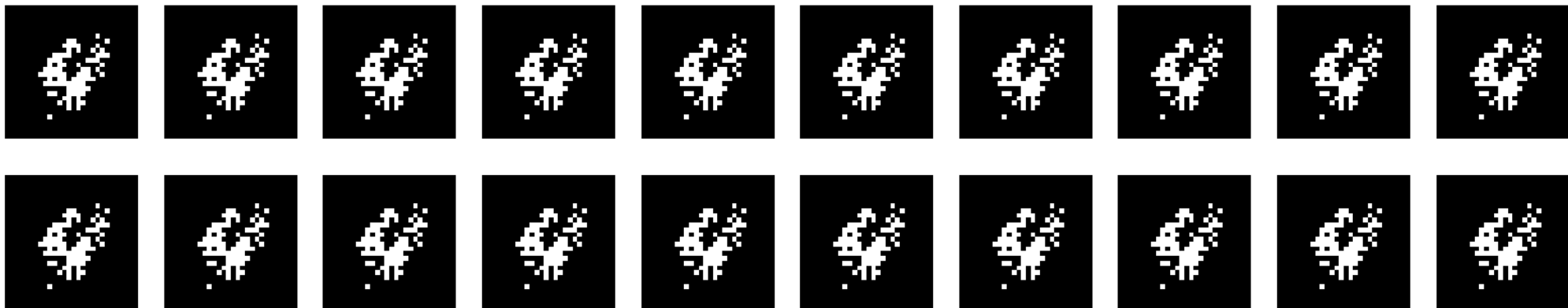
➤ 생성자와 판별자의 학습 비율을 조정하거나, 다양한 노이즈를 입력으로 사용하는 등
다양한 학습 기법을 도입하여 모드 붕괴 완화

Iteration 4000



모델 붕괴로 인하여 정상적인 생성을 못하는 모습

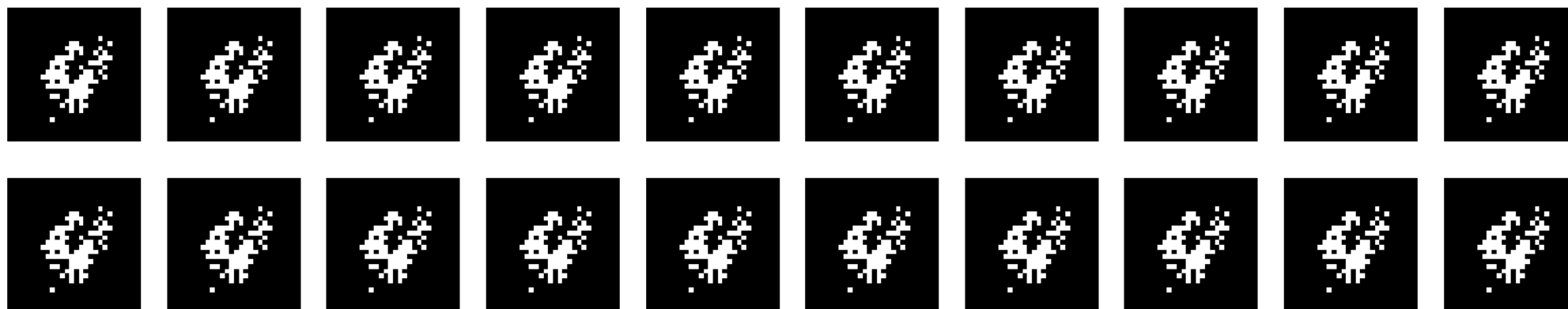
Iteration 5000



Iteration 6000



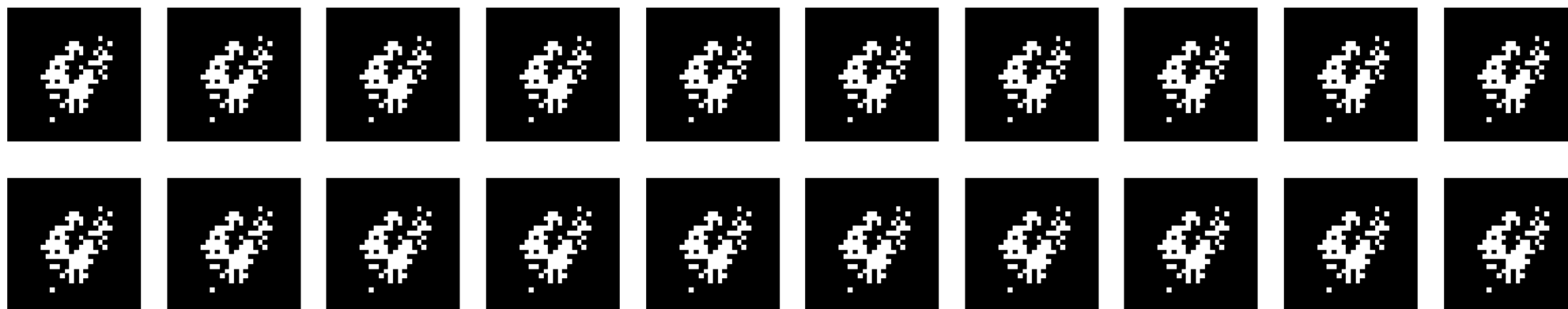
Iteration 7000

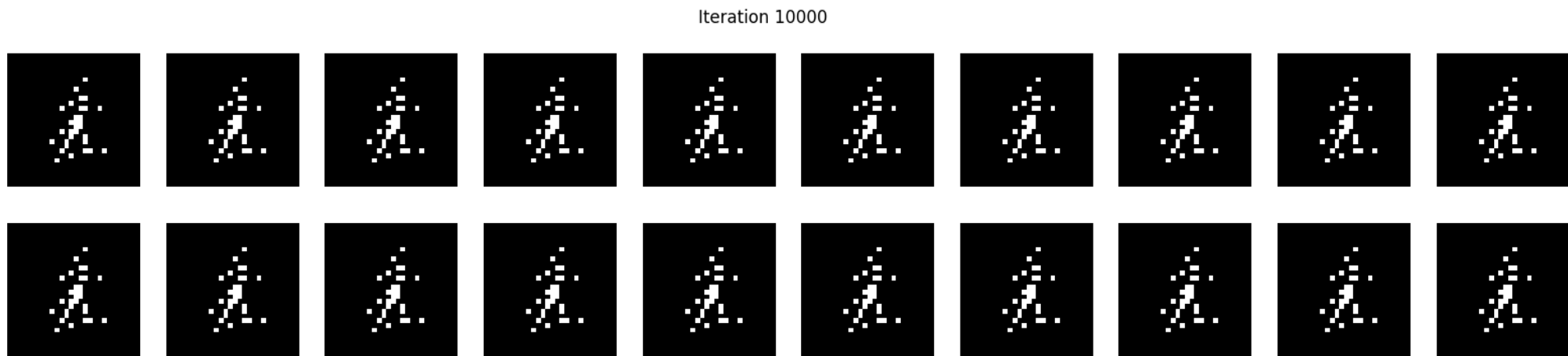


Iteration 8000

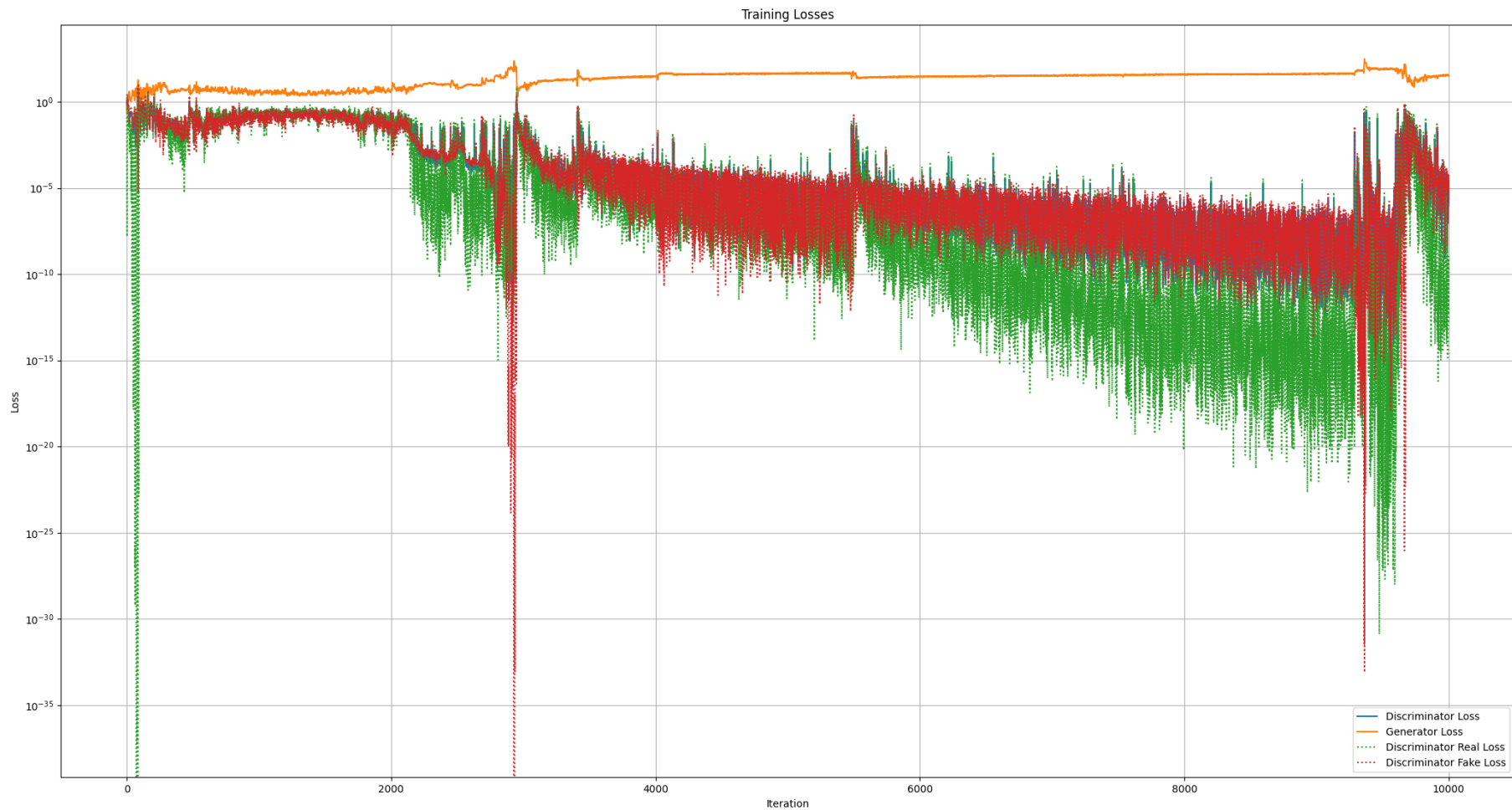


Iteration 9000





끝까지 모델 붕괴로 인하여 정상적인 이미지를 생성하지 못한 모습이다.



그래프에서 나타나듯, fake loss가 3000 iteration 부근에서 거의 0이 된 적이 있고, 이는 판별자가 생성자의 출력을 완전하게 가짜라고 인식했음을 뜻한다. 이로 인해 생성자는 특정 패턴에 집중하게 되고, 실제로 loss의 변동폭이 크지 않은 iteration 1000번 부근은 비교적 정상적으로 이미지를 생성했으나, loss의 변동폭이 급격히 커지는 iteration 2000년대부터 뭔가 이미지가 이상해지더니, fake loss가 0을 찍은 iteration 3000번 후 부터는 완전히 모드 붕괴가 일어났다.

실습 #2) 구조 변경 및 파라미터 조정에 따른 개선 및 비교분석

- Functional API를 사용하여 설계
- MNIST 데이터에 대해 GAN 학습 진행
- 다양한 방법으로 계층 변경하며 모델 개선
- 구조 개선을 통한 MNIST 이미지 생성
ex) Convolutional layer, Batch_normalization 추가 및 구조 변경 등등
- AE vs VAE vs GAN의 결과 비교분석 및 모델 구조에 따른 결과의 차이 분석

Deep Convolutional Generative Adversarial Networks (DCGAN)

```
def build_generator():  
    noise = Input(shape=(noise_dimension,), name='generator_input')  
    x = layers.Dense(7 * 7 * 256)(noise)  
    x = layers.LeakyReLU(alpha=0.2)(x)  
    x = layers.Reshape((7, 7, 256))(x)  
    x = layers.Conv2DTranspose(128, (4, 4), strides=(2, 2), padding='same')(x)  
    x = layers.LeakyReLU(alpha=0.2)(x)  
    x = layers.Conv2DTranspose(64, (4, 4), strides=(2, 2), padding='same')(x)  
    x = layers.LeakyReLU(alpha=0.2)(x)  
    x = layers.Conv2D(1, kernel_size=5, padding='same', activation='tanh')(x)  
    model = Model(noise, x, name='generator')  
    return model
```

달라진 부분만 살펴보도록 한다.

이 함수는 Generator를 생성하는 함수이다.

Dense로 7x7x256크기의 텐서를 생성하는데, 이는 MNIST가 28x28x1 크기이기 때문에 추후 업샘플링하기 용이하기 때문이다.

Leaky ReLU의 음의 값 부분의 기울기를 0.2로 설정하여 Dying ReLU 문제를 방지한다.

(7, 7, 256) 크기의 3D텐서로 값을 변환한 후, Transposed Convolution 연산을 수행한다.

7 by 7 사이즈를 14 by 14, 28 by 28로 업샘플링하여 원본 MNIST와 동일한 크기까지 만든다.

출력 레이어의 활성화함수를 hyperbolic tangent로 설정하여 출력의 범위를 -1~1로 제한한다.

연산을 마친 후, noise를 받아 (28, 28, 1)크기의 이미지를 출력하는 Generator 모델을 생성한다.

```
def build_discriminator():  
    img = Input(shape=(28, 28, 1), name='discriminator_input')  
    x = layers.Conv2D(64, (4, 4), strides=(2, 2), padding='same')(img)  
    x = layers.LeakyReLU(alpha=0.2)(x)  
    x = layers.Conv2D(128, (4, 4), strides=(2, 2), padding='same')(x)  
    x = layers.LeakyReLU(alpha=0.2)(x)  
    x = layers.Conv2D(256, (4, 4), strides=(2, 2), padding='same')(x)  
    x = layers.Flatten()(x)  
    x = layers.Dropout(0.3)(x)  
    x = layers.Dense(1, activation='sigmoid')(x)  
    model = Model(img, x, name='discriminator')  
    return model
```

달라진 부분만 살펴보도록 한다.

이 함수는 Discriminator를 생성하는 함수이다.

(28, 28, 1)크기의 이미지(진짜 또는 가짜 이미지)를 입력받는다.

Leaky ReLU의 음의 값 부분의 기울기를 0.2로 설정하여 Dying ReLU 문제를 방지한다.

일반 Convolution을 사용하여 (14, 14, 64)로 다운샘플링한다.

이후 동일하게 Convolution 연산을 사용하여 이미지의 크기를 줄이며 특징을 추출한다.

Flatten 레이어를 사용하여 차원을 줄인 후, Dropout으로 Overfitting을 방지하고, Dense 레이어에 sigmoid를 사용하여 결과를 0~1로 변환한다.

연산을 마친 후, 이미지를 입력받아 이 이미지가 진짜인지 가짜인지 판별하는 Discriminator 모델을 생성한다.

생성자와 판별자에서 활성화함수로 softmax와 sigmoid를 사용하고 있다. 이유는 아래와 같다.

1. Discriminator의 목적은 이진 분류이므로, sigmoid를 사용한다.
2. 이진 분류이기 때문에 다중 분류보다 모델이 더 간단하다. 따라서 다중 분류에 많이 쓰이는 softmax보다 sigmoid가 더 간단하고 적합하다.
3. sigmoid는 CEE와 잘 호환된다.

Model Architecture

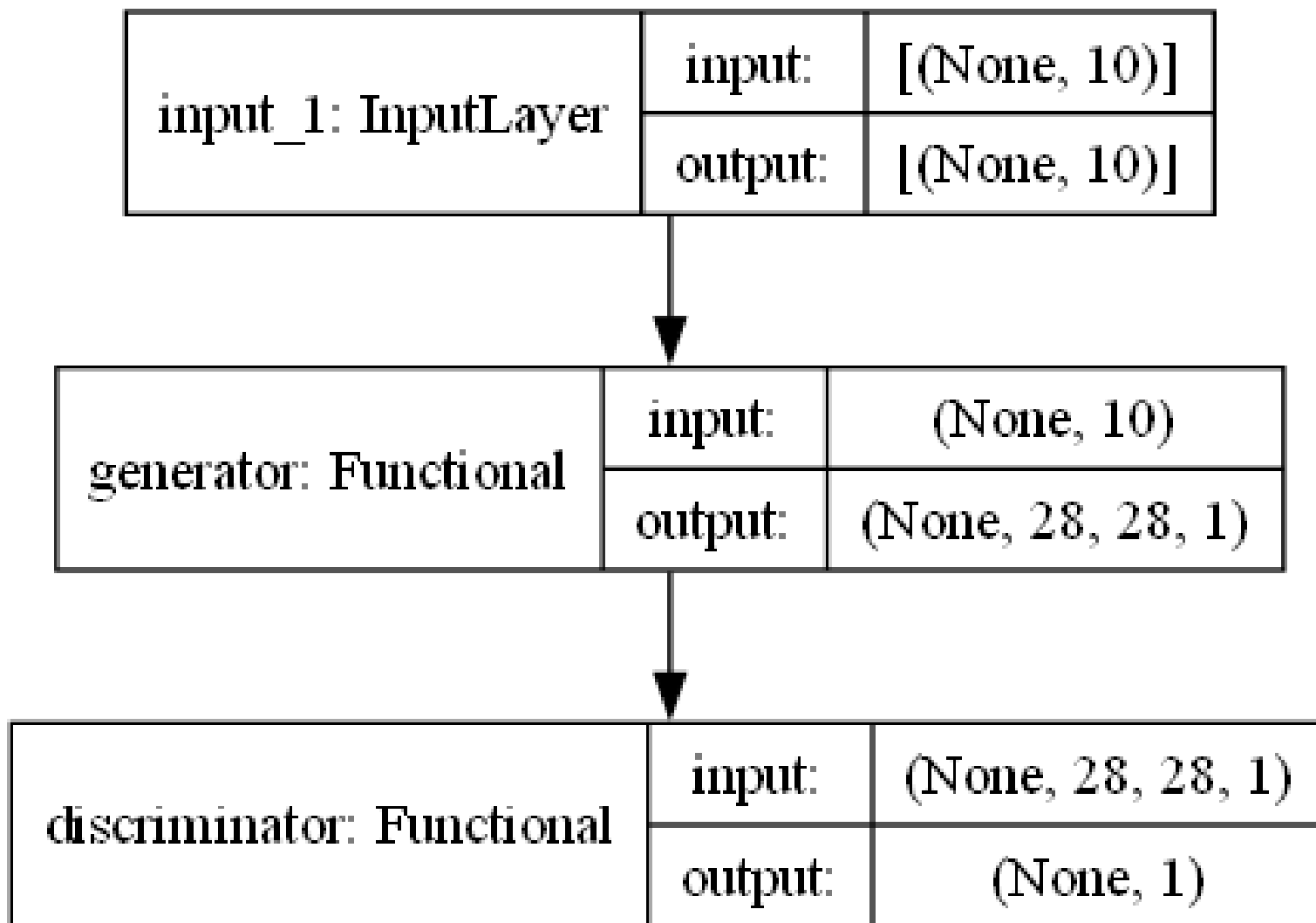
Model: "generator"

Layer (type)	Output Shape	Param #
=====		
generator_input (InputLayer)	[(None, 10)]	0
dense (Dense)	(None, 12544)	137984
leaky_re_lu (LeakyReLU)	(None, 12544)	0
reshape (Reshape)	(None, 7, 7, 256)	0
conv2d_transpose (Conv2DTran	(None, 14, 14, 128)	524416
leaky_re_lu_1 (LeakyReLU)	(None, 14, 14, 128)	0
conv2d_transpose_1 (Conv2DTr	(None, 28, 28, 64)	131136
leaky_re_lu_2 (LeakyReLU)	(None, 28, 28, 64)	0
conv2d (Conv2D)	(None, 28, 28, 1)	1601
=====		
Total params: 795,137		
Trainable params: 795,137		
Non-trainable params: 0		

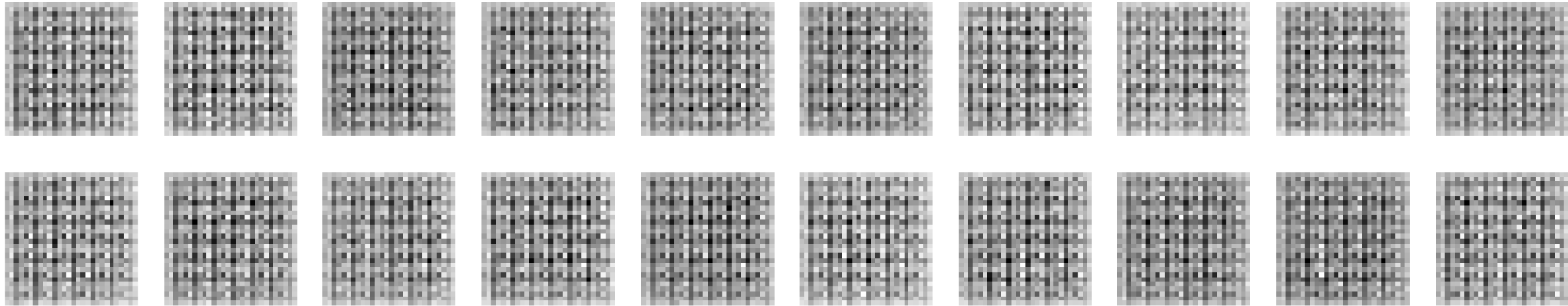
Model: "discriminator"

Layer (type)	Output Shape	Param #
=====		
discriminator_input (InputLa	[(None, 28, 28, 1)]	0
conv2d_1 (Conv2D)	(None, 14, 14, 64)	1088
leaky_re_lu_3 (LeakyReLU)	(None, 14, 14, 64)	0
conv2d_2 (Conv2D)	(None, 7, 7, 128)	131200
leaky_re_lu_4 (LeakyReLU)	(None, 7, 7, 128)	0
conv2d_3 (Conv2D)	(None, 4, 4, 256)	524544
flatten (Flatten)	(None, 4096)	0
dropout (Dropout)	(None, 4096)	0
dense_1 (Dense)	(None, 1)	4097
=====		
Total params: 660,929		
Trainable params: 660,929		
Non-trainable params: 0		

Model Architecture

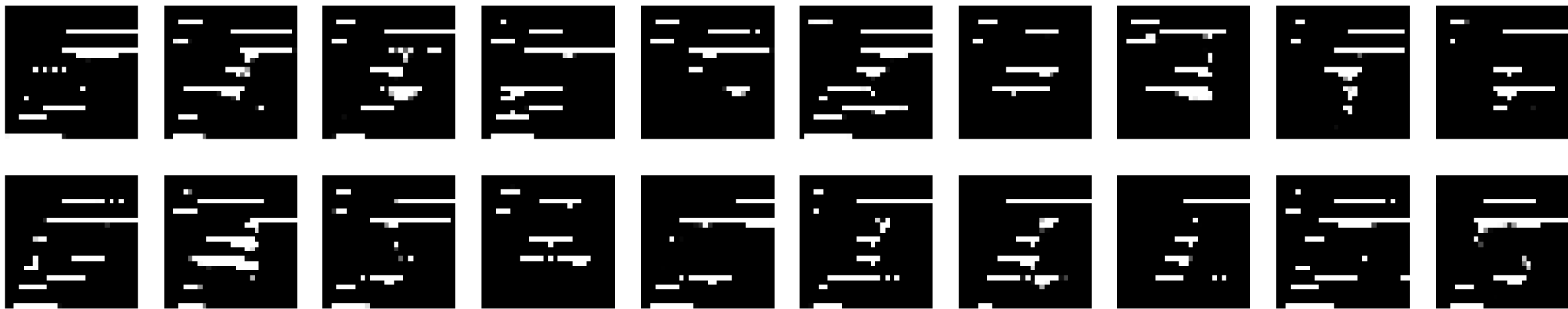


Iteration 0



DCGAN의 학습 전, 무작위 노이즈가 입력된 모습

Iteration 1000



Iteration이 1000번 진행된 후이다.
아직까지는 이상한 그림이다.
배경이 완전한 검은색으로 바뀌었다는게 고무적이다.

Iteration 2000



2000번의 학습 후, 이상하지만 점차 숫자의 모습을 갖추어가고 있다.

Iteration 3000



3000번 학습하니 이제 무슨 숫자인지까지 어렵פות이 알아낼 수 있다.

Iteration 4000



Iteration 5000



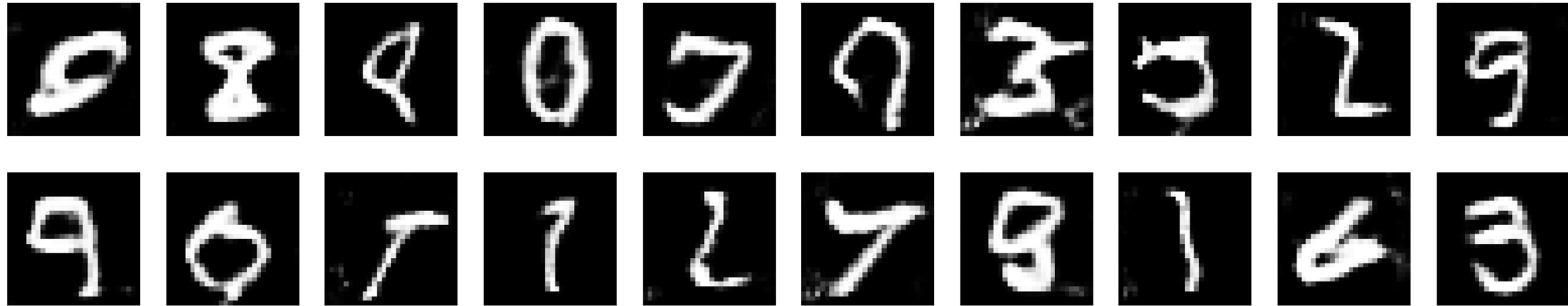
Iteration 6000



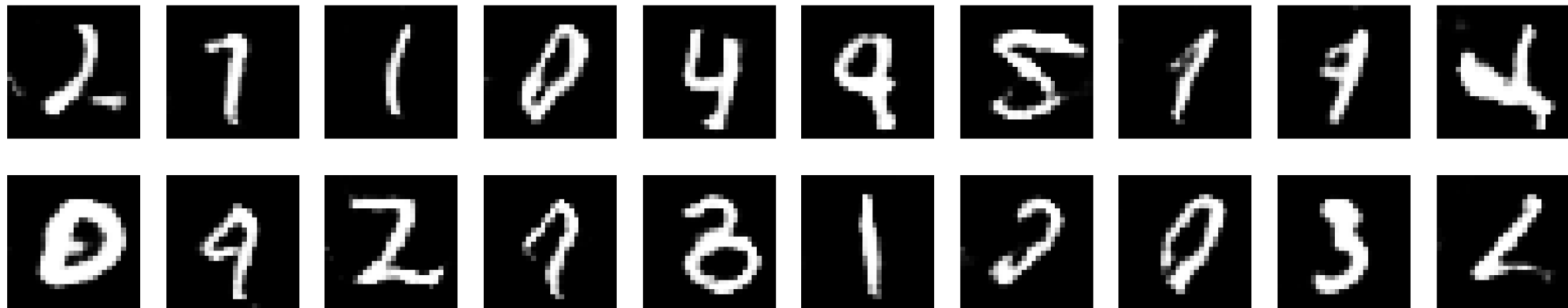
Iteration 7000



Iteration 8000



Iteration 9000

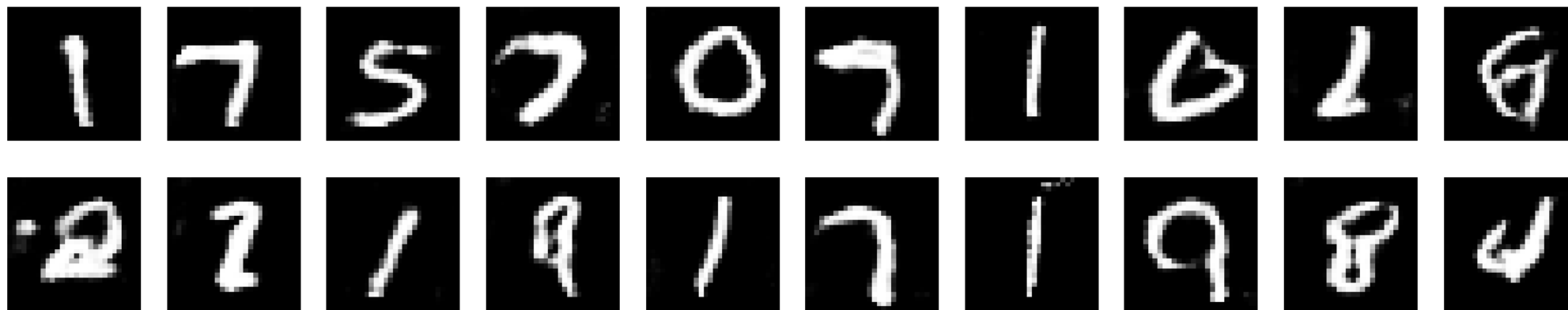


9000번 학습하였다.

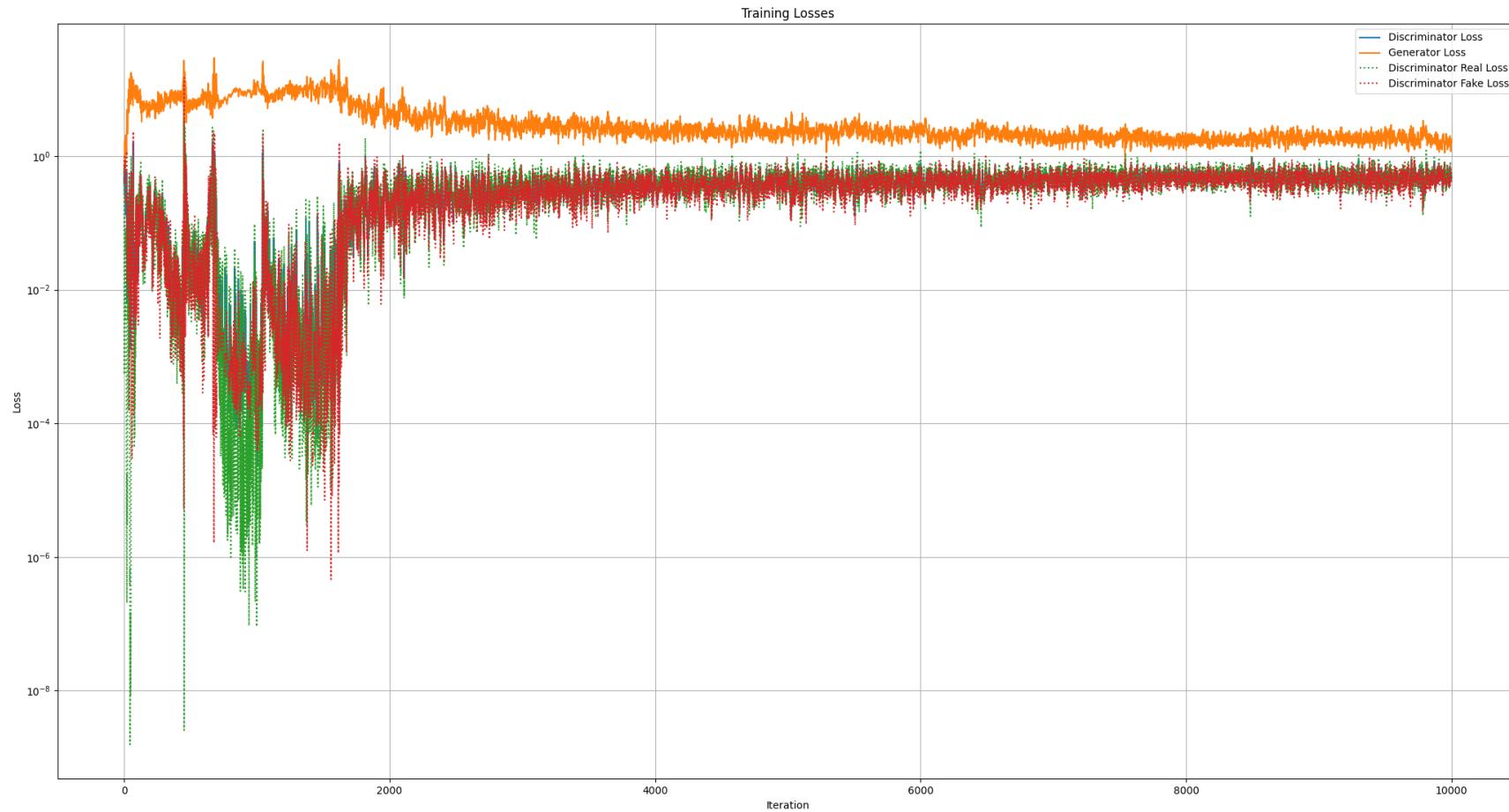
놀라운 수준으로 이미지가 생성된 것을 볼 수 있다.

아까의 Vanilla GAN과 대비하여 Mode Collapse가 일어나지 않고 잘 학습한 모습이다.

Iteration 10000



학습을 마친 모습이다.
실제 MNIST처럼 생성된 것을 볼 수 있다.



아까와는 대조적으로 그래프가 안정적인 모습을 볼 수 있다.
그래프가 안정적인만큼 생성된 이미지의 결과도 좋았다.

Conditional Generative Adversarial Networks (CGAN)

```
def build_generator():  
    noise = Input(shape=(noise_dimension,), name='generator_noise_input')  
    label = Input(shape=(num_classes,), name='generator_label_input')  
    x = layers.Concatenate()([noise, label])  
    x = layers.Dense(7 * 7 * 256)(x)  
    x = layers.LeakyReLU(alpha=0.2)(x)  
    x = layers.Reshape((7, 7, 256))(x)  
    x = layers.Conv2DTranspose(128, (4, 4), strides=(2, 2), padding='same')(x)  
    x = layers.LeakyReLU(alpha=0.2)(x)  
    x = layers.Conv2DTranspose(64, (4, 4), strides=(2, 2), padding='same')(x)  
    x = layers.LeakyReLU(alpha=0.2)(x)  
    x = layers.Conv2D(1, kernel_size=5, padding='same', activation='tanh')(x)  
    model = Model([noise, label], x, name='generator')  
    return model
```

달라진 부분만 살펴보도록 한다.

이 함수는 Generator를 생성하는 함수이다.

CGAN은 생성자의 입력으로 노이즈와 레이블을 넣어야 하므로, 추가적으로 입력하였다.

Concatenate를 이용하여 noise와 label을 결합하여 단일한 벡터로 만든다.

이후 DCGAN과 비슷하게 Dense, Leaky ReLU, Transposed Convolution 등등을 사용하여 연산을 수행한다.

활성화함수로는 hyperbolic tangent를 사용하여 출력을 -1~1로 만든다.

연산을 마친 후, noise와 label을 받아 (28, 28, 1)크기의 이미지를 출력하는 Generator 모델을 생성한다.

```
def build_discriminator():
    img = Input(shape=(28, 28, 1), name='discriminator_image_input')
    label = Input(shape=(num_classes,), name='discriminator_label_input')
    label_embedding = layers.Dense(28 * 28)(label)
    label_embedding = layers.Reshape((28, 28, 1))(label_embedding)
    x = layers.Concatenate()([img, label_embedding])
    x = layers.Conv2D(64, (4, 4), strides=(2, 2), padding='same')(x)
    x = layers.LeakyReLU(alpha=0.2)(x)
    x = layers.Conv2D(128, (4, 4), strides=(2, 2), padding='same')(x)
    x = layers.LeakyReLU(alpha=0.2)(x)
    x = layers.Conv2D(256, (4, 4), strides=(2, 2), padding='same')(x)
    x = layers.Flatten()(x)
    x = layers.Dropout(0.3)(x)
    x = layers.Dense(1, activation='sigmoid')(x)
    model = Model([img, label], x, name='discriminator')
    return model
```

달라진 부분만 살펴보도록 한다.

이 함수는 Discriminator를 생성하는 함수이다.

CGAN의 판별자는 입력된 이미지와 레이블이 서로 일치하는지 판별하는 것이 목적이다.

생성자와 마찬가지로 입력으로 이미지와 레이블을 입력받는다.

레이블을 MNIST 이미지와 동일한 형상으로 변환하여 입력 이미지와 결합하고, 이를 통하여 이미지와 레이블이 일치하는지 평가할 수 있다.

이후 DCGAN과 비슷하게 Dense, Leaky ReLU, Transposed Convolution 등등을 사용하여 연산을 수행한다.

활성화함수로는 sigmoid를 사용하여 이진 분류를 사용한다.

```
def train_discriminator(x_train, y_train):
    discriminator.trainable = True
    # Real 학습
    c = np.random.randint(0, x_train.shape[0], batch_size)
    real = x_train[c]
    real_labels = y_train[c]
    real_labels = tf.keras.utils.to_categorical(real_labels, num_classes) # convert label to 1 hot vector
    d_loss_real = discriminator.train_on_batch([real, real_labels], np.ones((batch_size, 1)))
    # Fake 학습
    p = np.random.normal(0, 1, (batch_size, noise_dimension))
    fake_labels = np.random.randint(0, num_classes, batch_size)
    fake_labels_one_hot = tf.keras.utils.to_categorical(fake_labels, num_classes)
    fake = generator.predict([p, fake_labels_one_hot])
    d_loss_fake = discriminator.train_on_batch([fake, fake_labels_one_hot], np.zeros((batch_size, 1)))
    return d_loss_real, d_loss_fake
```

이 함수는 Discriminator를 훈련하는 함수이다.

목표는 입력된 이미지가 생성된 이미지인지, 진짜 이미지인지 판별하는 것이다.

우선 x_train과 y_train에서 배치의 크기만큼 무작위로 샘플링한다.

utils.to_categorical을 이용하여 레이블을 one-hot encoding한다.

그 이후, 생성자에 랜덤한 노이즈를 입력하여 가짜 이미지를 생성하고, 해당 이미지가 가짜임을 예측하도록 학습한다.

loss를 반환하여 추적을 쉽게 할 수 있도록 하였다.

```
def train_generator():  
    discriminator.trainable = False  
    p = np.random.normal(0, 1, (batch_size, noise_dimension))  
    random_labels = np.random.randint(0, num_classes, batch_size)  
    random_labels_one_hot = tf.keras.utils.to_categorical(random_labels, num_classes)  
    g_loss = gan.train_on_batch([p, random_labels_one_hot], np.ones((batch_size, 1)))  
    return g_loss
```

이 함수는 Generator를 훈련하는 함수이다.

목표는 판별자가 생성된 이미지를 진짜로 판단할 수 있게 판별자를 속이는 것이다.

우선 무작위로 노이즈와 레이블을 사용하고, utils.to_categorical을 이용하여 레이블을 one-hot encoding한다.

이후 GAN 전체를 학습하는데, 목표는 판별자가 진짜라고 판단하게 만드는 것이므로 np.ones(0: 가짜, 1: 진짜)를 사용한다.

loss를 반환하여 추적을 쉽게 할 수 있도록 하였다.

```
while(True):  
    label = int(input("생성하고 싶은 숫자 입력(0~9): "))  
    label_one_hot = tf.keras.utils.to_categorical([label], num_classes)  
    noise = np.random.normal(0, 1, (1, noise_dimension))  
    generated_image = generator.predict([noise, label_one_hot])  
  
    plt.imshow(generated_image[0].reshape(28, 28), cmap='gray')  
    plt.axis('off')  
    plt.title(f"Input Int: {label}")  
    plt.xticks([])  
    plt.yticks([])  
    plt.show()
```

학습이 완료된 후 숫자를 입력하면 해당 이미지를 생성해주는 코드이다.

목표 레이블을 입력하면 해당 값을 one-hot encoding한 후, 랜덤 생성된 노이즈와 함께 모델에 입력한다.

이후 생성된 이미지를 표시한다.

Model Architecture

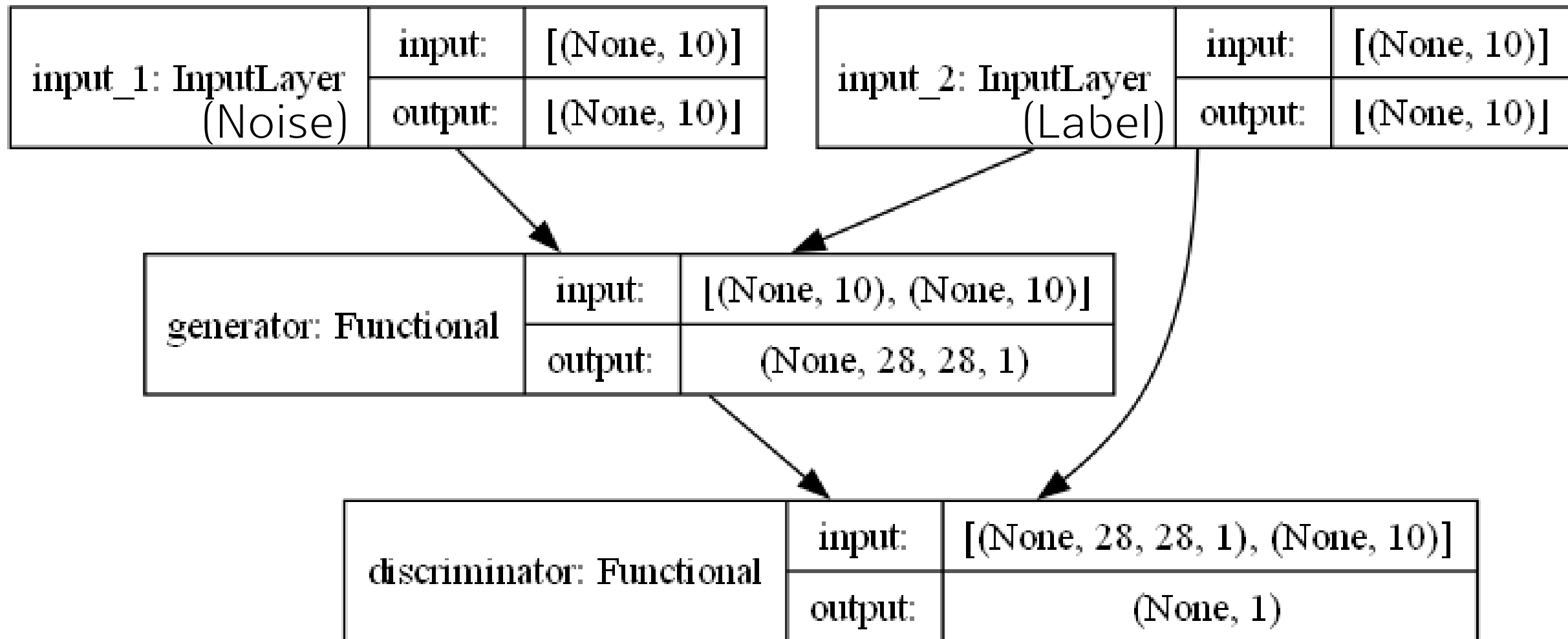
Model: "generator"

Layer (type)	Output Shape	Param #	Connected to
generator_noise_input (InputLayer)	[(None, 10)]	0	
generator_label_input (InputLayer)	[(None, 10)]	0	
concatenate (Concatenate)	(None, 20)	0	generator_noise_input[0][0] generator_label_input[0][0]
dense (Dense)	(None, 12544)	263424	concatenate[0][0]
leaky_re_lu (LeakyReLU)	(None, 12544)	0	dense[0][0]
reshape (Reshape)	(None, 7, 7, 256)	0	leaky_re_lu[0][0]
conv2d_transpose (Conv2DTranspose)	(None, 14, 14, 128)	524416	reshape[0][0]
leaky_re_lu_1 (LeakyReLU)	(None, 14, 14, 128)	0	conv2d_transpose[0][0]
conv2d_transpose_1 (Conv2DTranspose)	(None, 28, 28, 64)	131136	leaky_re_lu_1[0][0]
leaky_re_lu_2 (LeakyReLU)	(None, 28, 28, 64)	0	conv2d_transpose_1[0][0]
conv2d (Conv2D)	(None, 28, 28, 1)	1601	leaky_re_lu_2[0][0]
Total params: 920,577			
Trainable params: 920,577			
Non-trainable params: 0			

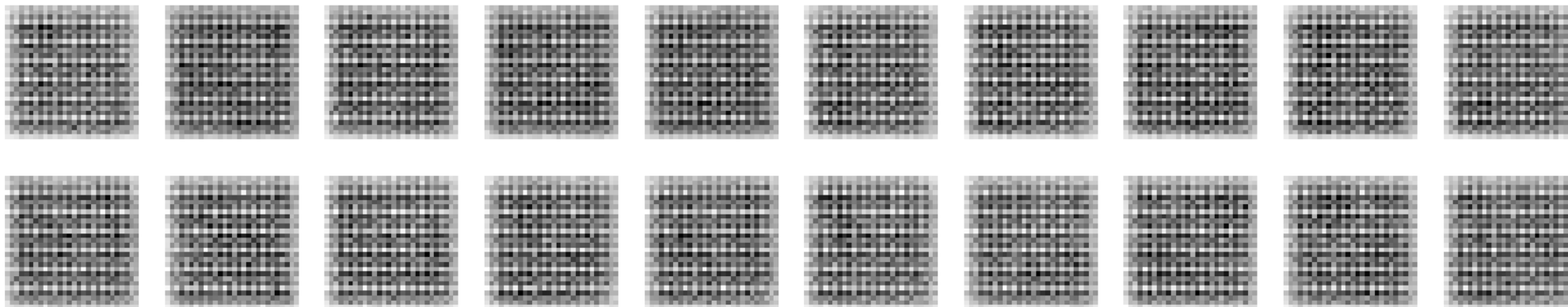
Model: "discriminator"

Layer (type)	Output Shape	Param #	Connected to
discriminator_label_input (InputLayer)	[(None, 10)]	0	
dense_1 (Dense)	(None, 784)	8624	discriminator_label_input[0][0]
discriminator_image_input (InputLayer)	[(None, 28, 28, 1)]	0	
reshape_1 (Reshape)	(None, 28, 28, 1)	0	dense_1[0][0]
concatenate_1 (Concatenate)	(None, 28, 28, 2)	0	discriminator_image_input[0][0] reshape_1[0][0]
conv2d_1 (Conv2D)	(None, 14, 14, 64)	2112	concatenate_1[0][0]
leaky_re_lu_3 (LeakyReLU)	(None, 14, 14, 64)	0	conv2d_1[0][0]
conv2d_2 (Conv2D)	(None, 7, 7, 128)	131200	leaky_re_lu_3[0][0]
leaky_re_lu_4 (LeakyReLU)	(None, 7, 7, 128)	0	conv2d_2[0][0]
conv2d_3 (Conv2D)	(None, 4, 4, 256)	524544	leaky_re_lu_4[0][0]
flatten (Flatten)	(None, 4096)	0	conv2d_3[0][0]
dropout (Dropout)	(None, 4096)	0	flatten[0][0]
dense_2 (Dense)	(None, 1)	4097	dropout[0][0]
Total params: 670,577			
Trainable params: 670,577			
Non-trainable params: 0			

Model Architecture

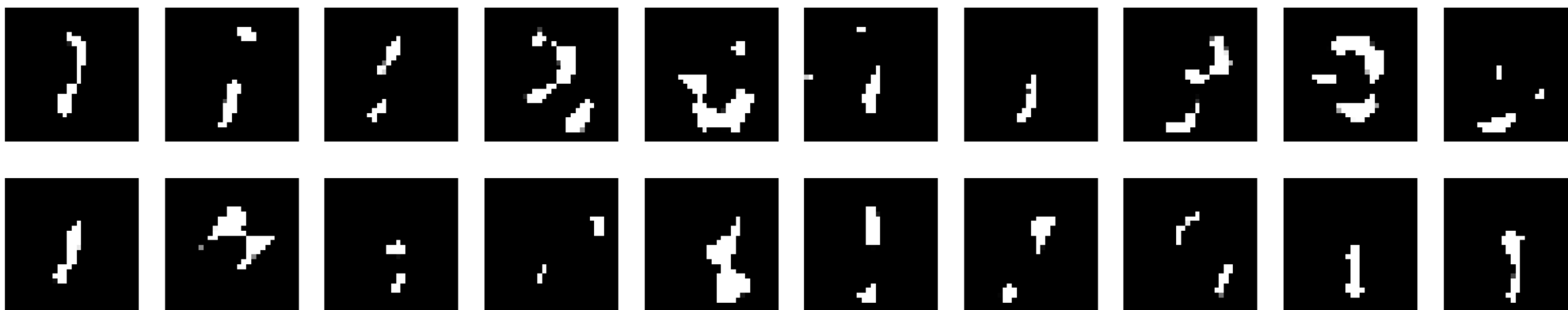


Iteration 0



CGAN의 학습 전, 무작위 노이즈가 입력된 모습

Iteration 1000

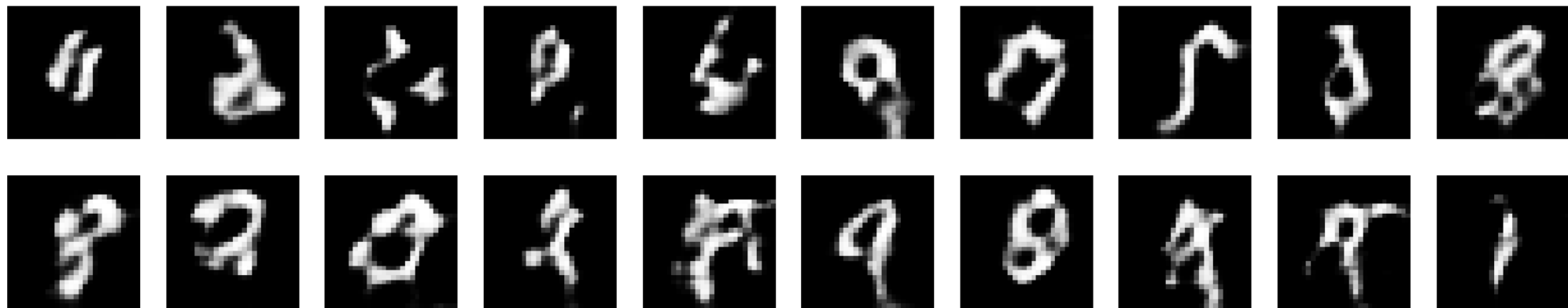


Iteration이 1000번 진행된 후이다.
아까 DCGAN보다는 나은 것 같지만, 아직 불만한 이미지는 생성되지 않았다.

Iteration 2000



Iteration 3000



3000번 정도 학습하니 드디어 뭔가 숫자다운 이미지가 나오기 시작한다.

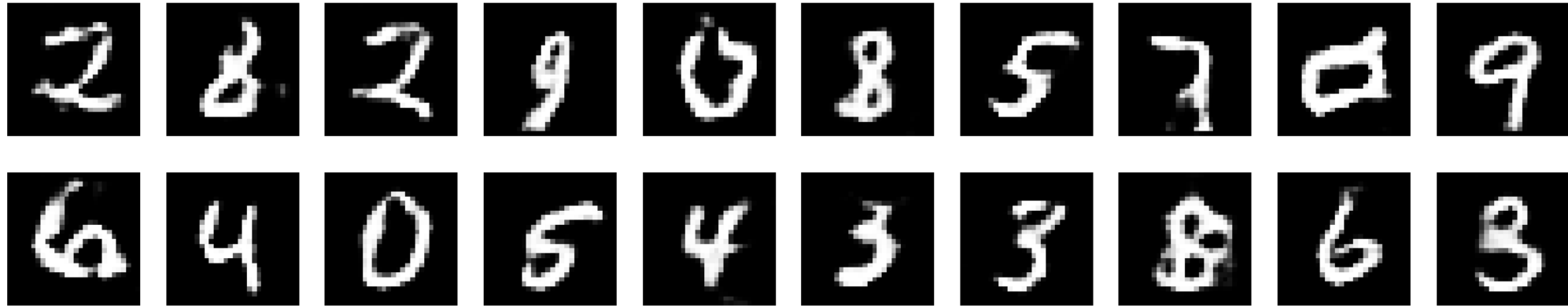
Iteration 4000



Iteration 5000



Iteration 6000

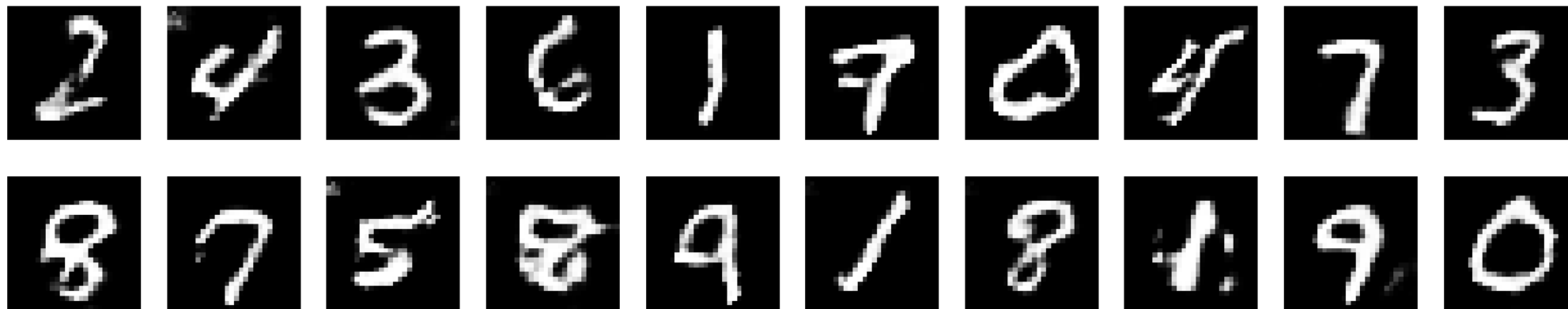


6000번 정도 학습하니 꽤나 숫자같이 생긴 이미지가 나오는 모습이다.

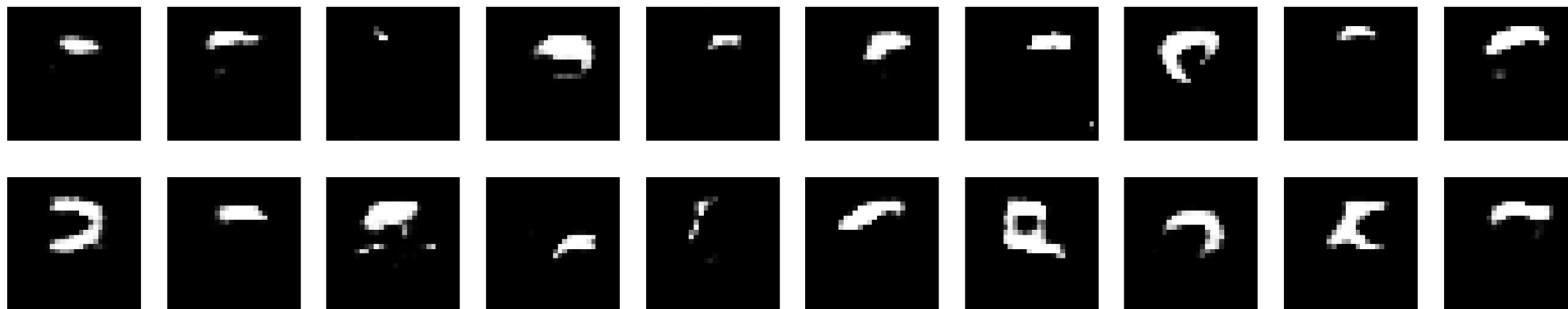
Iteration 7000



Iteration 8000



Iteration 9000



9000번 학습하니 오히려 이미지가 뭔가 잘린 듯 이상하게 나왔다.

이는 여러가지 원인때문이라고 추측되는데, GAN 계열 모델들은 생성자와 판별자의 학습 균형이 대단히 중요하다.

만약 특정 iteration에서 학습 균형이 깨지면 이러한 사진이 나올 수 있다.

또는 Mode Collapse 문제일 수도 있고, adam의 learning rate 문제일 수도 있다.

만약 GAN 계열 모델에서 판별자가 지나치게 강하면, 생성자가 판별자를 속이기 어려워지므로, 판별자를 가장 잘 속이는 특정한 패턴만 지속적으로 생성하게 된다.

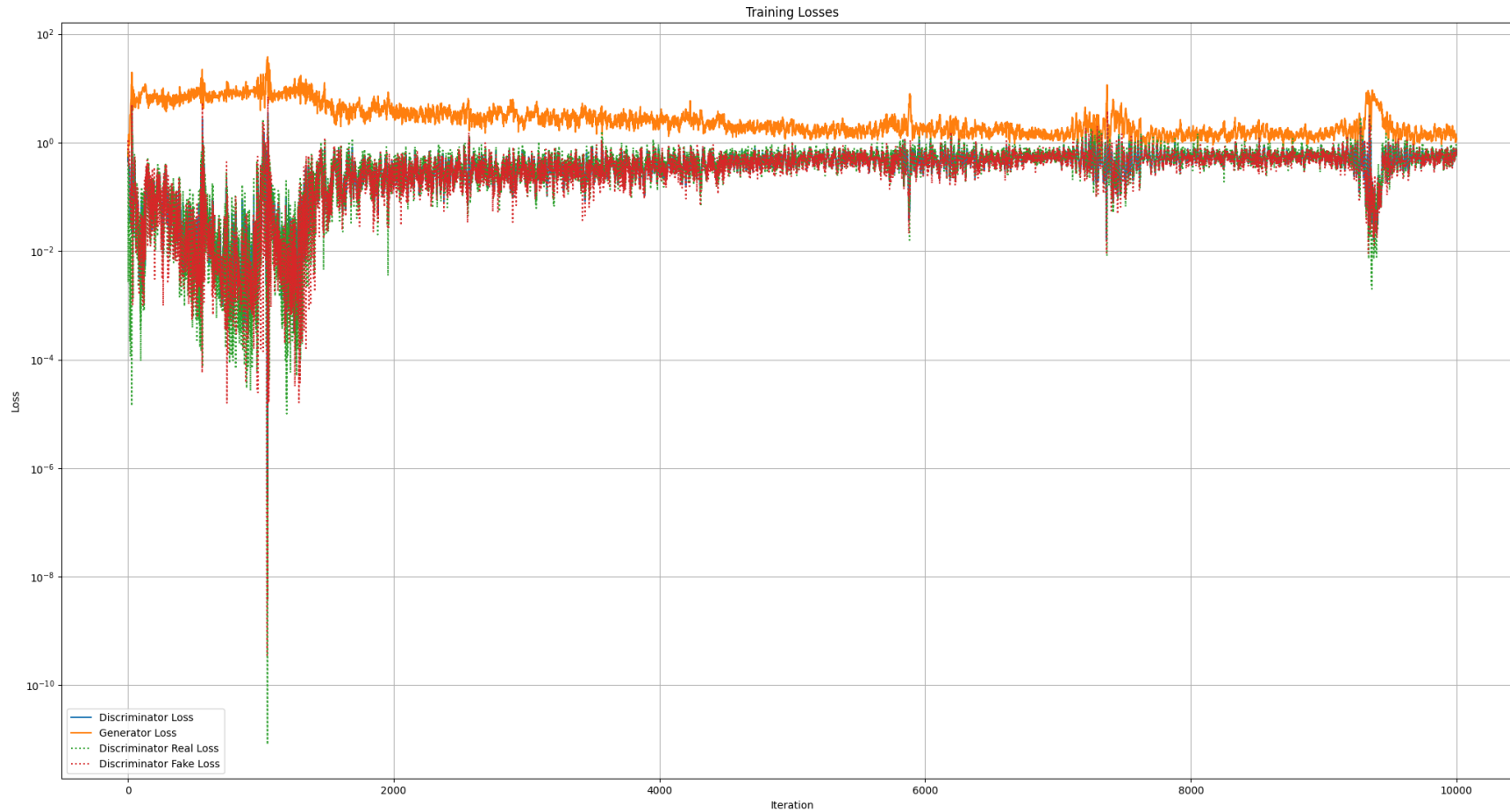
또는 그저 9000번째 iteration에서 운 나쁘게도 노이즈 샘플링이 잘못된 값으로 되었을 수도 있다.

Iteration 10000



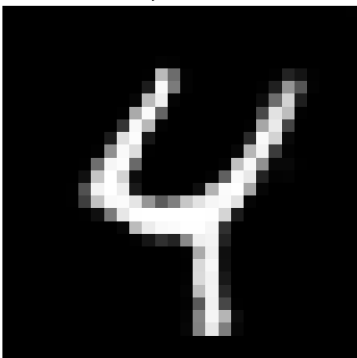
학습을 마친 모습이다.

9000번째 iteration에서의 불안정을 딛고 실제 MNIST처럼 생성된 것을 볼 수 있다.

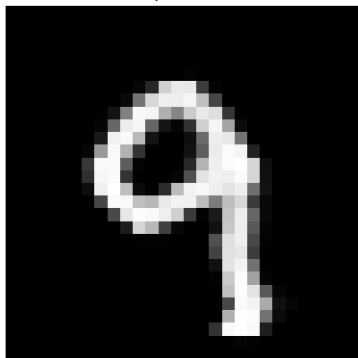


그래프에서 볼 수 있듯, iteration이 6000, 7000, 9000부근에서 loss값이 요동치는걸 볼 수 있다.
이러한 값이 쌓여 iteration이 9000일 때 이상한 이미지가 생성되었으리라 추측된다.

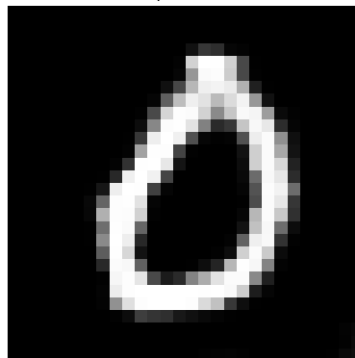
Input Int: 4



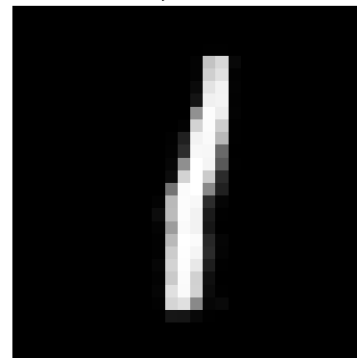
Input Int: 9



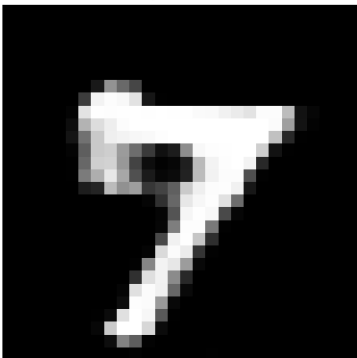
Input Int: 0



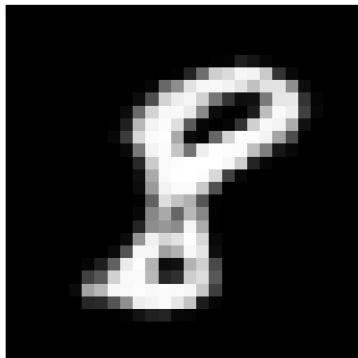
Input Int: 1



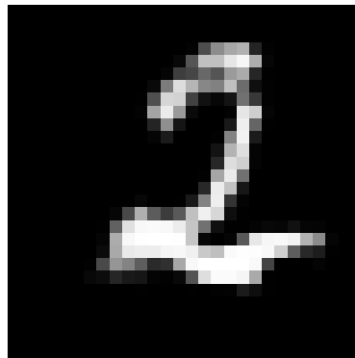
Input Int: 7



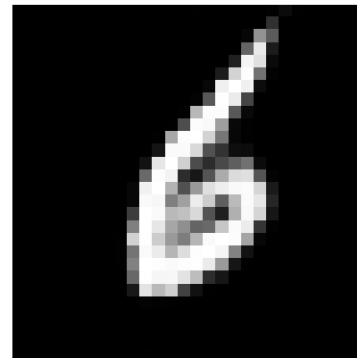
Input Int: 8



Input Int: 2

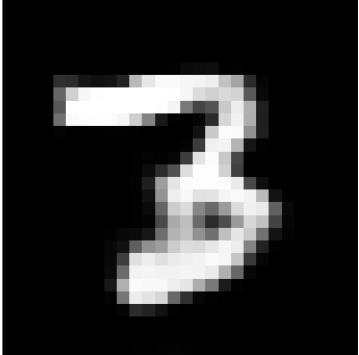


Input Int: 6

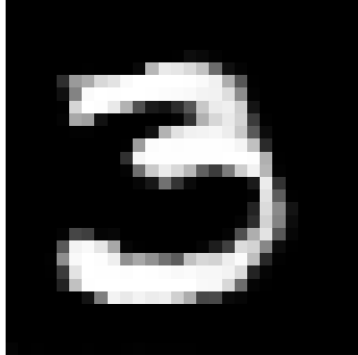


실제로 특정 레이블을 넣어 실행해본 모습이다.
매우 정확하고 볼만한 이미지를 생성하는 걸 볼 수 있다.

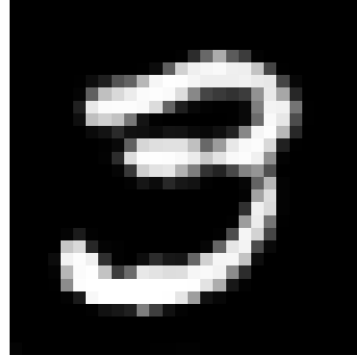
Input Int: 3



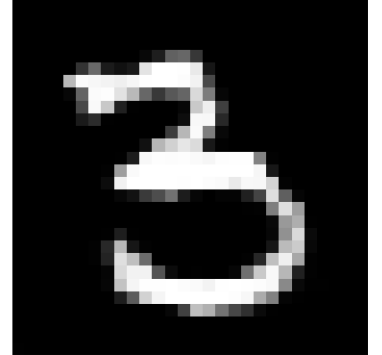
Input Int: 3



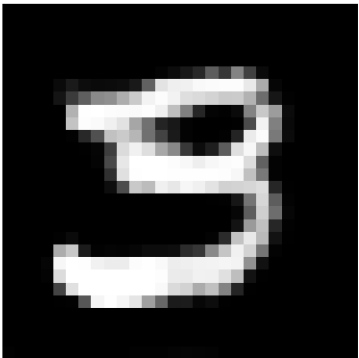
Input Int: 3



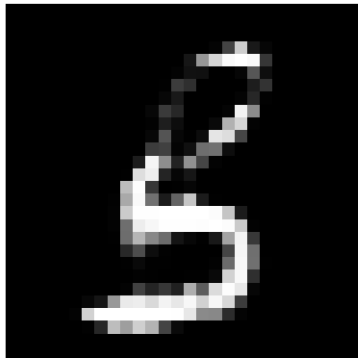
Input Int: 3



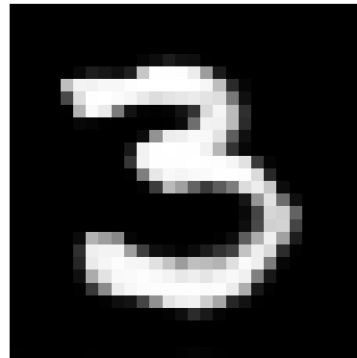
Input Int: 3



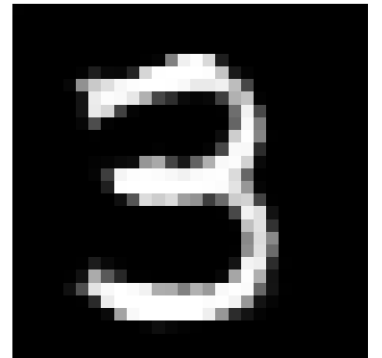
Input Int: 3



Input Int: 3



Input Int: 3



동일한 레이블을 여러번 입력해본 모습이다.
정확하지 않은 이미지도 있지만 전체적으로 누가 봐도 3이 출력된 모습이다.

Least Squares Generative Adversarial Networks (LSGAN)

```
def least_square_loss_discriminator(y_true, y_pred):  
    return 0.5 * tf.reduce_mean(tf.square(y_true - y_pred))  
  
def least_square_loss_generator(y_true, y_pred):  
    return 0.5 * tf.reduce_mean(tf.square(y_true - y_pred))
```

달라진 부분만 살펴보도록 한다.

이 함수들은 LSGAN에서의 손실 함수들을 정의하는 부분이다.

사실, CGAN과 LSGAN은 거의 동일하지만, 몇 가지의 차이점이 있다.

- CGAN에서는 일반적인 Binary Cross-Entropy를 사용하지만, LSGAN은 Least Squares Loss를 사용한다.
이는 Discriminator의 출력과 목표 값(진짜는 1, 가짜는 0) 사이의 제곱 오차를 최소화하는 방식으로 학습되며, Discriminator의 기울기 소멸 문제를 완화하여 학습을 더 안정적으로 만드는 데 도움을 준다.
- LSGAN은 Least Square 방식으로 손실을 정의하는 특성상, Discriminator와 Generator 간의 학습 균형을 CGAN대비 더 잘 맞추어 모드 붕괴나 불안정한 학습을 최소화한다.
- LSGAN의 Discriminator는 출력값이 0~1 사이에 분포하지 않을 가능성이 있으며, 대신 출력값을 원하는 목표(주로 1)로 수렴되도록 학습한다.

LSGAN의 논문에서는 loss function을 정의할 때 0.5를 곱하는 방식을 제안하며, 이는 학습이 더 안정적으로 되도록 돕는다.

해당 함수들은 Least Square 방식으로 손실을 계산하여 반환한다.

Model Architecture

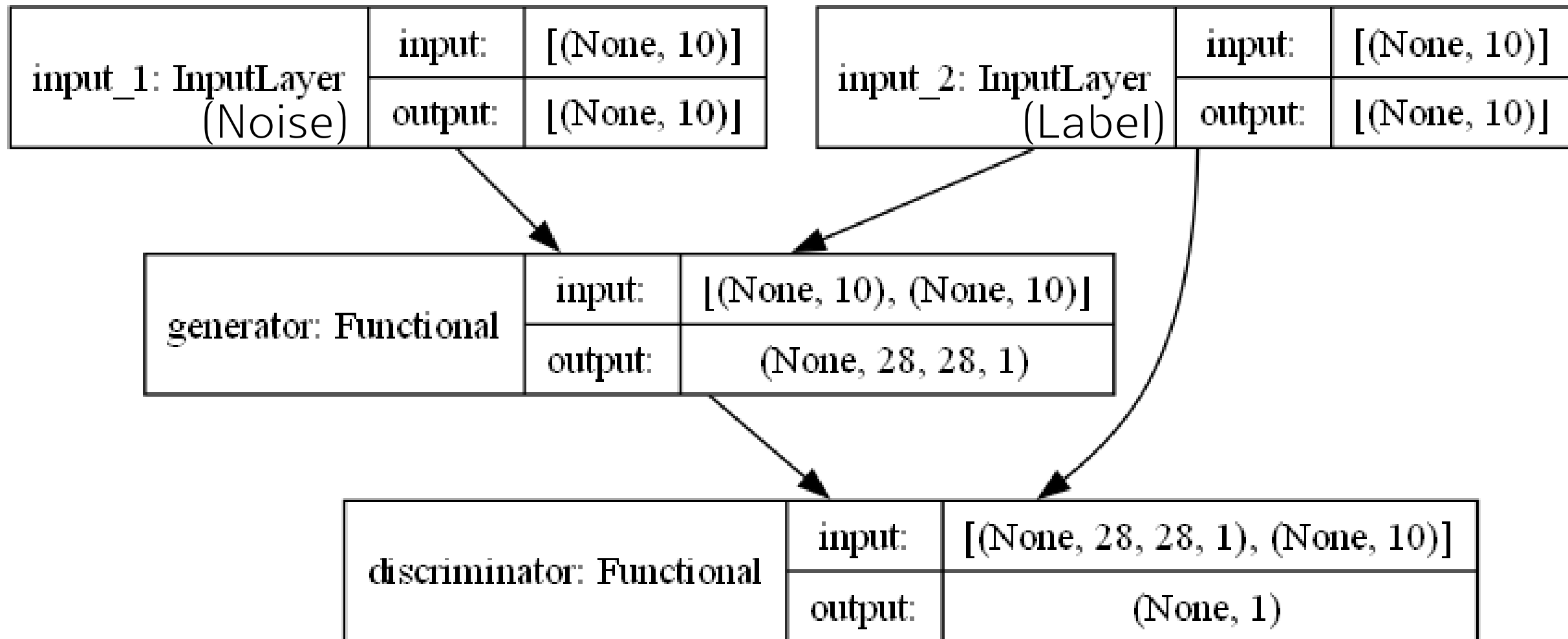
Model: "generator"

Layer (type)	Output Shape	Param #	Connected to
generator_noise_input (InputLayer)	[(None, 10)]	0	
generator_label_input (InputLayer)	[(None, 10)]	0	
concatenate (Concatenate)	(None, 20)	0	generator_noise_input[0][0] generator_label_input[0][0]
dense (Dense)	(None, 12544)	263424	concatenate[0][0]
leaky_re_lu (LeakyReLU)	(None, 12544)	0	dense[0][0]
reshape (Reshape)	(None, 7, 7, 256)	0	leaky_re_lu[0][0]
conv2d_transpose (Conv2DTranspose)	(None, 14, 14, 128)	524416	reshape[0][0]
leaky_re_lu_1 (LeakyReLU)	(None, 14, 14, 128)	0	conv2d_transpose[0][0]
conv2d_transpose_1 (Conv2DTranspose)	(None, 28, 28, 64)	131136	leaky_re_lu_1[0][0]
leaky_re_lu_2 (LeakyReLU)	(None, 28, 28, 64)	0	conv2d_transpose_1[0][0]
conv2d (Conv2D)	(None, 28, 28, 1)	1601	leaky_re_lu_2[0][0]
Total params: 920,577			
Trainable params: 920,577			
Non-trainable params: 0			

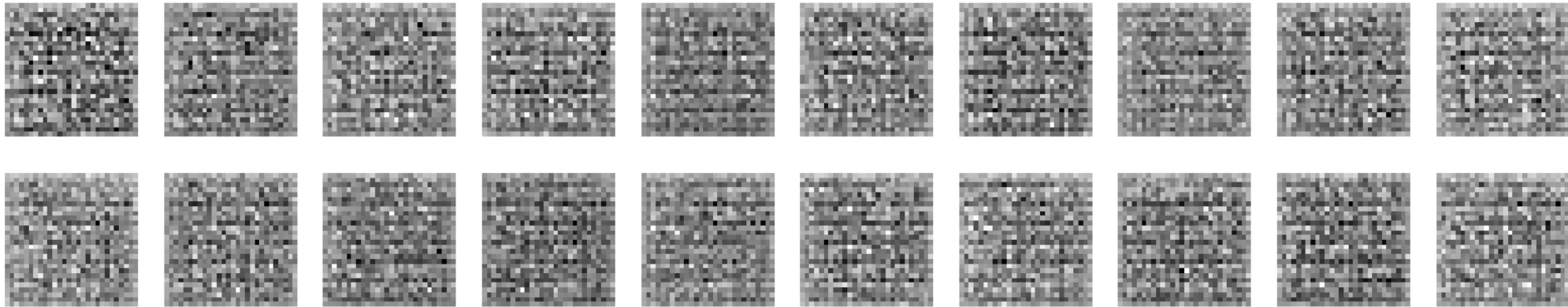
Model: "discriminator"

Layer (type)	Output Shape	Param #	Connected to
discriminator_label_input (InputLayer)	[(None, 10)]	0	
dense_1 (Dense)	(None, 784)	8624	discriminator_label_input[0][0]
discriminator_image_input (InputLayer)	[(None, 28, 28, 1)]	0	
reshape_1 (Reshape)	(None, 28, 28, 1)	0	dense_1[0][0]
concatenate_1 (Concatenate)	(None, 28, 28, 2)	0	discriminator_image_input[0][0] reshape_1[0][0]
conv2d_1 (Conv2D)	(None, 14, 14, 64)	2112	concatenate_1[0][0]
leaky_re_lu_3 (LeakyReLU)	(None, 14, 14, 64)	0	conv2d_1[0][0]
conv2d_2 (Conv2D)	(None, 7, 7, 128)	131200	leaky_re_lu_3[0][0]
leaky_re_lu_4 (LeakyReLU)	(None, 7, 7, 128)	0	conv2d_2[0][0]
conv2d_3 (Conv2D)	(None, 4, 4, 256)	524544	leaky_re_lu_4[0][0]
flatten (Flatten)	(None, 4096)	0	conv2d_3[0][0]
dropout (Dropout)	(None, 4096)	0	flatten[0][0]
dense_2 (Dense)	(None, 1)	4097	dropout[0][0]
Total params: 670,577			
Trainable params: 670,577			
Non-trainable params: 0			

Model Architecture



Iteration 0



LSGAN의 학습 전, 무작위 노이즈가 입력된 모습

Iteration 1000



Iteration이 1000번 진행된 후이다.
기존 VGAN, DCGAN, CGAN 대비 처음부터 꽤나 정확하게 생성된 모습이다.

Iteration 2000



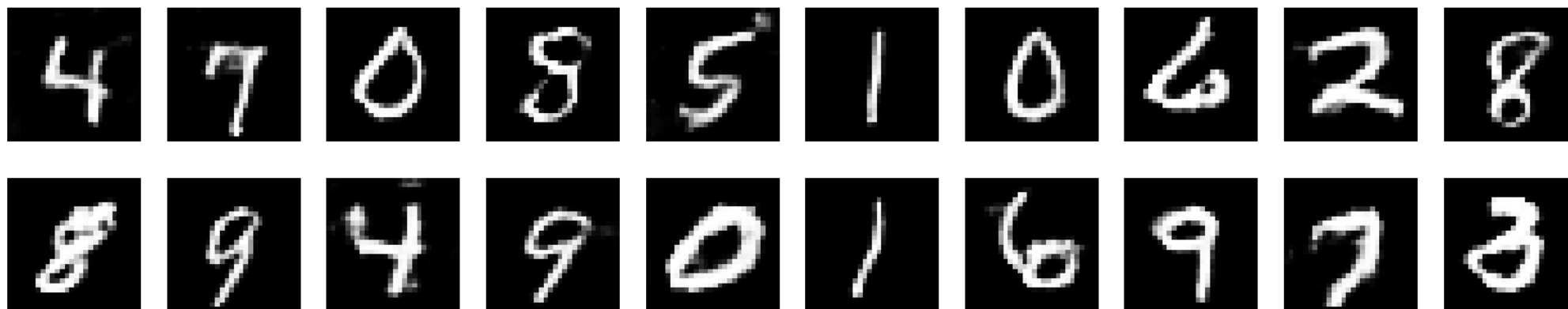
Iteration 3000



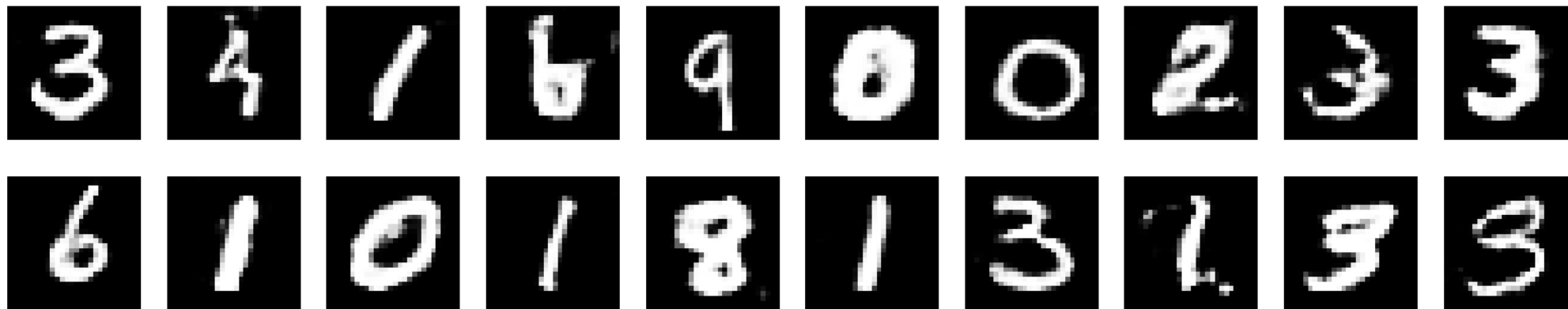
Iteration 4000



Iteration 5000



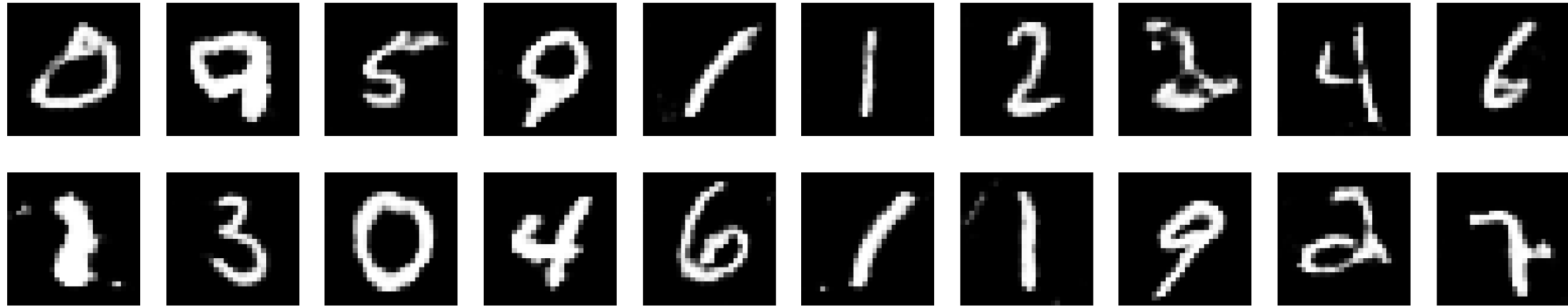
Iteration 6000



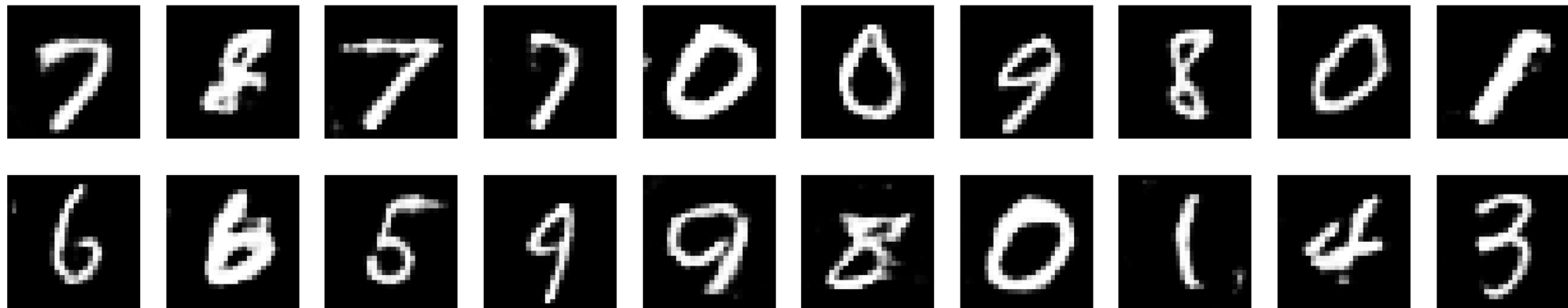
Iteration 7000



Iteration 8000

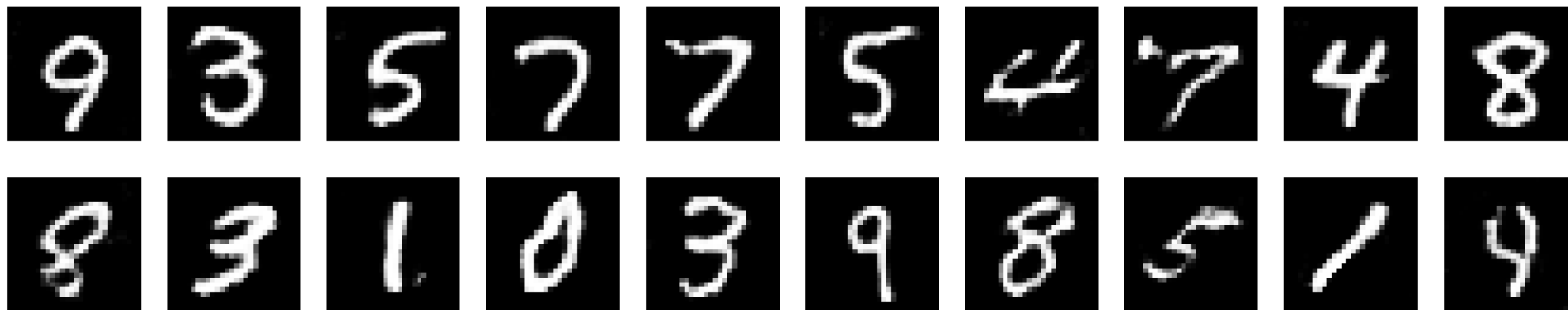


Iteration 9000

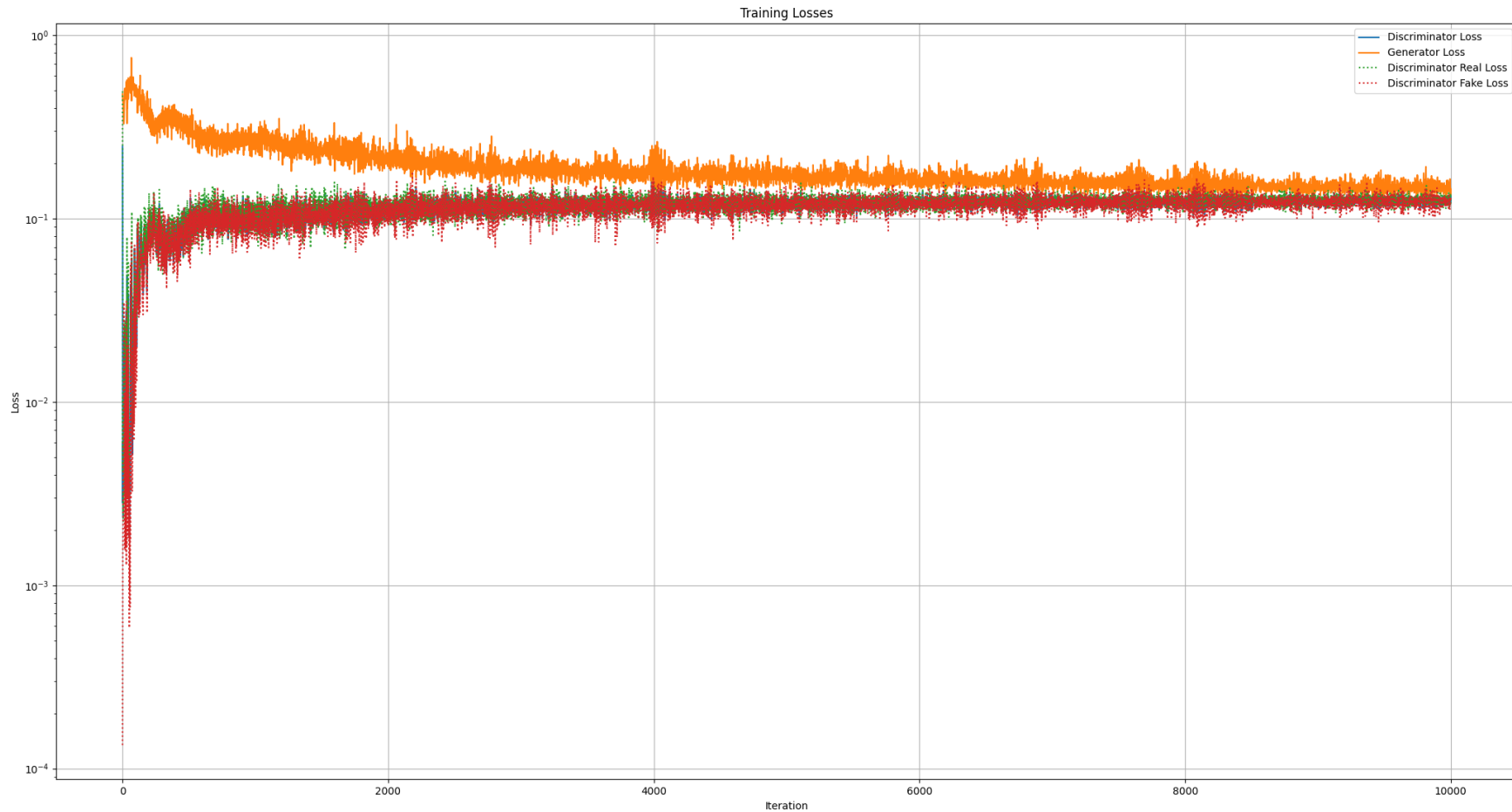


Iteration이 9000번 진행된 후이다.
맨 처음의 VGAN에 비하면 괄목할 만한 성능향상으로,
Mode Collapse가 일어나는 경우가 거의 없어진 모습을 보이고 있다.

Iteration 10000

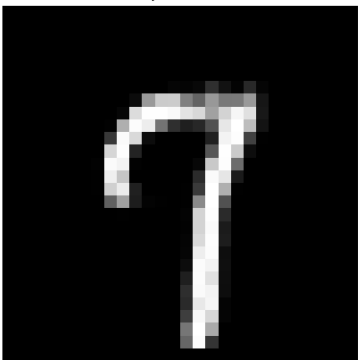


학습을 마친 모습이다.
여태까지의 모델들 중에서는 그나마 제일 나은 모습을 보이는 것 같다.

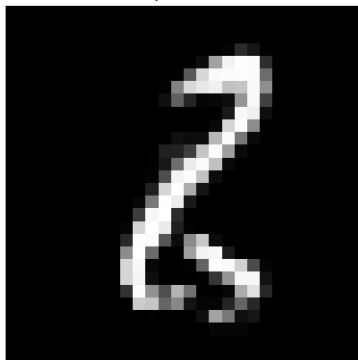


여태까지의 모델들 중에서 가장 안정적으로 수렴하는 모습을 보인다.
이는 Least Squares 방식을 사용함과 동시에, loss값에 0.5를 곱하여 한 층 더 안정적으로 학습되었기 때문으로 추정된다.

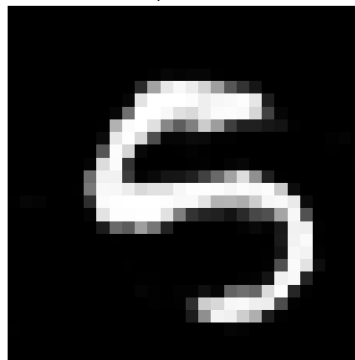
Input Int: 7



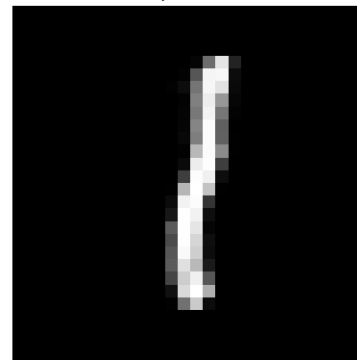
Input Int: 2



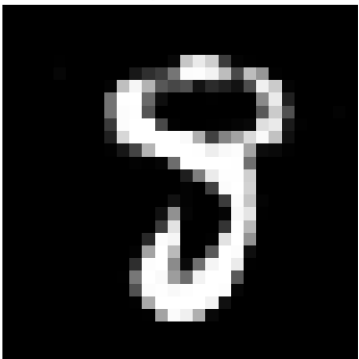
Input Int: 5



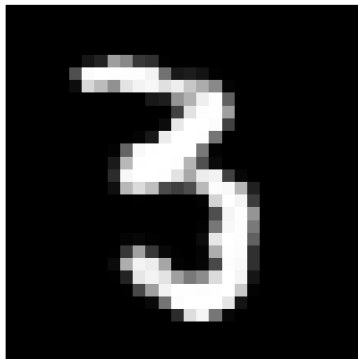
Input Int: 1



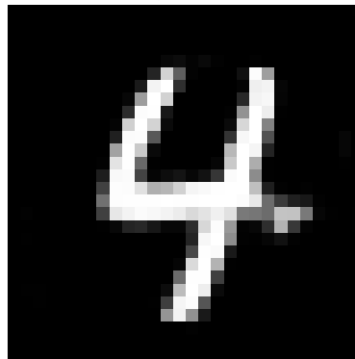
Input Int: 8



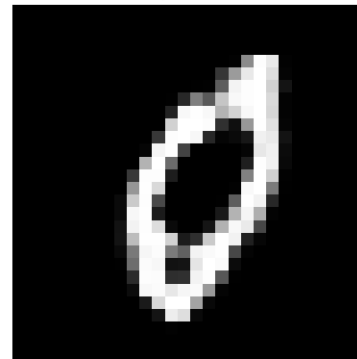
Input Int: 3



Input Int: 4

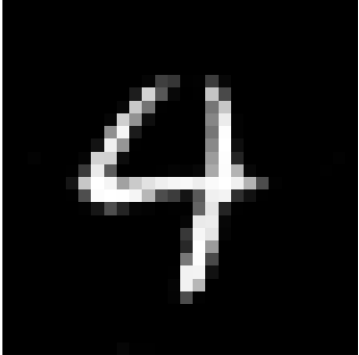


Input Int: 0

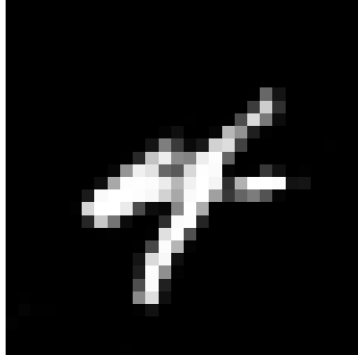


실제로 특정 레이블을 넣어 실행해본 모습이다.
꽤나 그럴듯한 이미지를 생성하지만, 2 처럼 가끔 이상하고 애매한 이미지가 생성되기도 한다.

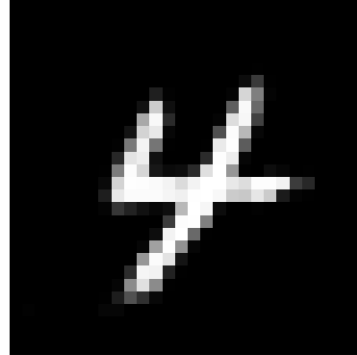
Input Int: 4



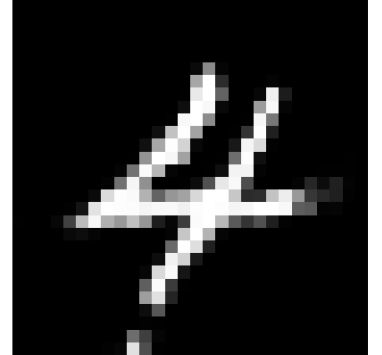
Input Int: 4



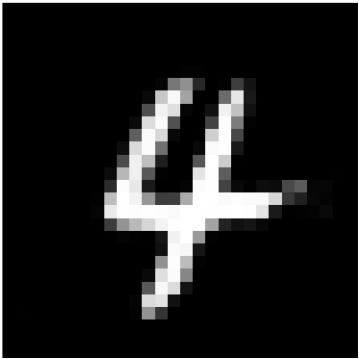
Input Int: 4



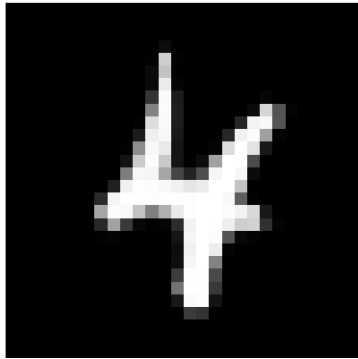
Input Int: 4



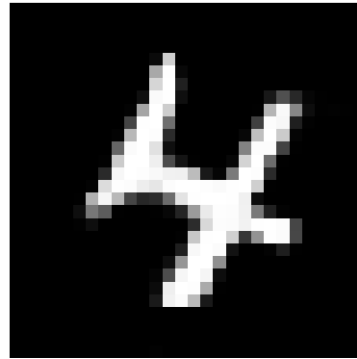
Input Int: 4



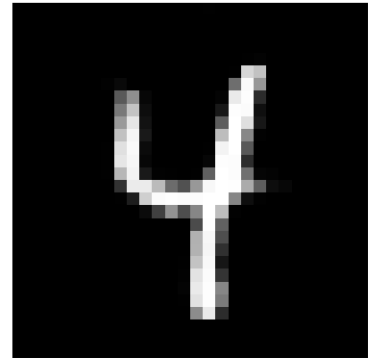
Input Int: 4



Input Int: 4



Input Int: 4



동일한 레이블을 여러번 입력해본 모습이다.
조금 전의 CGAN과 비교해 봐도 특별히 이상한 이미지 없이 잘 생성된 모습이다.

Wasserstein Generative Adversarial Networks (WGAN)

```
clip_value = 0.01  
critic_iterations = 5
```

달라진 부분만 살펴보도록 한다.

WGAN에서는 clip value와 critic의 학습 횟수라는 하이퍼파라미터가 새로 추가된다.

여기서는 critic의 가중치는 -0.01~0.01 사이로 클리핑되며, critic은 generator가 학습되기 전 5번 먼저 학습된다.

```
discriminator_optimizer = tf.keras.optimizers.RMSprop(learning_rate=0.00005)  
generator_optimizer = tf.keras.optimizers.RMSprop(learning_rate=0.00005)
```

옵티마이저로는 RMSProp을 사용한다.

WGAN에서 Momentum 기반의 옵티마이저는 Critic의 Lipschitz 조건을 만족하는 데 어려움이 있을 수 있기 때문에, 공식 논문에서는 RMSProp을 추천한다.

```
def wasserstein_loss(y_true, y_pred):  
    return tf.reduce_mean(y_true * y_pred)
```

Critic의 loss는 아래와 같다.

$$L_D = -\mathbb{E}_{x \sim \mathbb{P}_r} [D(x)] + \mathbb{E}_{z \sim p(z)} [D(G(z))]$$

Critic의 목표는 $D(x)$ 를 최대화, $D(G(z))$ 를 최소화하는 것이다.

Generator의 loss는 아래와 같다.

$$L_G = -\mathbb{E}_{z \sim p(z)} [D(G(z))]$$

Generator의 목표는 $D(G(z))$ 를 최대화하는 것이다.

이러한 관점에서, 이 함수를 살펴보면 아래와 같다.

```
def wasserstein_loss(y_true, y_pred):  
    return tf.reduce_mean(y_true * y_pred)
```

Critic의 loss는 아래와 같다.

$$L_D = -\mathbb{E}_{x \sim \mathbb{P}_r} [D(x)] + \mathbb{E}_{z \sim p(z)} [D(G(z))]$$

Critic의 목표는 $D(x)$ 를 최대화, $D(G(z))$ 를 최소화하는 것이다.

Generator의 loss는 아래와 같다.

$$L_G = -\mathbb{E}_{z \sim p(z)} [D(G(z))]$$

Generator의 목표는 $D(G(z))$ 를 최대화하는 것이다.

이러한 관점에서, 이 함수를 살펴보면 아래와 같다.

진짜 이미지를 학습할 때, y_true는 1로 설정된다.

반면에 가짜 이미지를 학습할 때는 y_true가 -1로 설정된다.

진짜 이미지에 대해서는 $1 * D(x)$ 가 되어 높은 값을 출력하도록 하고, 가짜 이미지에 대해서는 $-1 * D(G(z))$ 가 되어 낮은 값을 출력하도록 학습한다.

따라서 Critic은 진짜 이미지는 더 높은 점수를, 가짜 이미지는 더 낮은 점수를 주도록 학습하게 된다.

이 때, tf.reduce_mean은 텐서의 평균값을 계산하는 함수이다.

y_true와 y_pred는 각각 목표값(진짜/가짜 여부)과 Critic의 예측값을 나타내는데, 이 두 값을 곱하여 진짜 샘플과 가짜 샘플에 대해 각각의 손실을 계산한다.

이 곱셈의 결과는 각 샘플별 손실 값을 의미하며, tf.reduce_mean은 이러한 샘플별 손실 값을 전체 batch에 대해 평균을 구하는 것이다.

즉, 한 batch내의 모든 data point들에 대해 loss를 평가한 후, 이 값들의 평균을 반환한다.

Wasserstein loss함수의 목적은 Wasserstein distance를 구하는(혹은 근사하는) 것이므로, Critic이 출력한 값에 전체 배치에 대한 평균을 계산한다.

이렇게 평균을 사용하면 loss의 크기가 batch size에 의존하지 않게 되어 일정한 학습 속도를 유지할 수 있게 되며,

또한 loss 값의 크기가 크게 변하지 않게 되어 보다 안정적인 학습을 가능케 한다.

```
def train_critic(x_train, y_train):
    for _ in range(critic_iterations):
        discriminator.trainable = True
        # Real 학습
        c = np.random.randint(0, x_train.shape[0], batch_size)
        real = x_train[c]
        real_labels = y_train[c]
        real_labels = tf.keras.utils.to_categorical(real_labels, num_classes)
        d_loss_real = discriminator.train_on_batch([real, real_labels], -np.ones((batch_size, 1)))
        # Fake 학습
        p = np.random.normal(0, 1, (batch_size, noise_dimension))
        fake_labels = np.random.randint(0, num_classes, batch_size)
        fake_labels_one_hot = tf.keras.utils.to_categorical(fake_labels, num_classes)
        fake = generator.predict([p, fake_labels_one_hot])
        d_loss_fake = discriminator.train_on_batch([fake, fake_labels_one_hot], np.ones((batch_size, 1)))
        # Weight clipping
        for layer in discriminator.layers:
            if isinstance(layer, layers.Conv2D) or isinstance(layer, layers.Dense):
                weights = layer.get_weights()
                weights = [np.clip(w, -clip_value, clip_value) for w in weights]
                layer.set_weights(weights)
    return d_loss_real, d_loss_fake
```

Critic을 학습시키는 함수이다.

WGAN에서, Critic은 Generator보다 Critic iteration만큼 더 학습된다.

또한, 모든 레이어의 weigh를 clip value 범위 내로 제한하여 Critic이 Lipschitz 조건을 유지하도록 한다.

Model Architecture

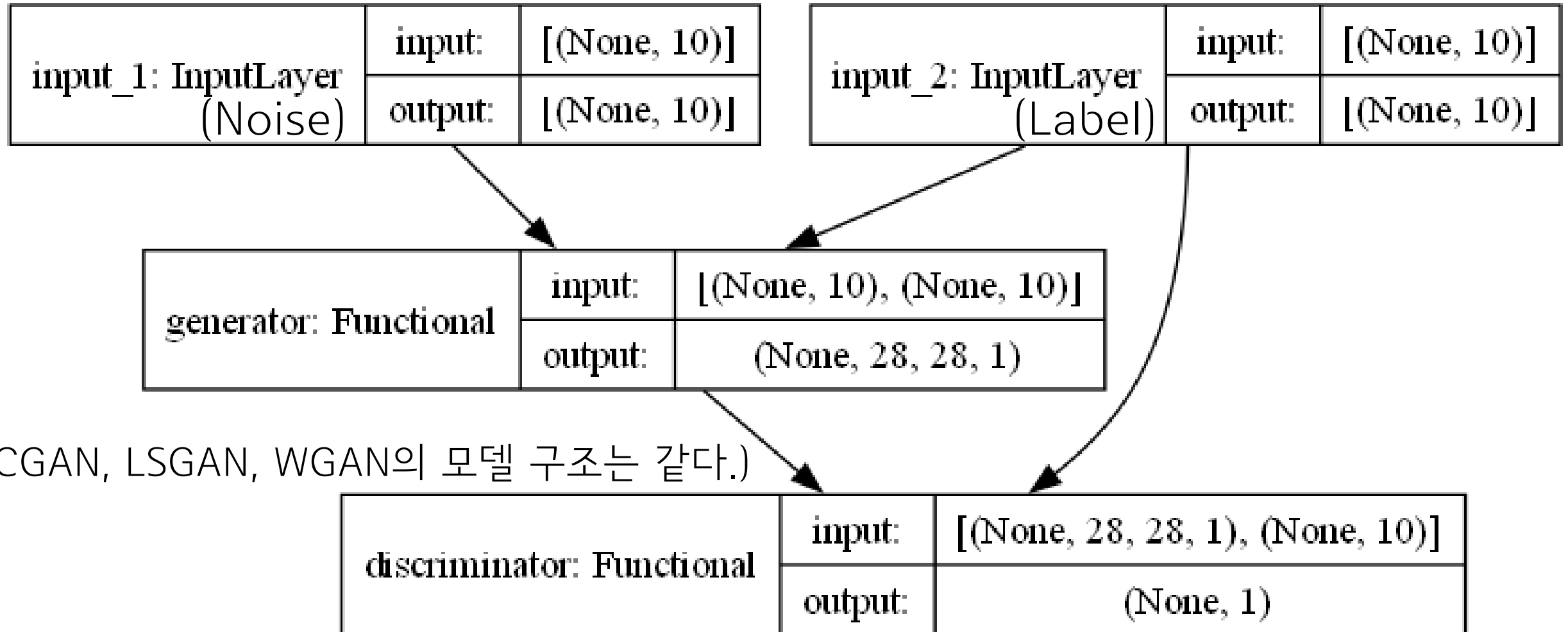
Model: "generator"

Layer (type)	Output Shape	Param #	Connected to
generator_noise_input (InputLayer)	[(None, 10)]	0	
generator_label_input (InputLayer)	[(None, 10)]	0	
concatenate (Concatenate)	(None, 20)	0	generator_noise_input[0][0] generator_label_input[0][0]
dense (Dense)	(None, 12544)	263424	concatenate[0][0]
leaky_re_lu (LeakyReLU)	(None, 12544)	0	dense[0][0]
reshape (Reshape)	(None, 7, 7, 256)	0	leaky_re_lu[0][0]
conv2d_transpose (Conv2DTranspose)	(None, 14, 14, 128)	524416	reshape[0][0]
leaky_re_lu_1 (LeakyReLU)	(None, 14, 14, 128)	0	conv2d_transpose[0][0]
conv2d_transpose_1 (Conv2DTranspose)	(None, 28, 28, 64)	131136	leaky_re_lu_1[0][0]
leaky_re_lu_2 (LeakyReLU)	(None, 28, 28, 64)	0	conv2d_transpose_1[0][0]
conv2d (Conv2D)	(None, 28, 28, 1)	1601	leaky_re_lu_2[0][0]
Total params: 920,577			
Trainable params: 920,577			
Non-trainable params: 0			

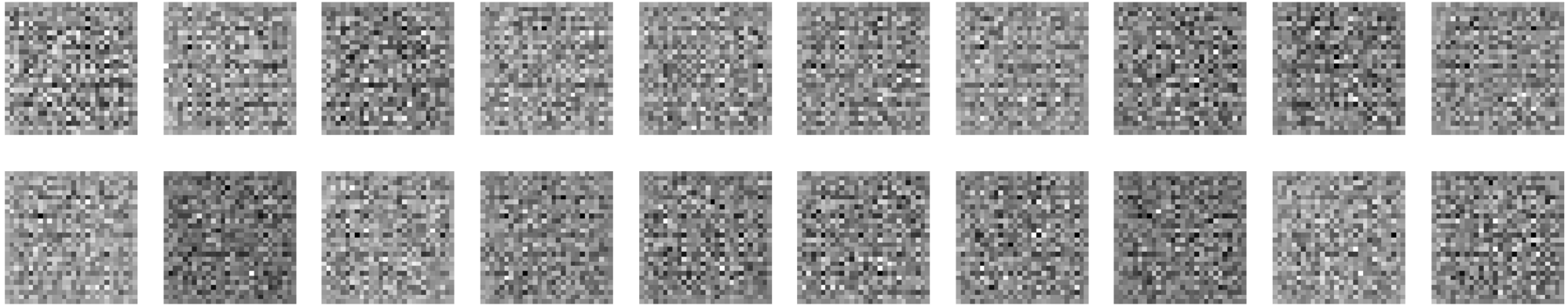
Model: "discriminator"

Layer (type)	Output Shape	Param #	Connected to
discriminator_label_input (InputLayer)	[(None, 10)]	0	
dense_1 (Dense)	(None, 784)	8624	discriminator_label_input[0][0]
discriminator_image_input (InputLayer)	[(None, 28, 28, 1)]	0	
reshape_1 (Reshape)	(None, 28, 28, 1)	0	dense_1[0][0]
concatenate_1 (Concatenate)	(None, 28, 28, 2)	0	discriminator_image_input[0][0] reshape_1[0][0]
conv2d_1 (Conv2D)	(None, 14, 14, 64)	2112	concatenate_1[0][0]
leaky_re_lu_3 (LeakyReLU)	(None, 14, 14, 64)	0	conv2d_1[0][0]
conv2d_2 (Conv2D)	(None, 7, 7, 128)	131200	leaky_re_lu_3[0][0]
leaky_re_lu_4 (LeakyReLU)	(None, 7, 7, 128)	0	conv2d_2[0][0]
conv2d_3 (Conv2D)	(None, 4, 4, 256)	524544	leaky_re_lu_4[0][0]
flatten (Flatten)	(None, 4096)	0	conv2d_3[0][0]
dense_2 (Dense)	(None, 1)	4097	flatten[0][0]
Total params: 670,577			
Trainable params: 670,577			
Non-trainable params: 0			

Model Architecture

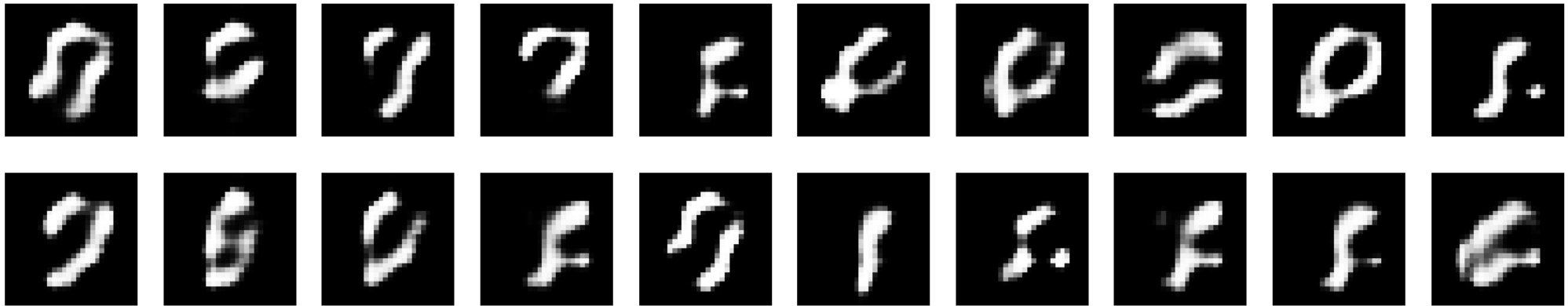


Iteration 0



WGAN의 학습 전, 무작위 노이즈가 입력된 모습

Iteration 1000



Iteration이 1000번 진행된 후이다.
LSGAN 대비 조금 아쉬운 성능이지만 그래도 뭔가 숫자스러운 이미지가 나오긴 했다.

Iteration 2000



Iteration 3000



Iteration 4000



Iteration 5000



5000번 정도 학습하니 드디어 뭔가 숫자스러운 이미지가 나오기 시작했다.

Iteration 6000



Iteration 7000



Iteration 8000



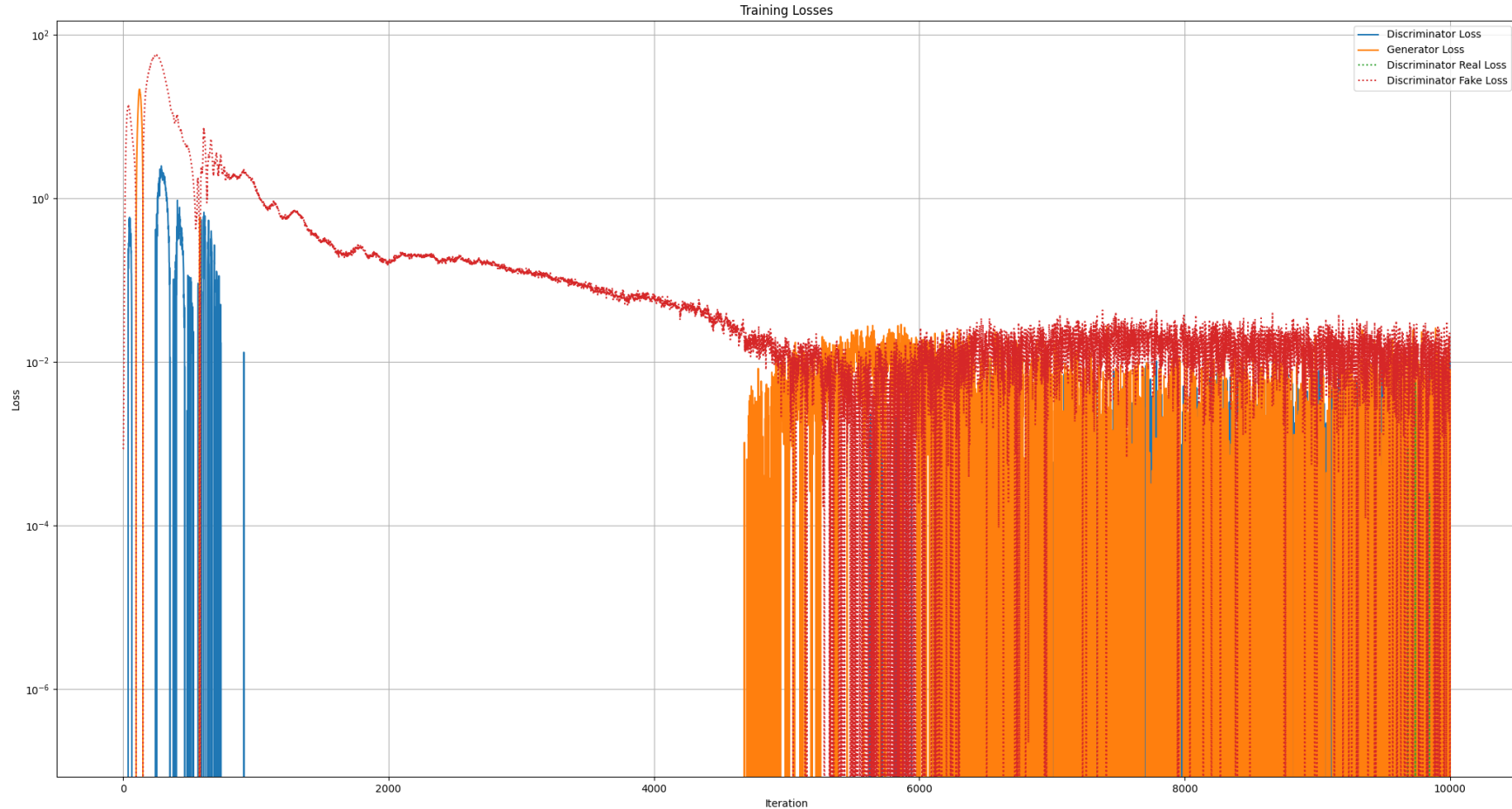
Iteration 9000



Iteration 10000

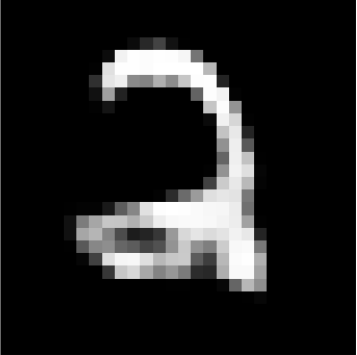


학습을 마친 모습이다.
LSGAN 대비 뭔가 좀 어설피지만 그래도 괜찮게 학습한 것 같다.

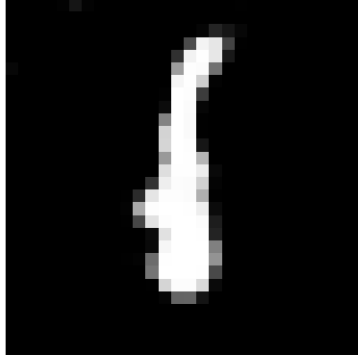


약 5000번째 iteration 부근에서 loss가 요동치는 것을 볼 수 있다.
이는 WGAN 특유의 하이퍼파라미터 민감성 때문이라고 추측되며,
loss가 이렇게 난리가 났는데도 불구하고 생성된 이미지는 비교적 멀쩡하다는 점에서 WGAN은 강력한 구조라 할 수 있을 것 같다.

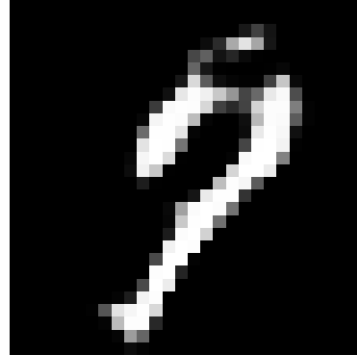
Input Int: 2



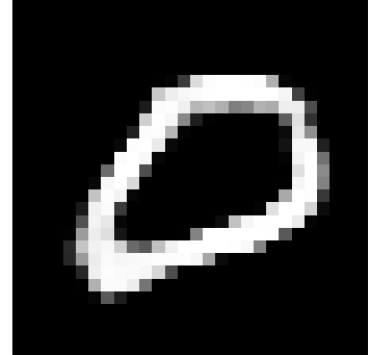
Input Int: 6



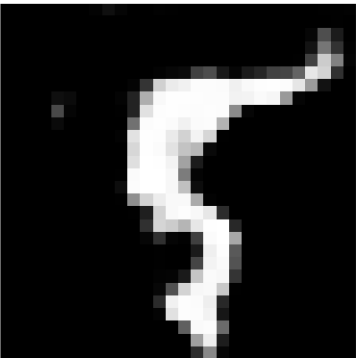
Input Int: 7



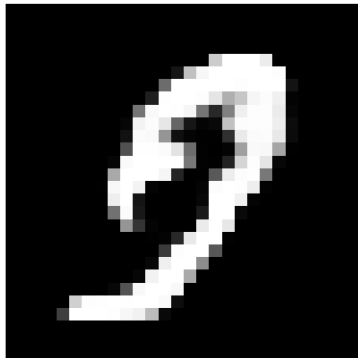
Input Int: 0



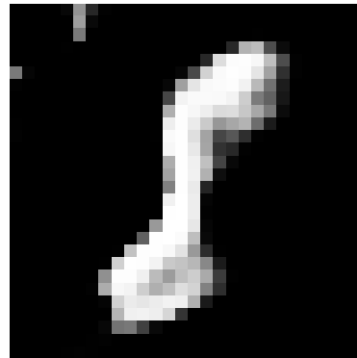
Input Int: 5



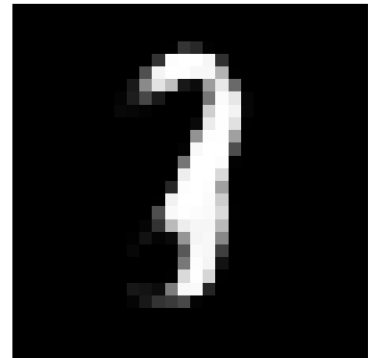
Input Int: 3



Input Int: 8



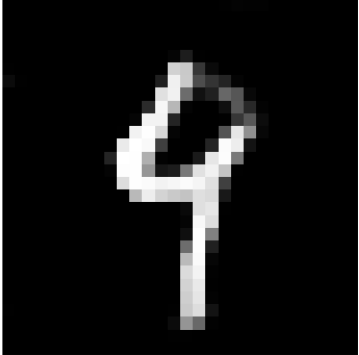
Input Int: 1



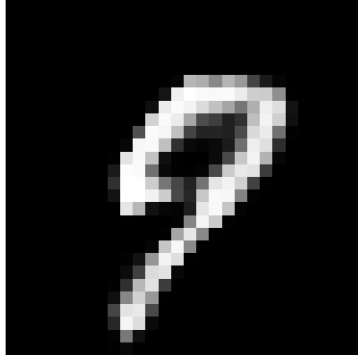
실제로 특정 레이블을 넣어 실행해본 모습이다.

놀랍게도 이전의 LSGAN보다 훨씬 뒤떨어지는 성능인데, 이는 LSGAN보다 훨씬 하이퍼파라미터에 민감한 WGAN의 특성 때문이라고 추측된다.
(clip value, critic iteration 등등)

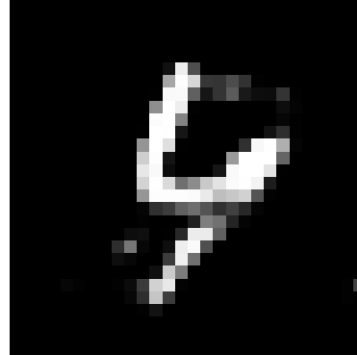
Input Int: 9



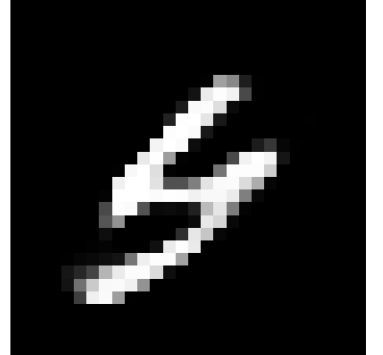
Input Int: 9



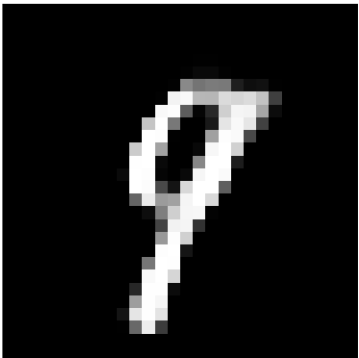
Input Int: 9



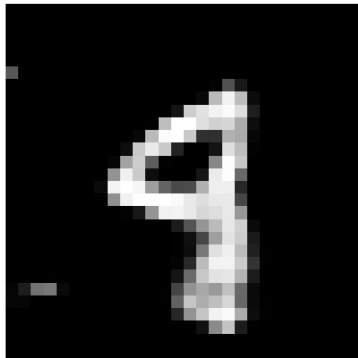
Input Int: 9



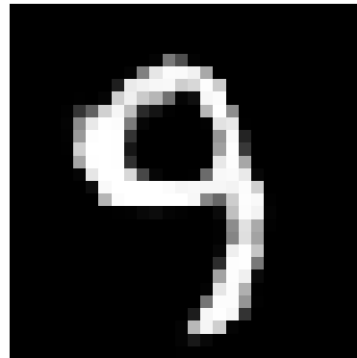
Input Int: 9



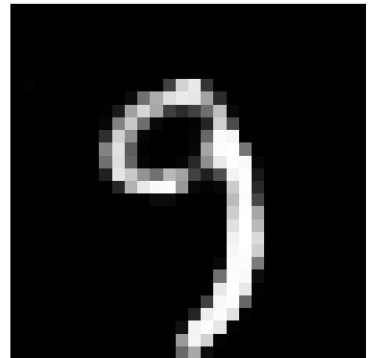
Input Int: 9



Input Int: 9



Input Int: 9



동일한 레이블을 여러번 입력해본 모습이다.

몇몇 이미지들은 잘 생성되었지만, 하이퍼파라미터 설정에 무척 민감한 WGAN의 특성상 괜찮은 이미지의 수율이 LSGAN보다 떨어지는 모습이다.

AE vs VAE vs (LS)GAN

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, Input, Model

# Load and preprocess data
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

noise_dimension = 10
num_classes = 10
batch_size = 64
iterations = 10000
early_stopping = tf.keras.callbacks.EarlyStopping(monitor='loss', patience=10, restore_best_weights=True)
```

여기서부터는 Autoencoder, VAE, GAN의 결과값 비교 분석이다.

GAN 모델 중에서는 가장 괜찮은 성능을 냈던 LSGAN을 사용하며, 오토인코더는 Convolution 레이어를 추가한 버전으로 사용했다.

필요한 라이브러리들을 임포트하고, MNIST를 로드한다.

각 모델들에 필요한 하이퍼파라미터들을 정의하고, early stopping 또한 적용한다.

(Convolutional) Autoencoder

```
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
x_train = np.reshape(x_train, (len(x_train), 28, 28, 1))
x_test = np.reshape(x_test, (len(x_test), 28, 28, 1))
input_img = Input(shape=(28, 28, 1))

x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(input_img)
x = layers.MaxPooling2D((2, 2), padding='same')(x)
x = layers.Conv2D(64, (3, 3), activation='relu', padding='same')(x)
x = layers.MaxPooling2D((2, 2), padding='same')(x)
encoder = Model(inputs=input_img, outputs=x, name='encoder_model')

x = layers.Conv2DTranspose(64, (3, 3), strides=2, activation="relu", padding="same")(x)
x = layers.Conv2DTranspose(32, (3, 3), strides=2, activation="relu", padding="same")(x)
x = layers.Conv2D(1, (3, 3), activation="sigmoid", padding="same")(x)
decoded = x

autoencoder = Model(input_img, decoded)
autoencoder.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.001), loss='binary_crossentropy')
autoencoder.fit(x_train, x_train, epochs=50, batch_size=256, shuffle=True, verbose=1, callbacks=[early_stopping])
```

오토인코더에 Convolutional Layer를 추가한 버전이다.
이미지를 0~1로 정규화한 후, 28, 28, 1로 reshape한다.

인코더와 디코더는 각각 두 개의 Convolution, Transposed Convolution 레이어로 이루어져 있다.

오토인코더는 입력과 출력이 동일한 형태이다.
따라서 입력된 이미지를 복원하는 방식으로 학습한다.

VAE

```
latent_dim = 2
encoder_inputs = tf.keras.Input(shape=(28, 28, 1))
x = layers.Conv2D(32, 3, activation='relu', strides=2, padding='same')(encoder_inputs)
x = layers.Conv2D(64, 3, activation='relu', strides=2, padding='same')(x)
x = layers.Flatten()(x)
x = layers.Dense(16, activation='relu')(x)
z_mean = layers.Dense(latent_dim, name="z_mean")(x)
z_log_var = layers.Dense(latent_dim, name="z_log_var")(x)
z = layers.Lambda(lambda p: p[0] + tf.exp(0.5 * p[1]) * tf.random.normal(tf.shape(p[0])), name="z")([z_mean, z_log_var])
encoder = tf.keras.Model(encoder_inputs, [z_mean, z_log_var, z], name="encoder")

latent_inputs = tf.keras.Input(shape=(latent_dim,))
x = layers.Dense(7 * 7 * 64, activation='relu')(latent_inputs)
x = layers.Reshape((7, 7, 64))(x)
x = layers.Conv2DTranspose(64, 4, activation='relu', strides=2, padding='same')(x)
x = layers.Conv2DTranspose(32, 4, activation='relu', strides=2, padding='same')(x)
decoder_outputs = layers.Conv2DTranspose(1, 3, activation='sigmoid', padding='same')(x)
decoder = tf.keras.Model(latent_inputs, decoder_outputs, name="decoder")
```

VAE의 인코더와 디코더를 정의하는 부분이다.

z_mean은 Latent Space의 평균을, z_log_var는 Latent Space의 로그 분산을 나타낸다.

VAE

```
class VAE(tf.keras.Model):
    def __init__(self, encoder, decoder, **kwargs):
        super(VAE, self).__init__(**kwargs)
        self.encoder = encoder
        self.decoder = decoder
        self.total_loss_tracker = tf.keras.metrics.Mean(name="total_loss")
        self.reconstruction_loss_tracker = tf.keras.metrics.Mean(name="reconstruction_loss")
        self.kl_loss_tracker = tf.keras.metrics.Mean(name="kl_loss")

    @property
    def metrics(self):
        return [
            self.total_loss_tracker,
            self.reconstruction_loss_tracker,
            self.kl_loss_tracker,
        ]

    def train_step(self, data):
        if isinstance(data, tuple):
            data = data[0]

        with tf.GradientTape() as tape:
            z_mean, z_log_var, z = self.encoder(data)
            reconstruction = self.decoder(z)
            reconstruction_loss = tf.reduce_mean(tf.keras.losses.binary_crossentropy(data, reconstruction), axis=(1, 2))
            kl_loss = -0.5 * tf.reduce_mean(tf.reduce_sum(1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var), axis=1))
            total_loss = reconstruction_loss + kl_loss

        grads = tape.gradient(total_loss, self.trainable_variables)
        self.optimizer.apply_gradients(zip(grads, self.trainable_variables))
        self.total_loss_tracker.update_state(total_loss)
        self.reconstruction_loss_tracker.update_state(reconstruction_loss)
        self.kl_loss_tracker.update_state(kl_loss)
        return {
            "loss": self.total_loss_tracker.result(),
            "reconstruction_loss": self.reconstruction_loss_tracker.result(),
            "kl_loss": self.kl_loss_tracker.result(),
        }
```

VAE 클래스를 정의하는 부분으로, [24-2] Chap.8_오토인코더.pdf의 코드를 참고하였다.
각 학습에서 reconstruction loss와 KLD를 계산하여 모델을 학습시킨다.

```
(x_train, _), (x_test, _) = tf.keras.datasets.mnist.load_data()
mnist_digits = np.concatenate([x_train, x_test], axis=0)
mnist_digits = np.expand_dims(mnist_digits, -1).astype("float32") / 255
```

```
vae = VAE(encoder, decoder)
vae.compile(optimizer=tf.keras.optimizers.Adam())
vae.fit(mnist_digits, epochs=50, batch_size=128, verbose=1, callbacks=[early_stopping])
```

VAE

VAE 모델을 실제로 학습하는 부분이며,
early stopping을 이용하여 학습 중 Overfitting을 방지하고 빠른 수렴을 가능하게 한다,

```
x_train = (x_train / 127.5) - 1
x_test = (x_test / 127.5) - 1
x_train = x_train.reshape(len(x_train), 28, 28, 1)
x_test = x_test.reshape(len(x_test), 28, 28, 1)

def build_generator():
    noise = Input(shape=(noise_dimension,), name='generator_noise_input')
    label = Input(shape=(num_classes,), name='generator_label_input')
    x = layers.Concatenate()([noise, label])
    x = layers.Dense(7 * 7 * 256)(x)
    x = layers.LeakyReLU(alpha=0.2)(x)
    x = layers.Reshape((7, 7, 256))(x)
    x = layers.Conv2DTranspose(128, (4, 4), strides=(2, 2), padding='same')(x)
    x = layers.LeakyReLU(alpha=0.2)(x)
    x = layers.Conv2DTranspose(64, (4, 4), strides=(2, 2), padding='same')(x)
    x = layers.LeakyReLU(alpha=0.2)(x)
    x = layers.Conv2D(1, kernel_size=5, padding='same', activation='tanh')(x)
    model = Model([noise, label], x, name='generator')
    return model
```

LSGAN 모델의 입력을 -1~1로 정규화하여 LSGAN의 tanh를 거친 출력과 동일하도록 한다.

이후 Generator를 정의한다.

입력으로는 노이즈와 one-hot encoding된 레이블을 입력받으며, tanh로 -1~1 사이의 값을 출력한다.

LSGAN

```
def build_discriminator():
    img = Input(shape=(28, 28, 1), name='discriminator_image_input')
    label = Input(shape=(num_classes,), name='discriminator_label_input')
    label_embedding = layers.Dense(28 * 28)(label)
    label_embedding = layers.Reshape((28, 28, 1))(label_embedding)
    x = layers.Concatenate()([img, label_embedding])
    x = layers.Conv2D(64, (4, 4), strides=(2, 2), padding='same')(x)
    x = layers.LeakyReLU(alpha=0.2)(x)
    x = layers.Conv2D(128, (4, 4), strides=(2, 2), padding='same')(x)
    x = layers.LeakyReLU(alpha=0.2)(x)
    x = layers.Conv2D(256, (4, 4), strides=(2, 2), padding='same')(x)
    x = layers.Flatten()(x)
    x = layers.Dropout(0.3)(x)
    x = layers.Dense(1)(x)
    model = Model([img, label], x, name='discriminator')
    return model

generator = build_generator()
discriminator = build_discriminator()

def least_square_loss_discriminator(y_true, y_pred):
    return 0.5 * tf.reduce_mean(tf.square(y_true - y_pred))
def least_square_loss_generator(y_true, y_pred):
    return 0.5 * tf.reduce_mean(tf.square(y_true - y_pred))
```

LSGAN의 판별자를 정의하고, 생성자와 판별자를 생성한다.
LSGAN에서의 loss를 least square 방식으로 정의한다.

```
discriminator.compile(loss=least_square_loss_discriminator, optimizer=tf.keras.optimizers.Adam(learning_rate=0.0002, beta_1=0.5))
discriminator.trainable = False
gan_noise = Input(shape=(noise_dimension,))
gan_label = Input(shape=(num_classes,))
fake_image = generator([gan_noise, gan_label])
gan_output = discriminator([fake_image, gan_label])
gan = Model([gan_noise, gan_label], gan_output, name='gan')
gan.compile(loss=least_square_loss_generator, optimizer=tf.keras.optimizers.Adam(learning_rate=0.0002, beta_1=0.5))

def train_discriminator(x_train, y_train):
    discriminator.trainable = True
    # Real 학습
    c = np.random.randint(0, x_train.shape[0], batch_size)
    real = x_train[c]
    real_labels = y_train[c]
    real_labels = tf.keras.utils.to_categorical(real_labels, num_classes)
    d_loss_real = discriminator.train_on_batch([real, real_labels], np.ones((batch_size, 1)))
    # Fake 학습
    p = np.random.normal(0, 1, (batch_size, noise_dimension))
    fake_labels = np.random.randint(0, num_classes, batch_size)
    fake_labels_one_hot = tf.keras.utils.to_categorical(fake_labels, num_classes)
    fake = generator.predict([p, fake_labels_one_hot])
    d_loss_fake = discriminator.train_on_batch([fake, fake_labels_one_hot], np.zeros((batch_size, 1)))
    return d_loss_real, d_loss_fake

def train_generator():
    discriminator.trainable = False
    p = np.random.normal(0, 1, (batch_size, noise_dimension))
    random_labels = np.random.randint(0, num_classes, batch_size)
    random_labels_one_hot = tf.keras.utils.to_categorical(random_labels, num_classes)
    g_loss = gan.train_on_batch([p, random_labels_one_hot], np.ones((batch_size, 1)))
    return g_loss
```

생성자와 판별자를 결합하여 LSGAN 모델을 만들고, 판별자와 생성자의 학습 함수를 정의한다.

```
for i in range(iterations + 1):
    d_loss_real, d_loss_fake = train_discriminator(x_train, y_train)
    d_loss = 0.5 * np.add(d_loss_real, d_loss_fake)
    g_loss = train_generator()

    if i % 100 == 0:
        print(f"Iteration {i} [D loss: {d_loss:.4f}] [G loss: {g_loss:.4f}] (Real: {d_loss_real:.4f}, Fake: {d_loss_fake:.4f})")
```

LSGAN

이는 LSGAN의 학습 루프이며, 총 10000번의 iteration 동안 100번마다 한 번씩 loss를 출력하도록 하였다.

```
def generate_comparison_images():
    plt.figure(figsize=(15, 15))

    # AE(CNN)
    ae_images = autoencoder.predict(x_test[:10])
    for i, image in enumerate(ae_images):
        plt.subplot(6, 5, i + 21)
        plt.imshow(image.reshape(28, 28), cmap='gray')
        plt.title("AE")
        plt.axis('off')

    # VAE
    z_sample = np.random.normal(0, 1, (10, latent_dim))
    vae_images = decoder.predict(z_sample)
    for i, image in enumerate(vae_images):
        plt.subplot(6, 5, i + 1)
        plt.imshow(image.reshape(28, 28), cmap='gray')
        plt.title("VAE")
        plt.axis('off')

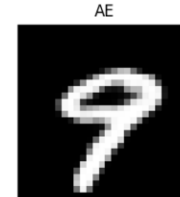
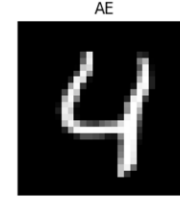
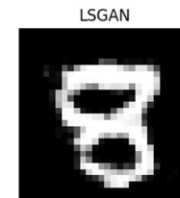
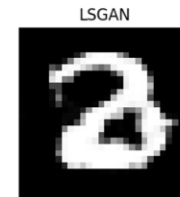
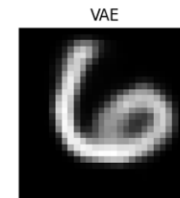
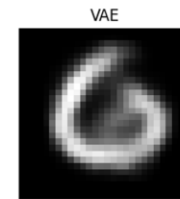
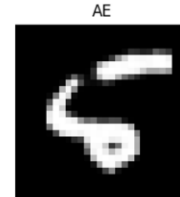
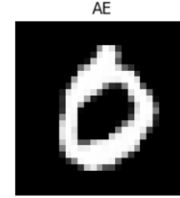
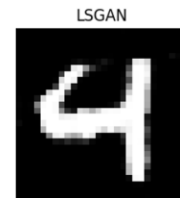
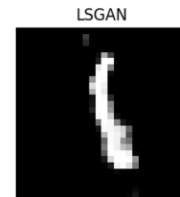
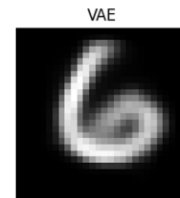
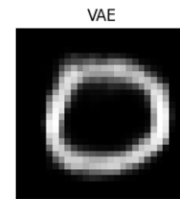
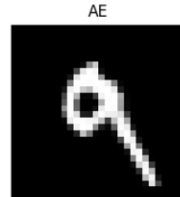
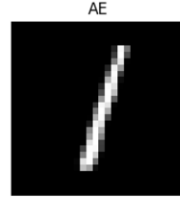
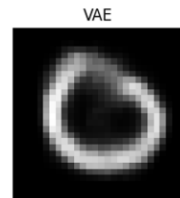
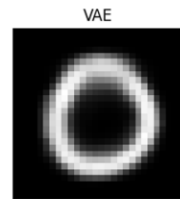
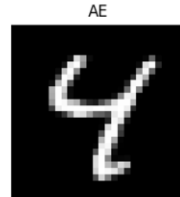
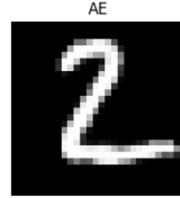
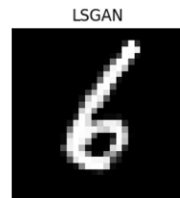
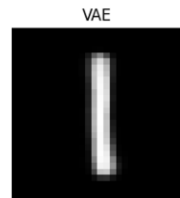
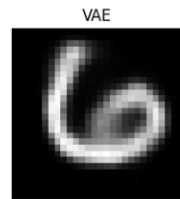
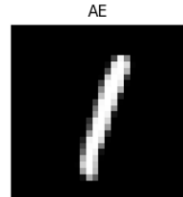
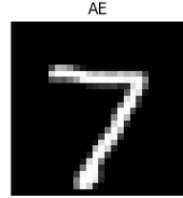
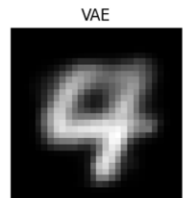
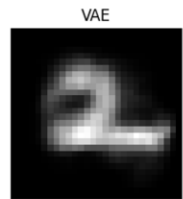
    # LSGAN
    noise = np.random.normal(0, 1, (10, noise_dimension))
    random_labels = np.random.randint(0, num_classes, 10)
    random_labels_one_hot = tf.keras.utils.to_categorical(random_labels, num_classes)
    wgan_images = generator.predict([noise, random_labels_one_hot])
    for i, image in enumerate(wgan_images):
        plt.subplot(6, 5, i + 11)
        plt.imshow(image.reshape(28, 28), cmap='gray')
        plt.title("LSGAN")
        plt.axis('off')

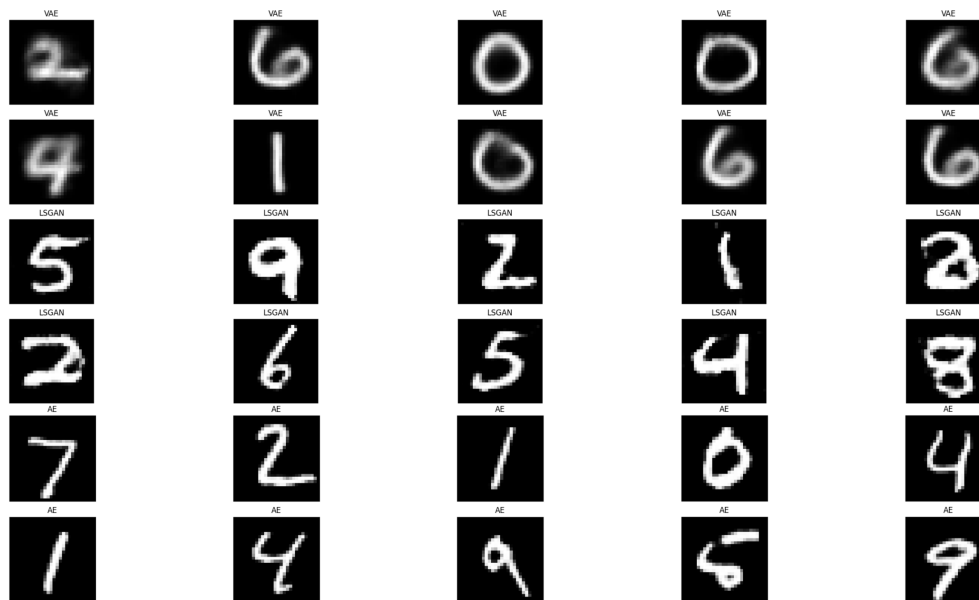
    plt.tight_layout()
    plt.show()

generate_comparison_images()
```

최종적으로, AE, VAE, LSGAN에서 생성된 이미지를 비교한다.

AE에서는 입력 이미지를 복원한 이미지가, VAE에서는 Latent Space에서 샘플링한 값을 기반으로 생성된 이미지가, LSGAN은 입력 레이블에 따른 이미지가 생성될 것이다.





VAE는 숫자가 나타났으나 흐릿하게 표현되었다.

이는 VAE가 잠재 공간에서 샘플링한 벡터를 기반으로 이미지를 생성하기 때문에, 생성된 이미지가 평균적 형태로 나타나는 경향이 있기 때문이다.
이러한 VAE의 특성으로 인해, 생성 이미지의 다양성은 보장할 수 있으나 출력 품질은 상대적으로 떨어진다.

LSGAN은 출력 이미지가 매우 선명하고 명확하다.

일부 이미지(a처럼 생긴 이미지. 아마 3인 것 같다.)는 비정상적이나 다른 이미지들은 매우 잘 생성되었다.
해당 비정상적인 이미지들은 학습이 충분치 않았거나 판별자가 완벽하게 학습되지 않았기 때문이라고 추측된다.
셋 중 가장 발전된 방법이기에, 실제 데이터와 가장 유사한 이미지를 생성한다.

AE는 전체적으로 입력을 잘 복원하였으나, 이미지에 약간의 뭉개짐 현상이 관측된다.

loss로 binary CEE를 사용하였기 때문에 전체적으로는 잘 복원되었으나, (LS)GAN 보다는 덜 선명하게 생성되었음을 볼 수 있다.
이러한 특성때문에, 단순한 복원 작업에는 AE가 가장 적합하다.

03

트랜스포머

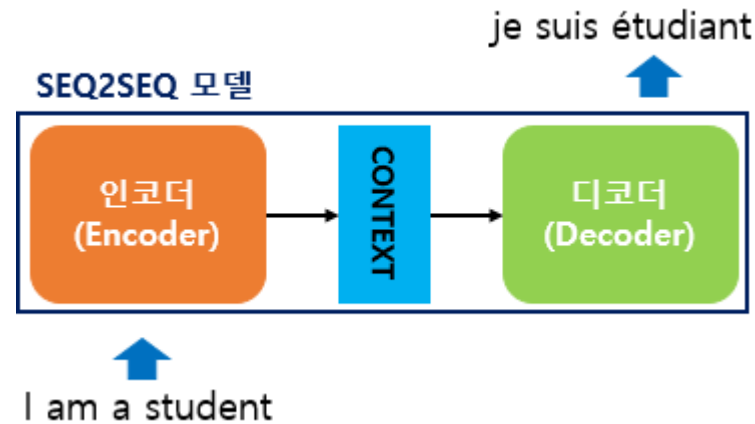
Transformer

여기서부터는 추가적으로 공부한 내용을 정리했습니다.

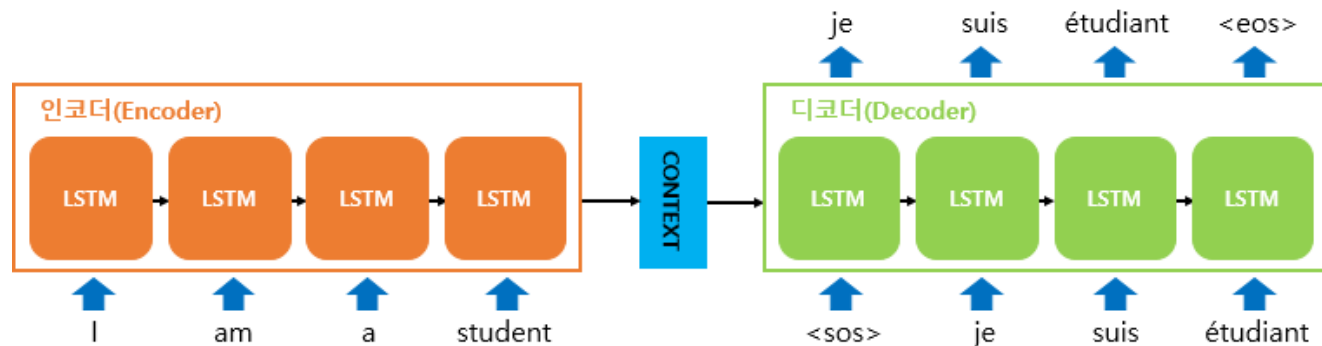
Seq2Seq Sequence to Sequence

- 입력 시퀀스를 받아 출력 시퀀스를 생성하는 딥러닝 아키텍처
- 인코더는 입력 시퀀스를 받아 이를 고정된 길이의 벡터인 context vector로 변환, 디코더는 인코더에서 생성된 context vector 기반으로 출력 시퀀스 생성 (시퀀스 입력 => 시퀀스 출력: 시퀀스 to 시퀀스)

1. 인코더는 입력 시퀀스를 순차적으로 처리하여 최종 은닉 상태를 생성
2. 이 은닉 상태는 디코더로 전달되어 초기 상태로 사용됨
3. 디코더는 이전에 생성된 출력을 입력으로 받아 다음 출력을 예측하며, 이를 반복하여 전체 출력 시퀀스 생성



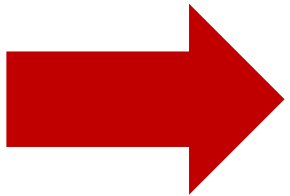
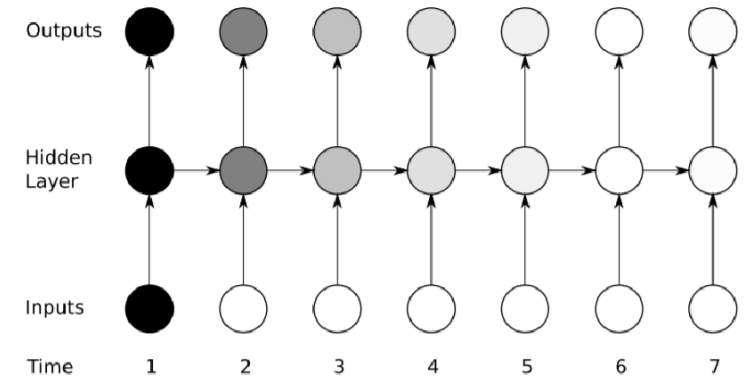
Seq2Seq의 구조



자세한 Seq2Seq의 구조

Seq2Seq의 문제 Problem of Sequence to Sequence

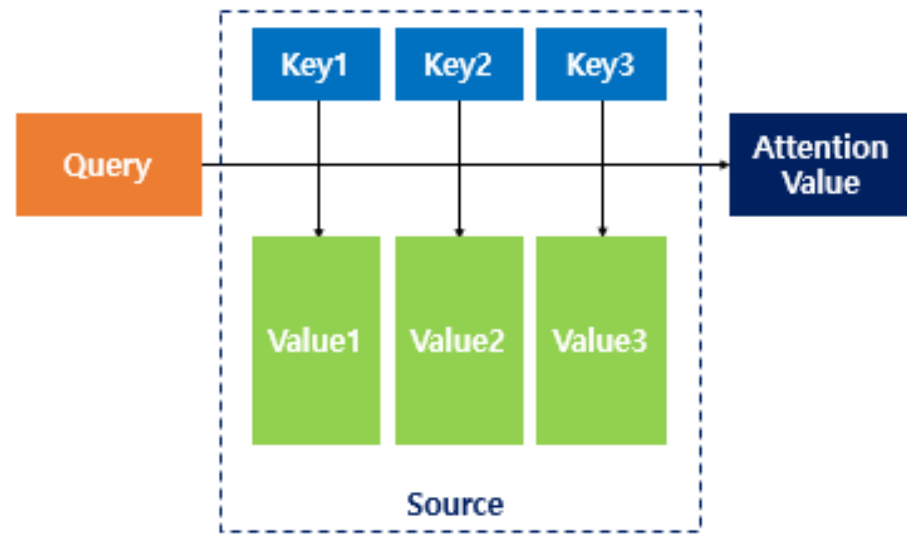
- 입력 시퀀스를 고정된 벡터로 압축
=> 정보 손실 발생
- RNN 기반의 구조
=> Vanishing Gradient Problem



Attention Mechanism의 등장

Attention

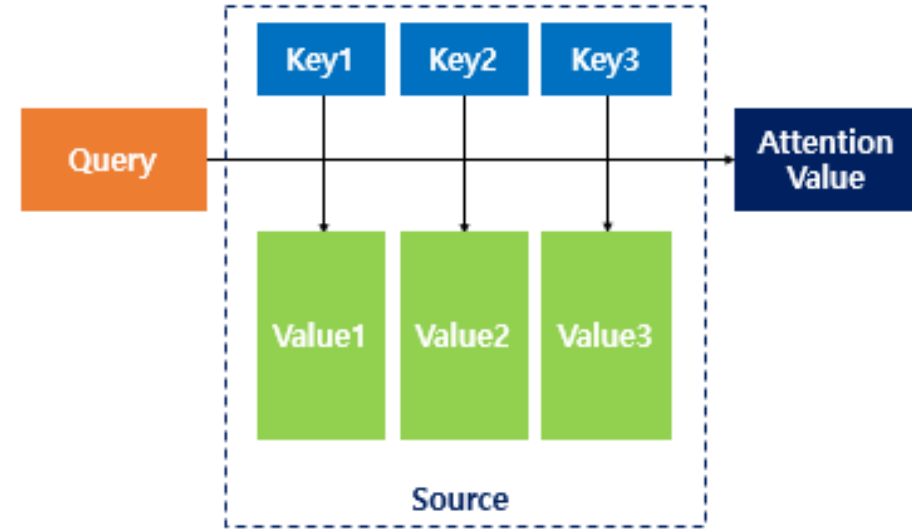
- 디코더에서 출력을 예측할 때, 인코더로 들어온 전체 정보를 다시 한번 확인
이 때, 해당 출력과 가장 연관이 높은 부분을 Attention해서 확인
- Query, Key, Value로 구성됨
 - Query(Q): 모델이 알고자 하는 정보에 대한 질문을 나타내며,
디코더의 t 시점의 Hidden State로 표현됨
 - Key(K): 모든 시점에 대한 인코더 셀의 Hidden State들
 - Value(V): 모든 시점에 대한 인코더 셀의 Hidden State들 (Key와 동일)
- 이후, 쿼리와 각 키 사이의 유사도를 계산하여 Attention Score 산출



Attention의 구성

Attention

1. Query와 각 Key 사이의 유사도 계산
=> Attention Score
2. Attention Score에 Softmax를 취해 Attention weight 생성.
각 입력 요소의 중요도를 Probability Distribution 형태로 나타낼 수 있음
=> Attention Distribution
3. Attention Distribution과 각 Value를 Weighted Sum 으로 계산
=> Attention Value



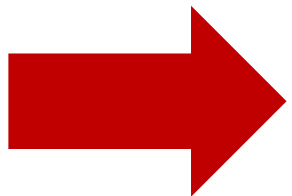
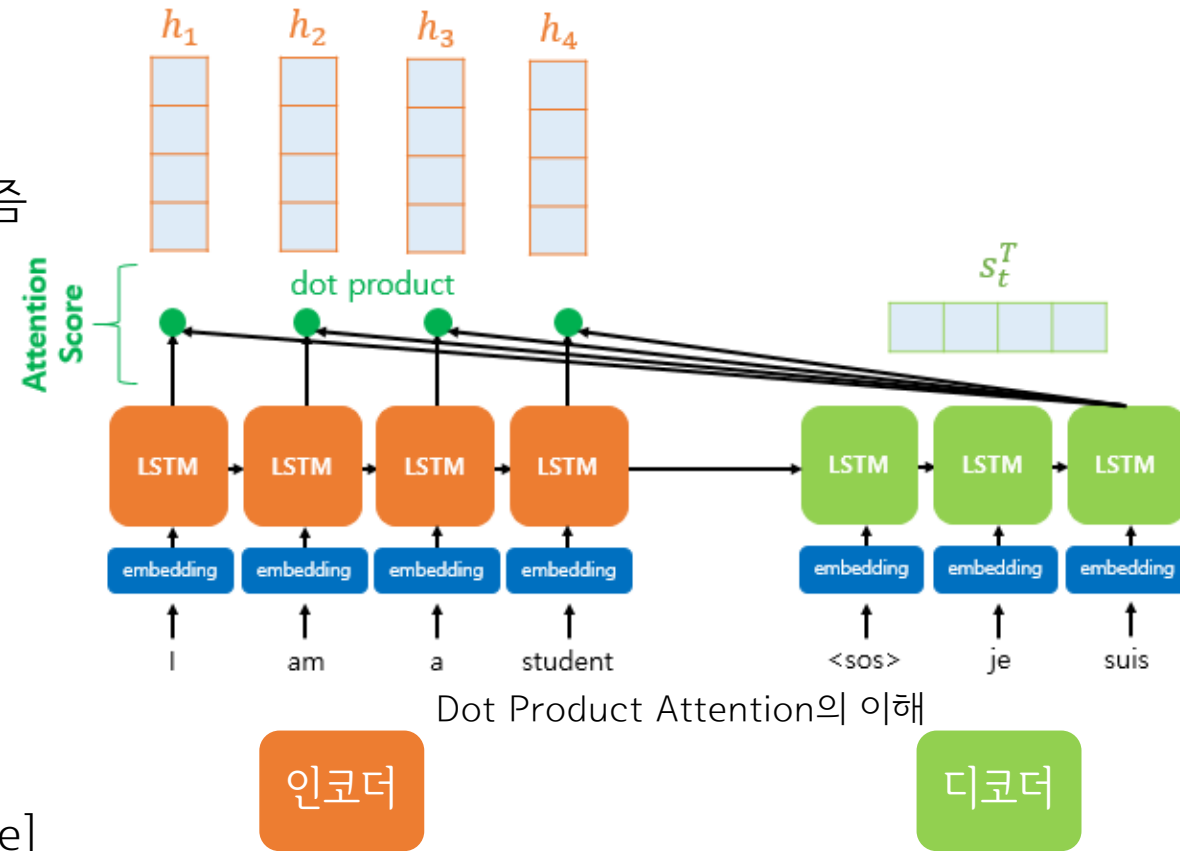
Attention의 구성

Dot-Product Attention

- 쿼리와 키 사이의 유사도를 내적을 통해 계산하는 어텐션 매커니즘 (Luong Attention 이라고도 부름)
- Q : t 시점의 디코더 셀의 Hidden State
 $\Rightarrow s_t$
- K : 모든 시점의 인코더 셀의 Hidden State
 $\Rightarrow [h_1 h_2 h_3 h_4 \dots]$
- V : 모든 시점의 인코더 셀의 Hidden State (키와 동일)
 $\Rightarrow [h_1 h_2 h_3 h_4 \dots]$

[목표: t 시점에서 단어 출력하기]

[입력: $t-1$ 시점에서 출력된 단어와 Hidden State, Attention Value]



앞으로의 과정은 Attention Value를 구하는 과정

Dot-Product Attention

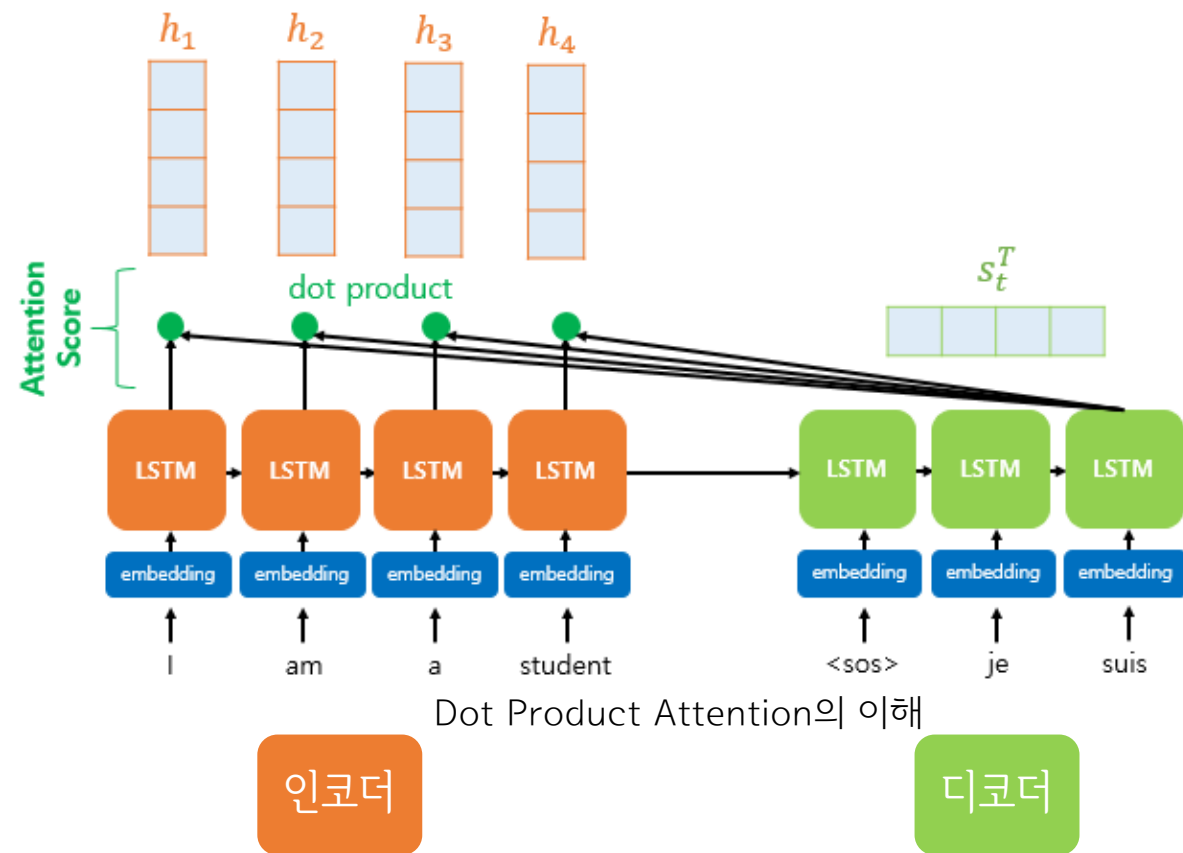
Attention Score

Attention Score: 현재 디코더의 시점 t 에서 출력을 예측할 때, 인코더의 모든 Hidden State 각각이 디코더의 현재 시점의 Hidden State와 얼마나 유사한지에 대한 값

여기서는 이 값을 구하기 위해 디코더의 Hidden State를 Transpose하고 인코더의 각 Hidden State와 dot-product 수행 (Attention Score 값은 Scalar)

$$e^t = [s_t^T h_1, s_t^T h_2, s_t^T h_3, s_t^T h_4, \dots, s_t^T h_i],$$

 e^t 는 Attention Score를 모아놓은 벡터를 뜻한다.
(Attention Score는 스칼라이므로, 모아놓으면 벡터가 된다.)



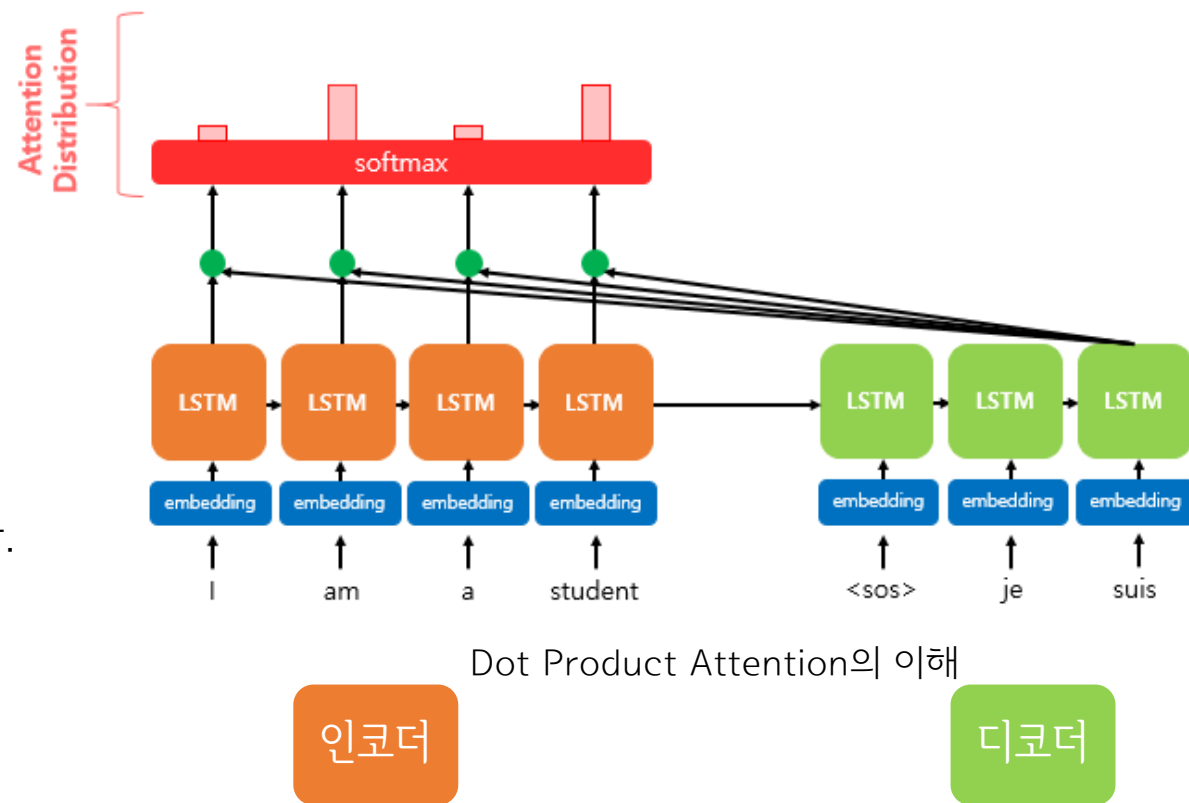
Dot-Product Attention

Attention Distribution

아까 구한 e^t 에 Softmax를 적용하여
모든 값을 합하면 1이 되는 Probability Distribution을 얻어냄
⇒ Attention Distribution

(확률 분포를 얻어내야 하는데, 확률 분포를 전체 구간에서 적분하면 1이다.
따라서 벡터에 softmax를 적용하면 확률 분포를 구할 수 있다.)

Attention Distribution의 각 값은 Attention Weight라 하며,
 t 시점에서의 Attention Distribution을 α^t 라 하면
 $\alpha^t = \text{softmax}(e^t)$ 이다.



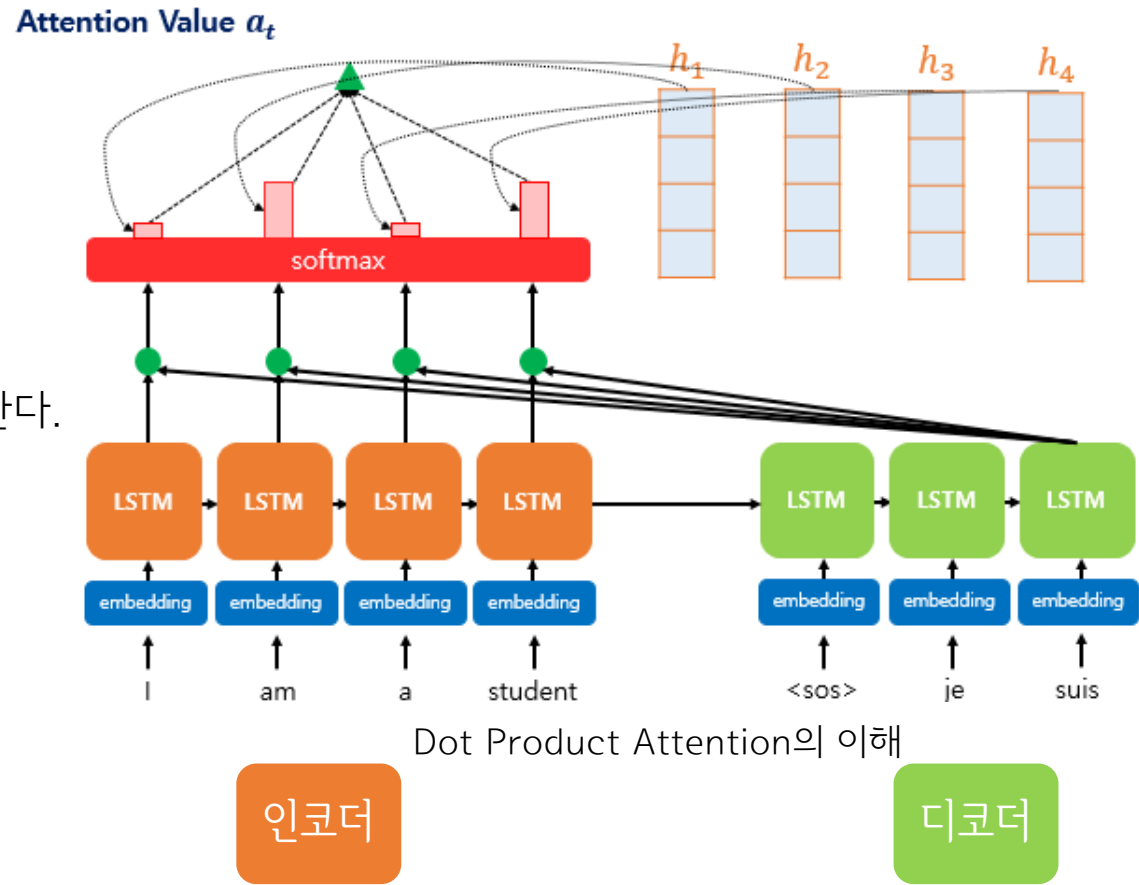
Dot-Product Attention

Attention Value

각 인코더의 Hidden State와 Attention Weight를 곱하고, 그 값을 모두 더한다.
t 시점에서의 Attention Value를 a^t 라 하면

$$a^t = \sum_{i=1}^N \alpha_i^t h_i \circ |\Box|.$$

Attention Value는 인코더의 문맥 정보를 포함하여 Context Vector라고도 불린다.



Dot Product Attention의 이해

인코더

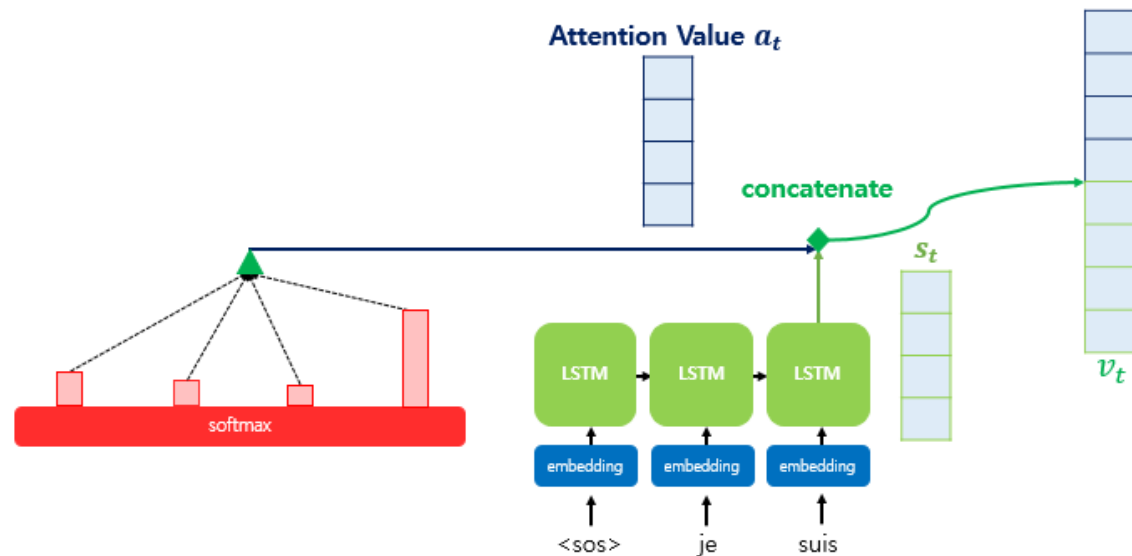
디코더

Dot-Product Attention

Concatenate

구한 Attention Value와 디코더의 Hidden State를 concatenate 한다.

이 벡터를 디코더가 예측할 때의 입력으로 사용함으로써,
인코더로부터 입력받은 정보를 모두 보고 출력을 생성하게 된다.



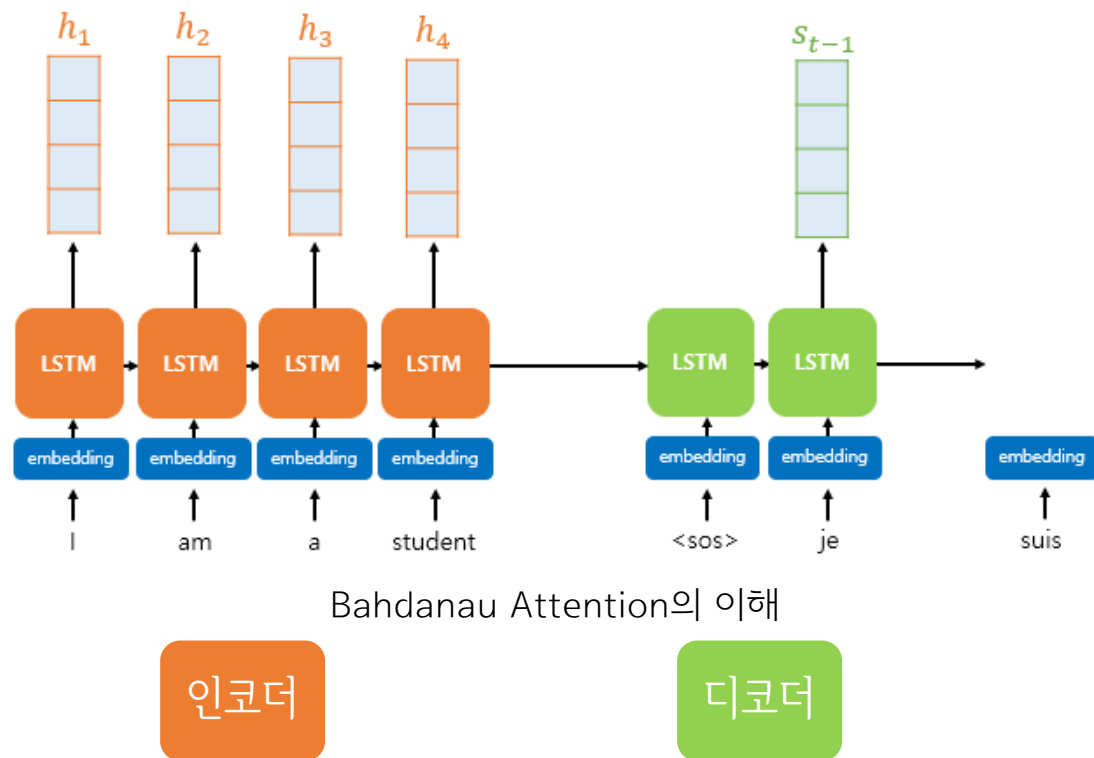
Dot Product Attention의 이해

Bahdanau Attention

Q, K, V 정의

- Q : $t-1$ 시점의 디코더 셀의 Hidden State
 $\Rightarrow s_{t-1}$
- K : 모든 시점의 인코더 셀의 Hidden State
 $\Rightarrow [h_1 h_2 h_3 h_4 \dots]$
- V : 모든 시점의 인코더 셀의 Hidden State (키와 동일)
 $\Rightarrow [h_1 h_2 h_3 h_4 \dots]$

바다나우 어텐션에서는 Q 가 t 시점이 아닌 $t-1$ 시점의 Hidden State이다.



Bahdanau Attention

Attention Score

우선, 인코더와 디코더의 Hidden State의 차원은 동일하다고 가정한다.

W_a, W_b, W_c 는 학습이 가능한 어떠한 weight matrix라 하자.
이 때, 각 행렬들의 크기는 아래와 같다.

W_a : (attn_dim by 1)

W_b : (attn_dim by 디코더의 Hidden State의 차원)

W_c : (attn_dim by 인코더의 Hidden State의 차원)

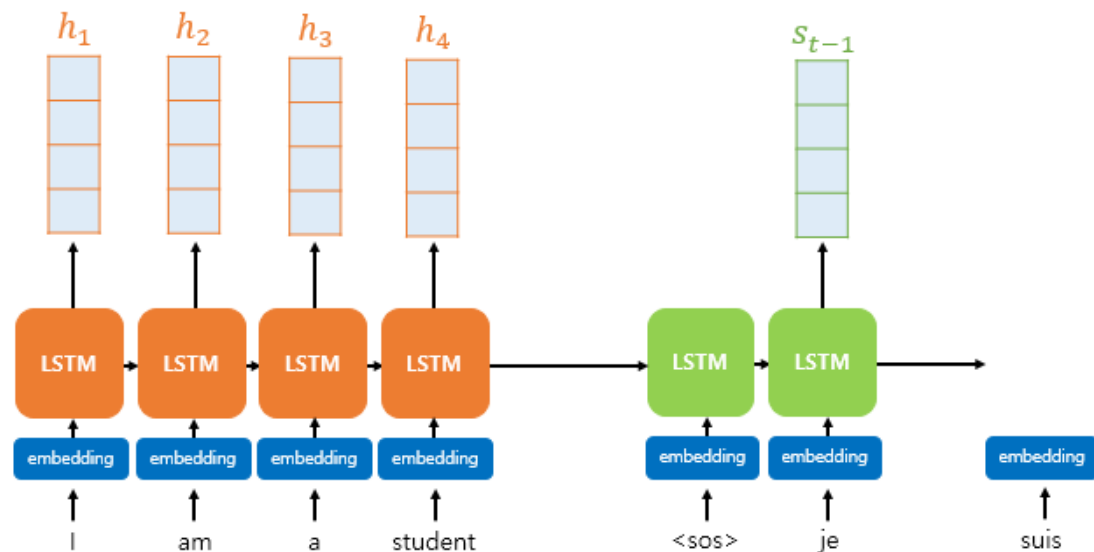
이 때, attn_dim은 어텐션 메커니즘의 내부 차원이며, 하이퍼파라미터이다.

벡터 $h_1, h_2, h_3, \dots$ 들을 모두 합쳐 matrix H 라 하자.

이 때, 바다나우 어텐션에서의 Score는 아래와 같이 계산된다.

$$score(s_{t-1}, h_i) = W_a^T \tanh(W_b s_{t-1} + W_c H)$$

최종적으로 얻어진 Attention Score Vector e^t 는 이렇게 생겼다.



Bahdanau Attention의 이해

인코더

디코더

Attention Score



$h_1 \ h_2 \ h_3 \ h_4$

Bahdanau Attention

Attention Distribution

아까 구한 Attention Score Vector e^t 에 아까와 동일하게 Softmax를 적용하여 총 합이 1이 되는 Probability Distribution을 얻어낸다.

이전과 동일하게 Attention Distribution의 각 값은 Attention Weight라 한다.

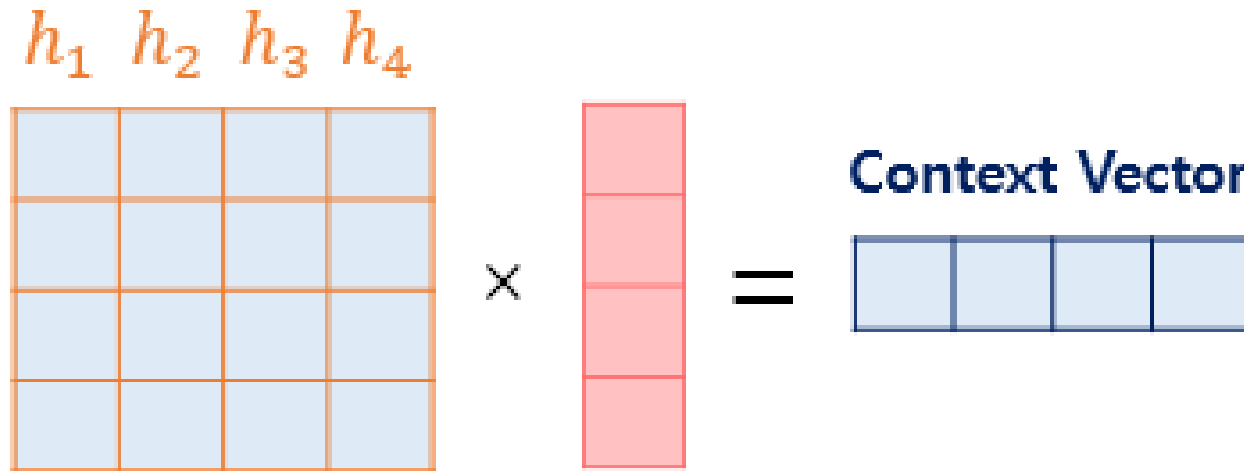
$$\text{softmax} \left(\begin{array}{c} \text{Attention Score} \\ \text{[Green Bar]} \\ h_1 \quad h_2 \quad h_3 \quad h_4 \end{array} \right) = \begin{array}{c} \text{Attention} \\ \text{Distribution} \\ \text{[Pink Bar]} \end{array}$$

Bahdanau Attention의 이해

Bahdanau Attention

Attention Value

인코더의 Hidden State와 Attention Distribution을 Weighted Sum 한다.
크기를 맞추기 위해 Attention Distribution Vector를 Transpose 하여 연산한다.
이렇게 하여 얻어낸 벡터는 인코더로 들어온 정보들의 문맥을 포함하고 있다고 하여 Context Vector 라고도 불린다.



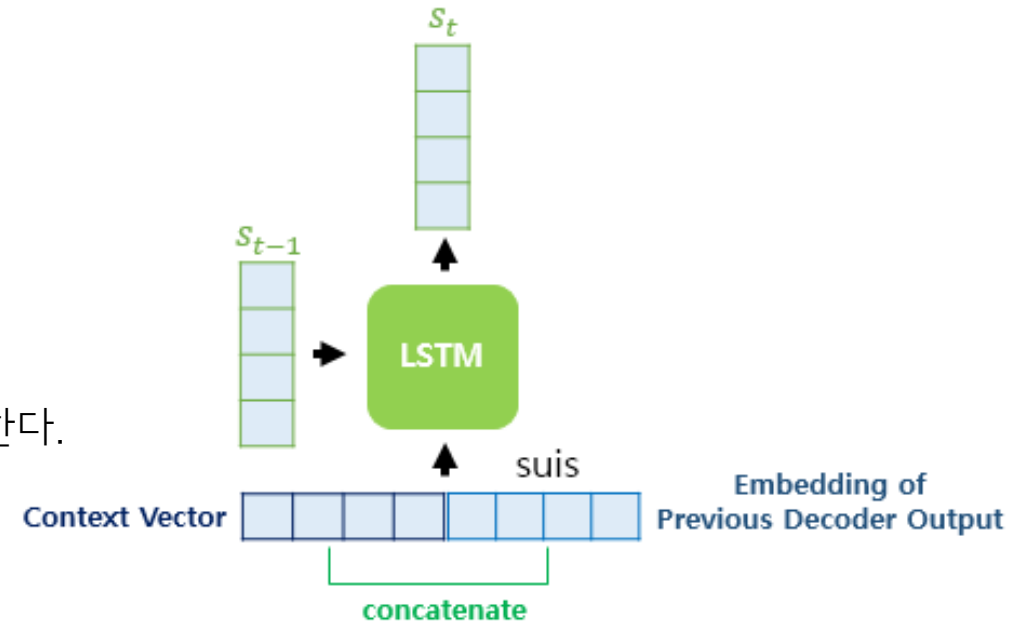
Bahdanau Attention의 이해

Bahdanau Attention

Concatenate

앞서 생성된 Context Vector와 이전 시점의 출력인 벡터를 Concatenate 한다.

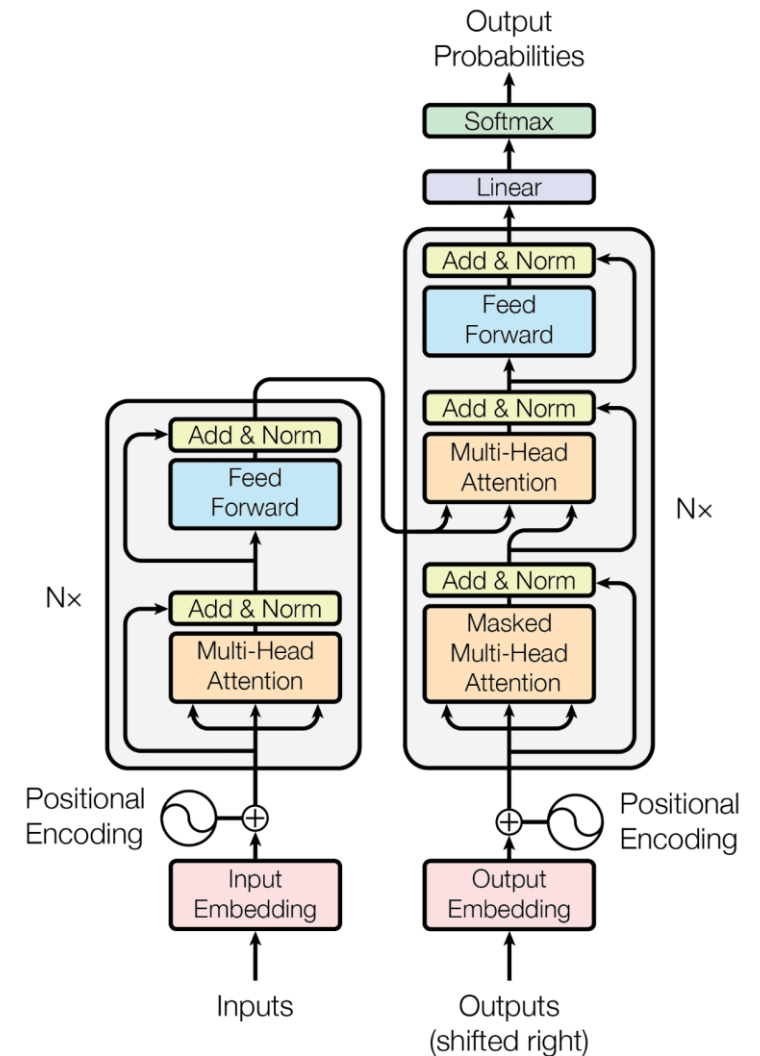
그 후, 모델에 이전 시점의 출력과 Context Vector를 모두 입력하여 예측값을 생성한다.



Bahdanau Attention의 이해

Attention Is All You Need

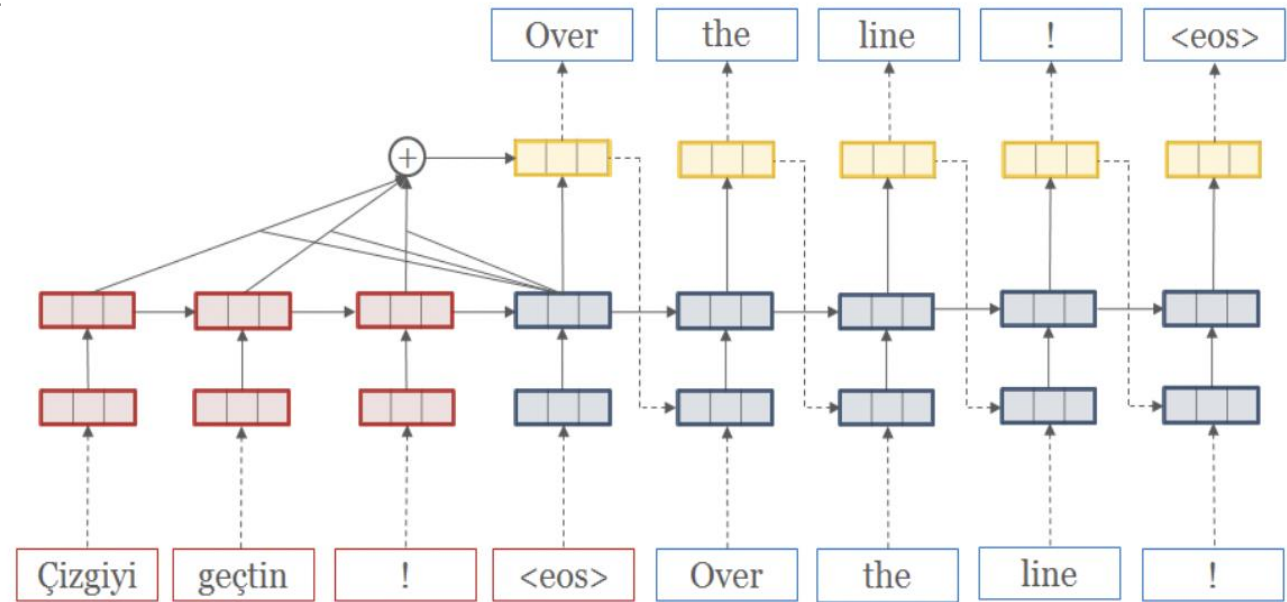
- 2017년 발표된 Transformer를 소개한 논문
- 기존의 시퀀스 변환 모델들은 순차적 처리라는 구조적 한계로 인하여 병렬화가 어렵고, 긴 시퀀스를 처리할 때 성능이 저하됨
- 이를 해결하기 위해 RNN, CNN 등을 사용하지 않은 Transformer를 제안
- En-De, En-Fr 번역 실험에서 높은 성능을 보임
- 이후 NLP, 컴퓨터 비전 등의 분야에 널리 응용되었으며, BERT(Bidirectional Encoder Representations from Transformers), GPT(Generative Pre-trained Transformer) 등의 구조를 탄생시킴



논문에서 제안한 Transformer의 구조

Sequence Modeling

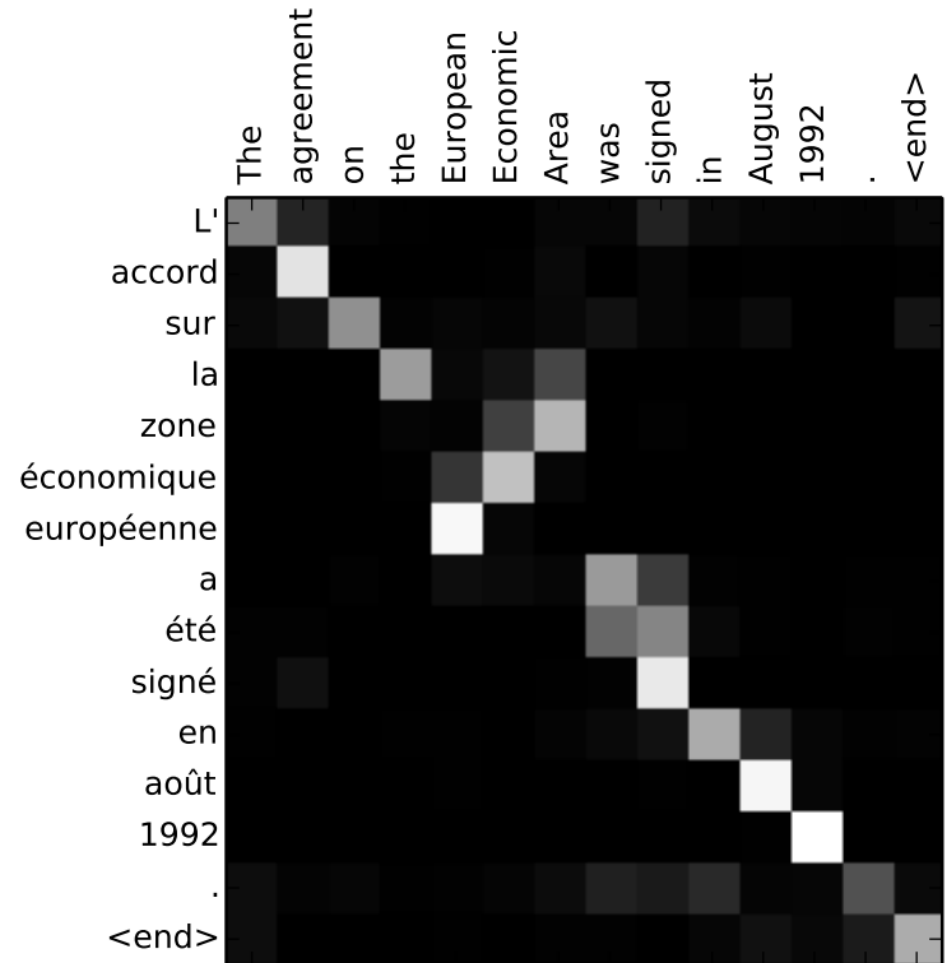
- 시계열 데이터로부터 또 다른 시계열 데이터를 생성하는 작업
ex) 기계번역, 챗봇 등
- 기존에는 이러한 시퀀스 모델링에 RNN 계열의 모델이 사용됨
=> 여러 번 언급했듯 병렬화의 어려움 등의 문제점이 있음



튀르키예어 -> 영어로 번역하는
시퀀스 모델링

Attention Mechanism

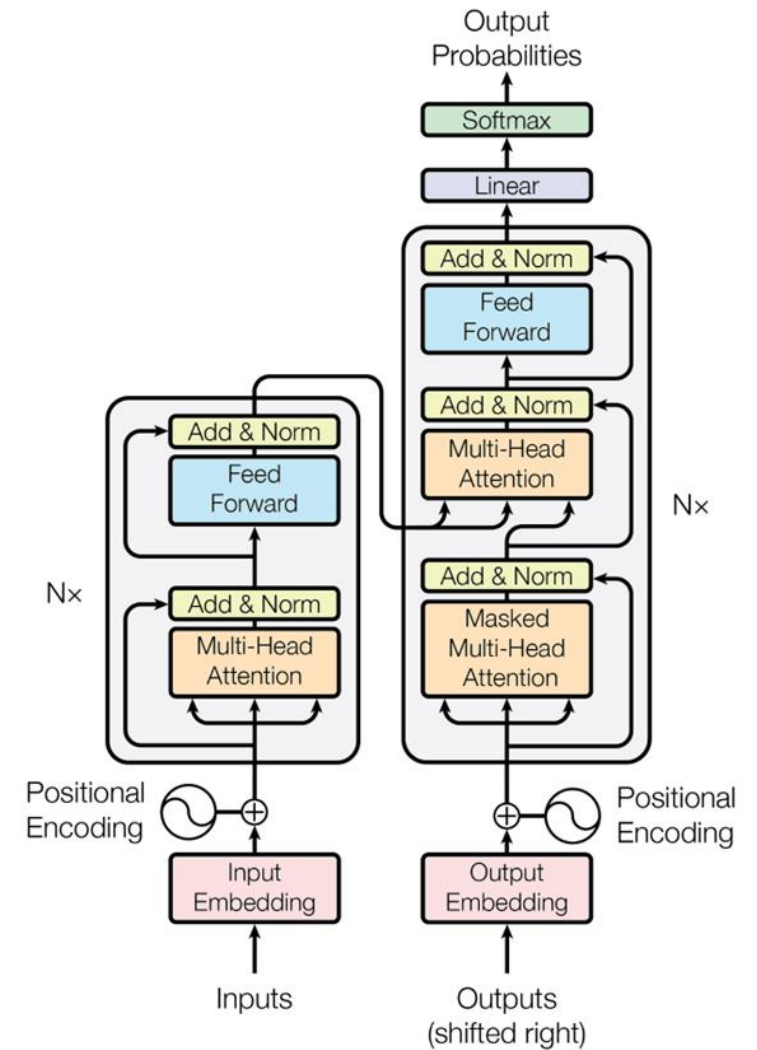
- 입력 또는 출력 데이터에서 Sequence Distance에 무관하게 서로 간의 종속성을 모델링한다.
즉, 서로가 서로 간에 얼마나 닮았는지 나타낸다.
- 이 이미지는 Bahdanau Attention을 나타낸 이미지로, Sequence Modeling 에서 Attention을 사용할 때의 Correlation Matrix (서로 얼마나 닮았는가)를 나타낸다. (프랑스어 -> 영어 번역 과정이다.)
- 논문에서는 기존의 CNN이나 RNN구조를 사용하지 않고, 오로지 이 Attention만을 이용하여 구축한 Transformer라는 모델을 제안한다.



Bahdanau Attention

Transformer Architecture

- 이 사진은 트랜스포머의 구조이다.
- 트랜스포머 또한 인코더 - 디코더 구조를 가지고 있으며, 다른 모델들과 다르게 내부는 Self Attention과 Fully Connected Layer로만 구성되어 있다.
- 트랜스포머에 사용되는 Attention은 앞서 살펴보았던 Attention들과 구조가 살짝 다르다.
- 트랜스포머는 Scaled Dot-Product Attention, Multi-Head Attention, Position-wise Feed-Forward Networks, Embedding, Softmax, Positional Encoding으로 구성되어 있다.



Transformer's Architecture

Tokenization

- NLP 등에서 정보를 작은 단위로 나누는 과정을 의미하여, 그 작은 단위는 Token이라고 한다.
- 자연어처리에서, 주로 3가지의 토큰화 방법이 있다.
 - ① Word Tokenization: 텍스트를 단어 단위로 분리
 - ② Sentence Tokenization: 텍스트를 문장 단위로 분리
 - ③ Subword Tokenization: 텍스트를 단어보다 더 작게 분리
- 영어나 중국어같은 고립어는 공백을 기준으로 쉽게 토큰화가 가능하지만, 한국어나 일본어같은 교착어는 공백을 기준으로 토큰화하면 의미를 상실하는 경우가 많다.
실제로 한국어를 토큰화 할 때 가장 많이 사용되는 방법은 형태소 기반 토큰화이다.

Time is an illusion.

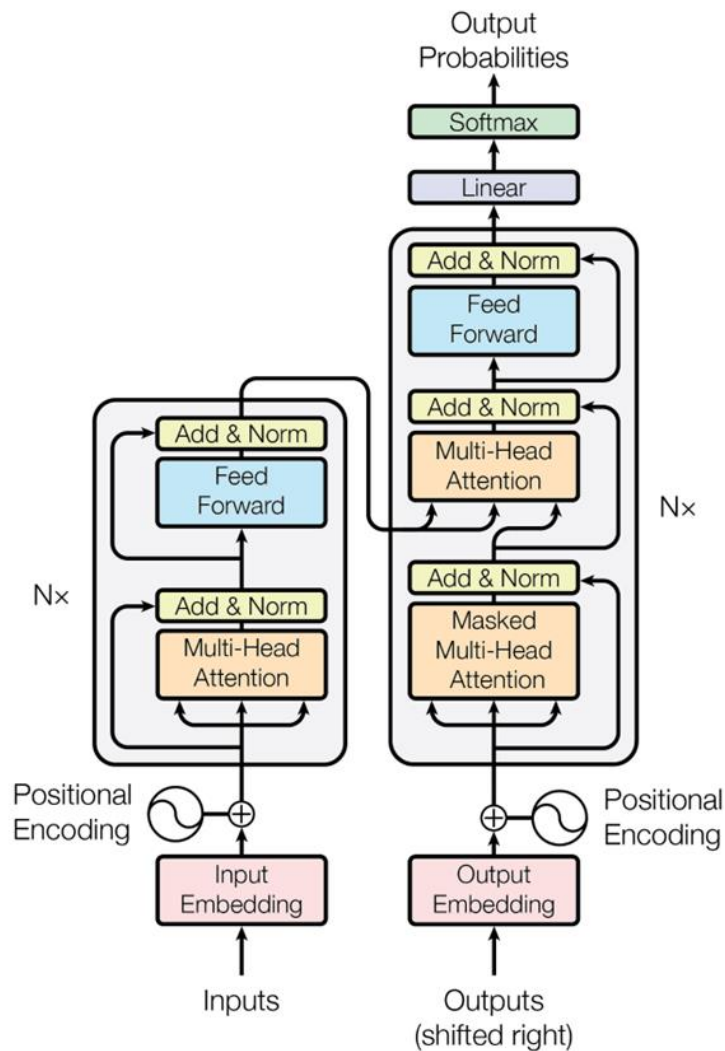


"Time", "is", "an", "illusion"

NLP에서의 Tokenization 예시

Embedding

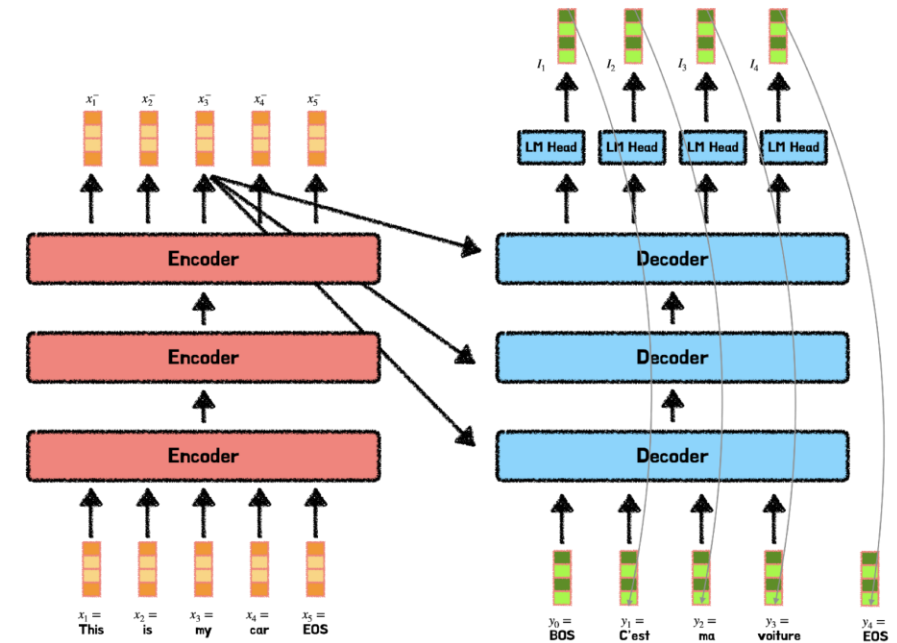
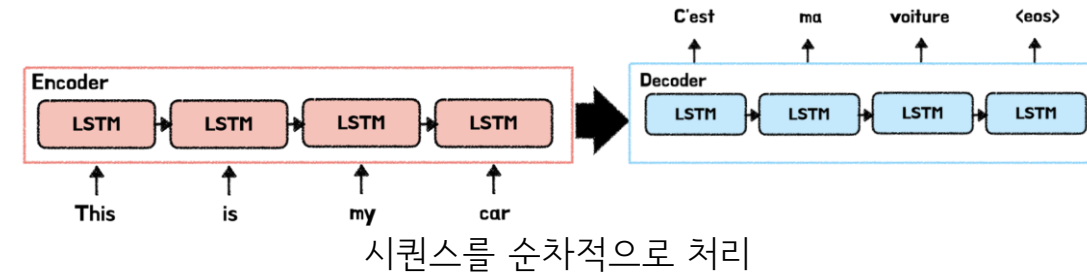
- 논문에서는 입력과 출력을 Embed하여 사용한다.
- 임베딩이란, 입력된 정보를 숫자의 나열인 벡터로 바꾸는 것이다.
자연어처리에서는 입력된 문장을 토큰화하고 숫자로 바꾸는 것을 임베딩이라 한다.
- Input Embedding: 인코더에 입력되는 단어들을 고차원 벡터로 변환하는 과정이며, 각 단어는 고유한 임베딩 벡터로 표현된다.
이는 모델이 단어의 의미와 문맥을 이해하는 데 도움을 준다.
- Output Embedding: 디코더에서 출력되는 단어들을 고차원 벡터로 변환하는 과정이며, 디코더는 이전에 생성된 단어들을 입력으로 받아 다음 단어를 예측한다.
이때 출력 임베딩을 활용하여 단어의 의미를 벡터 형태로 표현한다.



Transformer's Architecture

Positional Encoding

- 트랜스포머 이전에는 RNN 계열들로 구성된 모델들이 사용되었는데, 이들은 입력되는 시퀀스를 순차적으로 처리한다는 특징이 있다. 따라서 계산 유닛이 아무리 많아도 한 번에 하나의 정보만 처리할 수 있기 때문에 속도가 매우 느리다.
- 트랜스포머는 입력되는 시퀀스를 병렬적으로 한 번에 처리한다. 그러나 값이 한 번에 다 들어오기 때문에 단어의 순서 및 위치를 알 수 없다. 단어의 위치에 따라 문장의 의미가 완전히 달라지는 경우가 있기 때문에, 이는 상당히 큰 문제다.
- 논문에서는 이 문제를 해결하기 위해 Positional Encoding을 제안한다.



시퀀스를 병렬적으로 처리

Positional Encoding

- 트랜스포머는 Attention 만으로 시퀀스 데이터를 처리한다.
RNN이나 CNN을 사용하지 않기 때문에, 입력되는 시퀀스의 순서 정보를 명시적으로 제공해야 하며, 이를 위해 Positional Encoding을 사용한다.

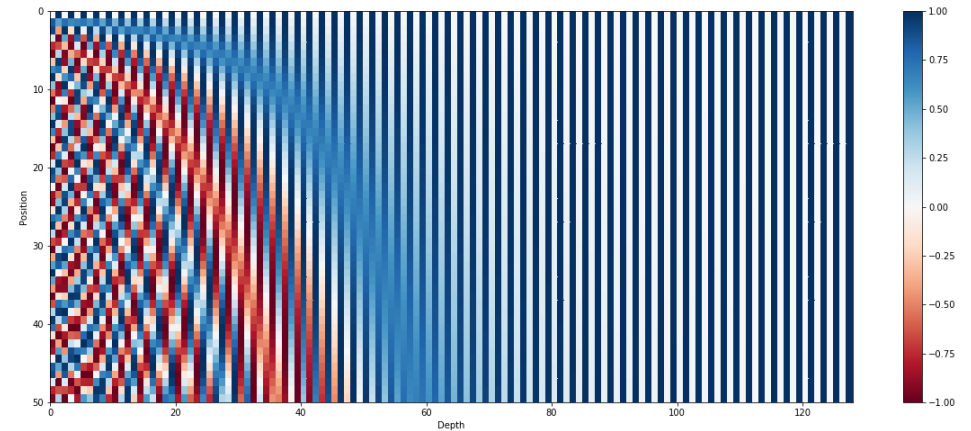
- 논문에서는 sin과 cos를 이용한 Positional Encoding을 제안한다.

짝수 인덱스: $PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$

홀수 인덱스: $PE_{(pos, 2i)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$

(pos 는 단어의 위치, i 는 Embedding차원의 인덱스, d_{model} 은 모델의 Embedding 차원의 수를 나타낸다.)

- 이런 복잡한 방식으로 Positional Encoding을 수행하는 이유는 단어 정보 대비 위치 정보가 더 작아야 모델이 단어의 위치가 아닌 의미에 더 집중할 수 있고, 같은 위치의 토큰은 항상 같은 위치 벡터값을 가지고 있어야 하기 때문이다.



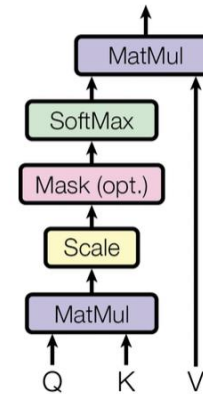
Positional Encoding

Attention

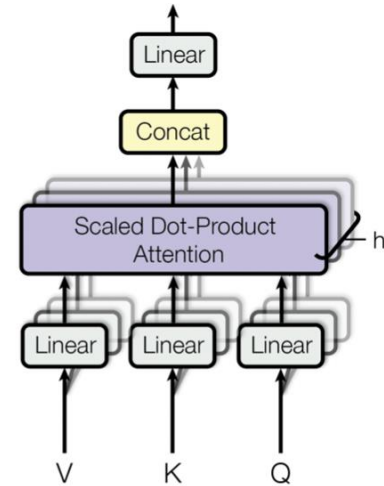
Scaled Dot-Product Attention

- 트랜스포머의 Multi-Head Attention은 Scaled Dot-Product Attention으로 이루어져 있다.
- 이 Attention은 입력 시퀀스 내의 각 요소가 동일한 시퀀스의 다른 모든 요소와의 관계를 평가하는 구조이다. 이러한 특성 때문에 '자기 자신에 대한 Attention'이라는 의미로 'Self Attention'이라고도 부른다.

Scaled Dot-Product Attention



Multi-Head Attention



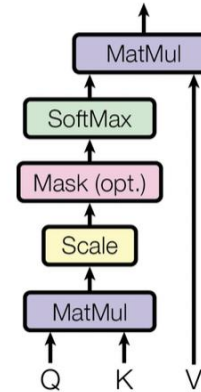
Attention

Attention

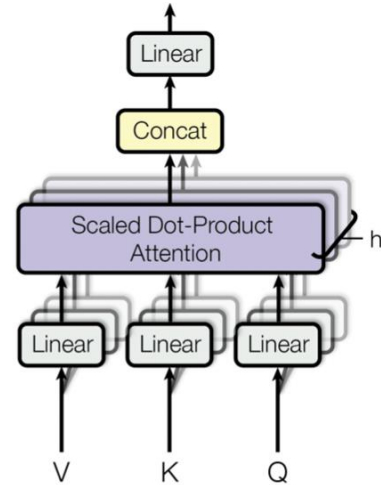
Self Attention

- 일반 Attention과 동일하게 Q, K, V를 입력받는다.
- 옵션으로 Mask layer가 있는데,
이는 참조하고 싶지 않은 correlation을 masking할 때 사용된다.
- 세 개의 weight matrix를 정의한다.
 - W_Q : d_{model} by d_Q
 - W_K : d_{model} by d_K
 - W_V : d_{model} by d_v(d_{model} 은 입력 임베딩의 차원(하이퍼파라미터)이며, d_Q, d_K, d_v 는 같은 값으로, Q, K, V 벡터의 차원을 나타낸다. 일반적으로 $\frac{d_{model}}{h}$ 의 값이며, h 는 Multi-Head Attention의 헤드의 개수를 나타낸다.)

Scaled Dot-Product Attention



Multi-Head Attention

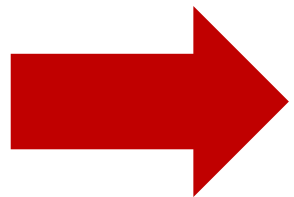


Attention

Attention

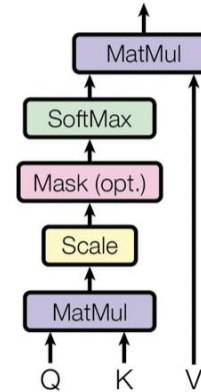
Self Attention

- ① Embedding 및 Positional Encoding된 Input Sequence에 대해, 세 개의 weight matrix를 곱하여 Q, K, V 벡터를 생성한다.
- ② 각 Q와 모든 K 벡터에 대해, dot product 연산으로 Attention Score를 계산한다. (QK^T)
- ③ Vanishing Gradient를 완화하기 위해 Attention Score를 $\sqrt{d_k}$ 로 나눈다. ($\frac{QK^T}{\sqrt{d_k}}$)
- ④ Scaled Attention Score에 Softmax를 적용하여 Attention Distribution을 얻는다. ($\text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right)$)
- ⑤ Attention Weight (Attention Distribution의 각 원소)에 V 벡터를 곱하고, 이를 합산하여 최종 Context Vector를 생성한다.

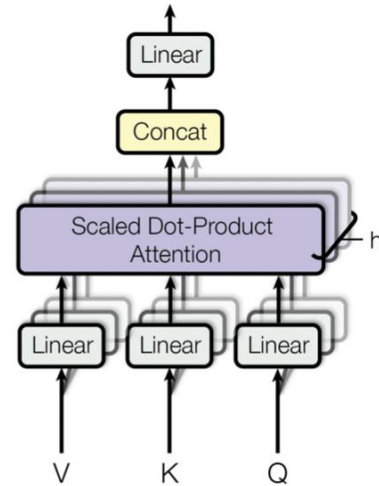


$$Attention(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right)$$

Scaled Dot-Product Attention



Multi-Head Attention



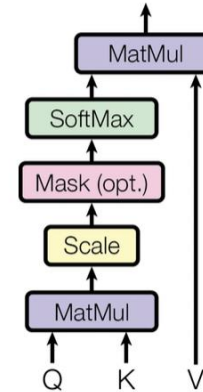
Attention

Attention

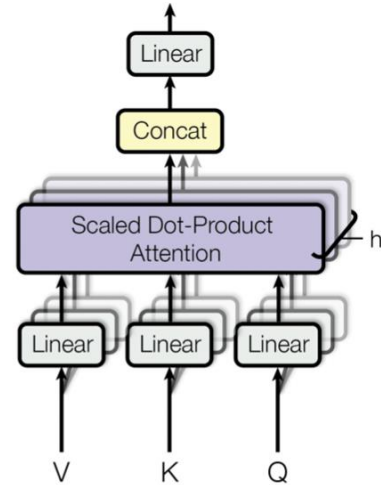
Multi-Head Attention

- 실험 결과, 하나의 Attention을 사용하는 것보다 Q, K, V를 linear projection을 통하여 각각 다른 값들을 입력으로 하는 여러 개의 Attention을 사용하는 것이 더 효율적이라 한다.
- 이 때 사용되는 Attention은 head라 불리며, 이 head의 개수를 h 로 표기한다.
- 각 Attention들에서 나온 출력값들은 Concatenate된 후 linear function을 한번 더 거친다.

Scaled Dot-Product Attention



Multi-Head Attention



Attention

Attention

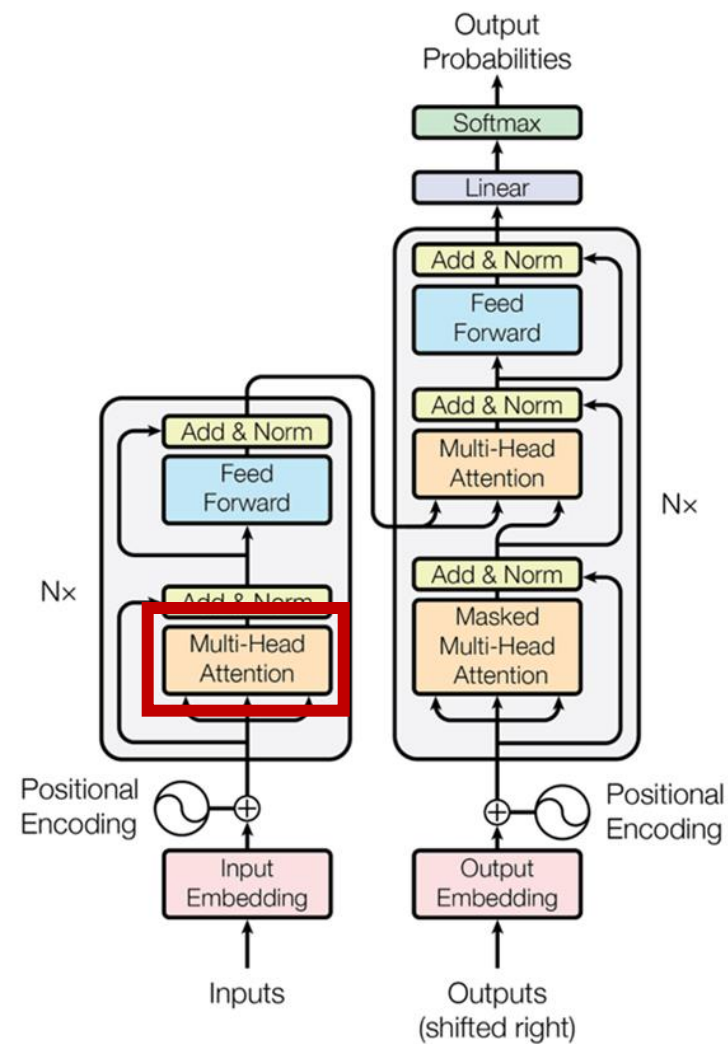
Attention in Encoder

인코더의 각 레이어에서 입력 시퀀스의 모든 위치에 대해 Q, K, V 벡터를 생성한다.

각 위치의 Q 벡터는 동일한 레이어의 모든 위치의 키 벡터와의 유사도를 계산하여 Attention Value를 얻는다.

이 가중치를 V 벡터에 적용하여 해당 위치의 출력을 생성한다.

이로서 입력 시퀀스 내의 각 단어가 다른 모든 단어와의 관계를 학습하여 문맥 정보를 효과적으로 파악한다.



Encoder의 Attention

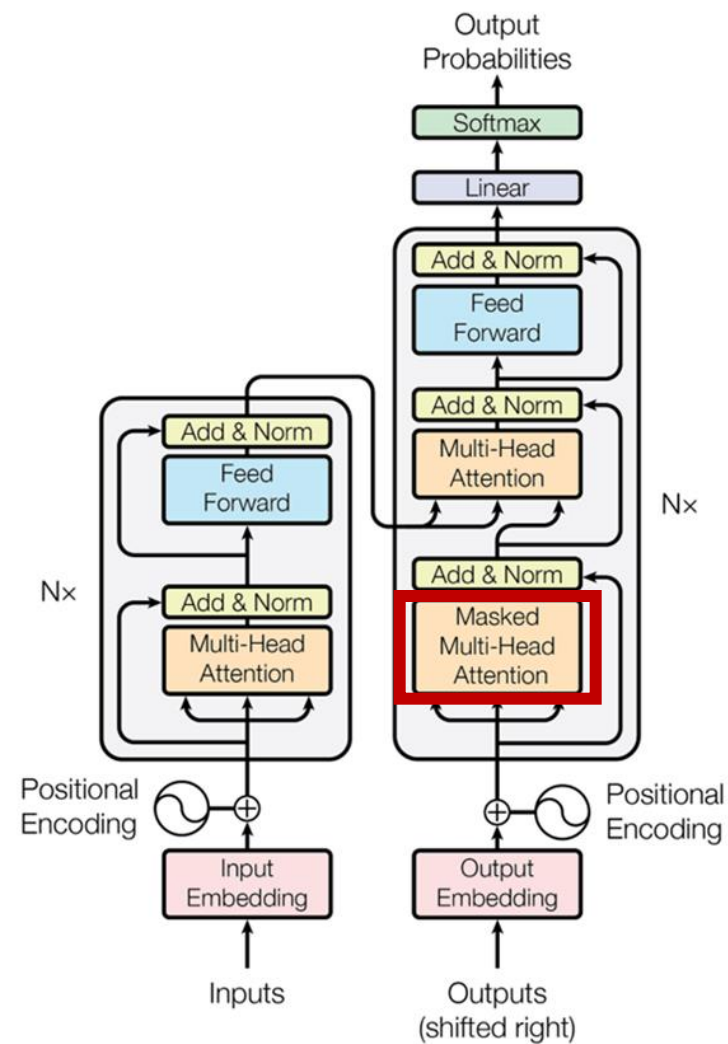
Attention

Attention in Decoder

인코더의 Attention과 유사하지만, 여기서는 미래의 정보를 볼 수 없도록 Masking을 적용한다.

즉, 현재 위치의 단어는 이전 위치의 단어들을 참조하여 어텐션을 수행한다.

이로서 출력 시퀀스를 생성할 때, 이전에 생성된 단어들을 기반으로 다음 단어를 예측하여 Autoregressive 특성(현재의 출력이 이전의 출력에 의존하는 구조로, 순차적인 데이터 생성에 활용)을 유지할 수 있다.



Encoder의 Attention

Attention

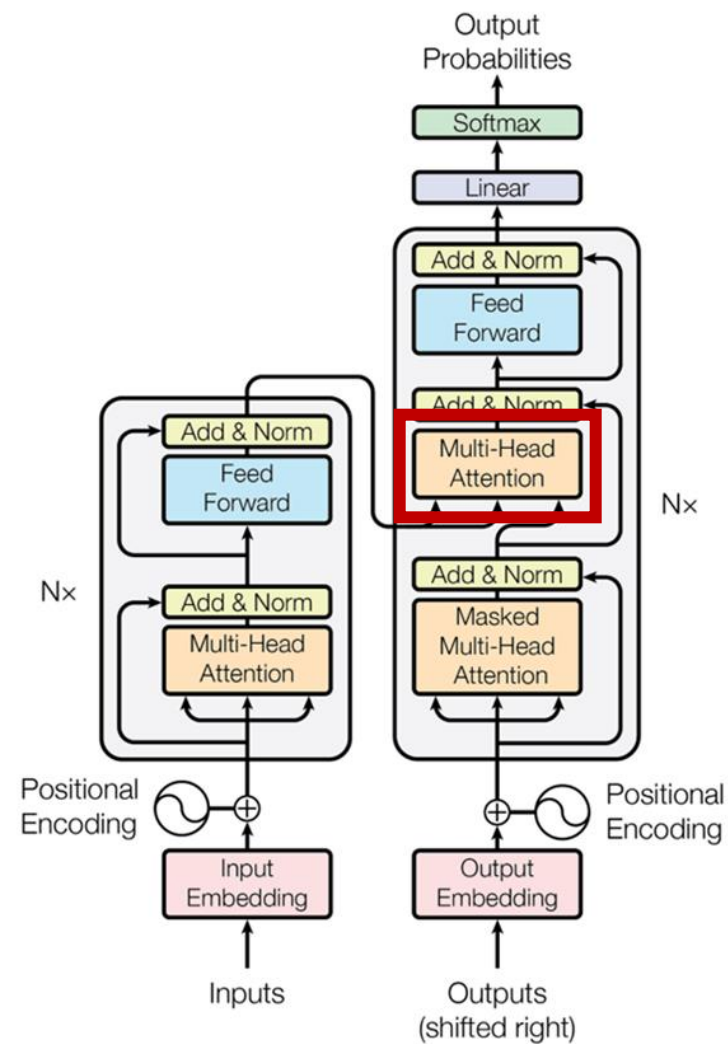
Attention in Encoder - Decoder

디코더의 각 레이어에서 self attention 이후에 수행된다.

여기서 Q 벡터는 디코더의 이전 레이어에서 가져오고,
K와 V 벡터는 인코더의 출력에서 가져온다.

이를 통해 디코더의 각 위치는 인코더의 모든 위치와의 관계를 학습할 수 있다.

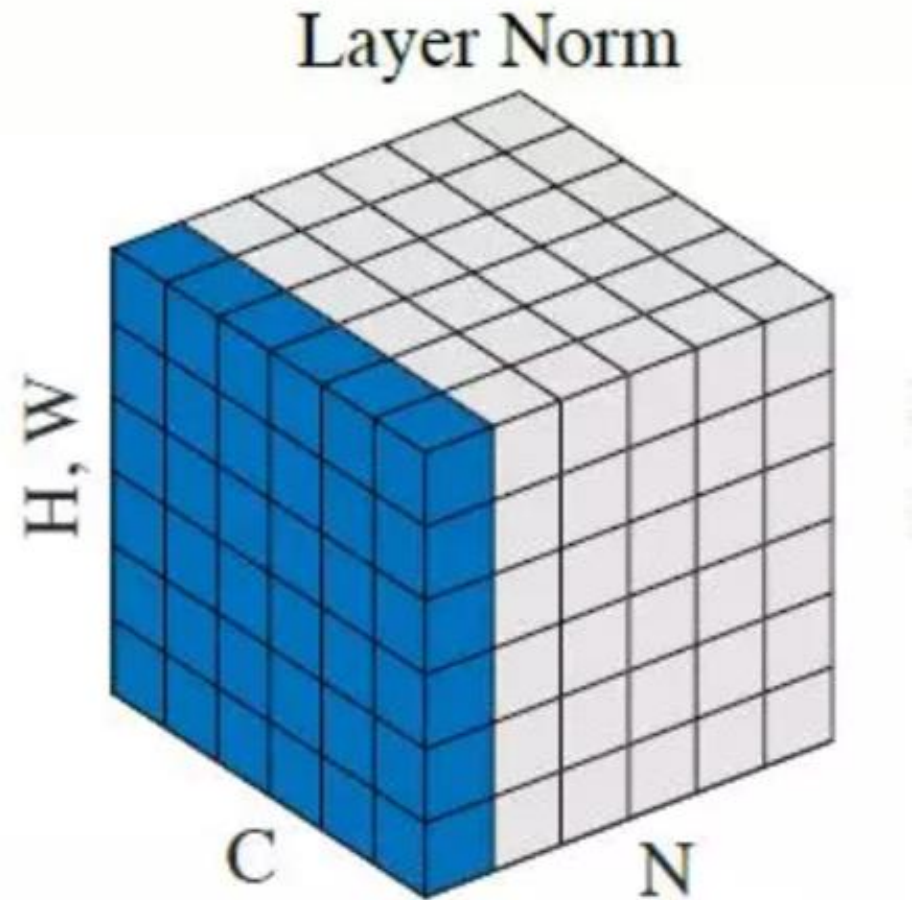
이로서 디코더가 입력 시퀀스의 정보를 효과적으로 활용하여
출력 시퀀스를 생성할 수 있도록 지원할 수 있다.



Encoder의 Attention

Layer Normalization

- 한 데이터 샘플의 feature 차원별로 정규화한다.
- Batch size와는 무관하게 동작하기 때문에, Batch size가 작아도 안정적으로 작동할 수 있다.
- 각 샘플에 대해 개별적으로 정규화되고, 시간 축으로 정규화가 가능하기 때문에 시퀀스 데이터에 적합하다.
- $\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$
- x_{ij} 는 i 번째 feature,
- μ_j 는 샘플 내 모든 feature의 평균,
- σ_j^2 는 샘플 내 모든 feature의 분산,
- ϵ 은 분모가 0이 되는걸 방지하기 위한 매우 작은 값이다.

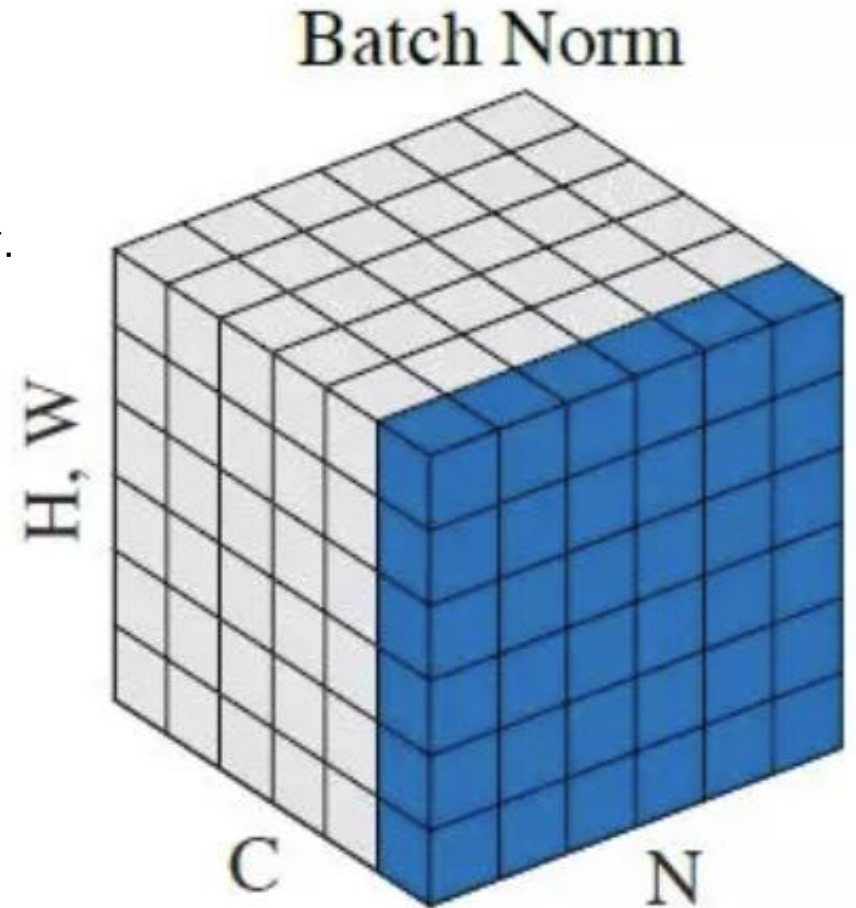


Layer Normalization

Batch Normalization (비교용)

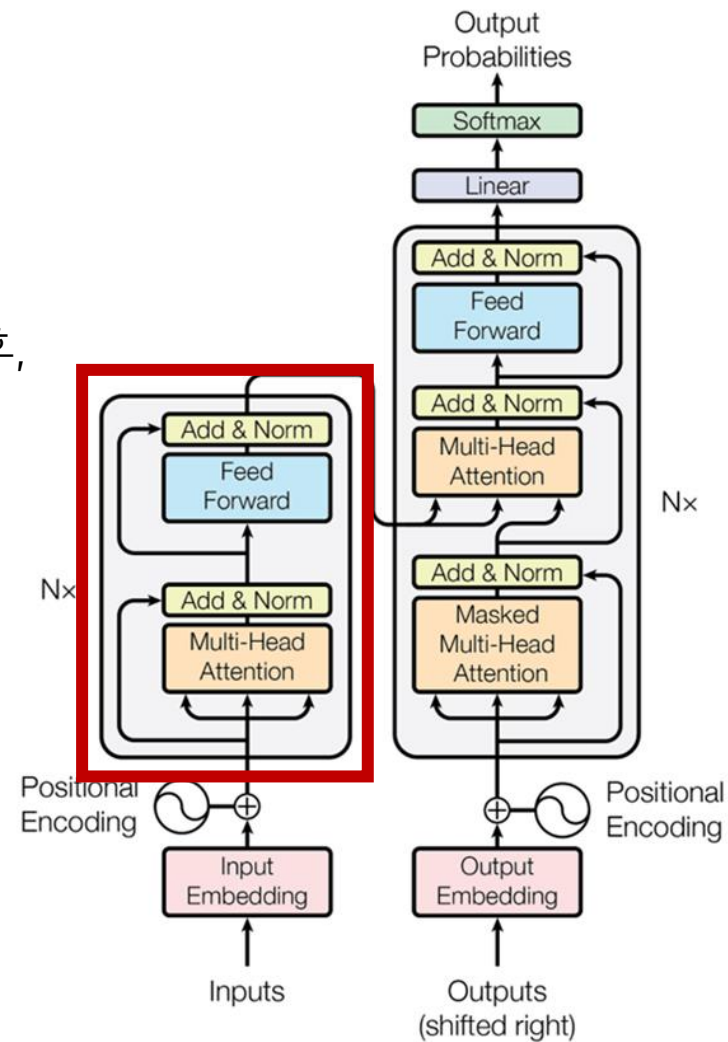
(합성곱 신경망 보고서에서 발췌함)

- Local Response Normalization과 달리 각 레이어의 입력을 정규화한다.
- Minibatch의 평균과 분산을 사용해 정규화한다.
- 빠른 학습, 과적합 방지, 가중치 초기화에 덜 민감하다.
- 계산량 증가, Minibatch가 작을수록 효과 감소한다.
- $$\hat{x}_{ij} = \frac{x_{ij} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$
- x_{ij} 는 i 번째 배치의 j 번째 feature,
- μ_j 는 Minibatch 내 j 번째 feature의 평균,
- σ_j^2 는 Minibatch 내 j 번째 feature의 분산,
- ϵ 은 분모가 0이 되는걸 방지하기 위한 매우 작은 값이다.



Encoder

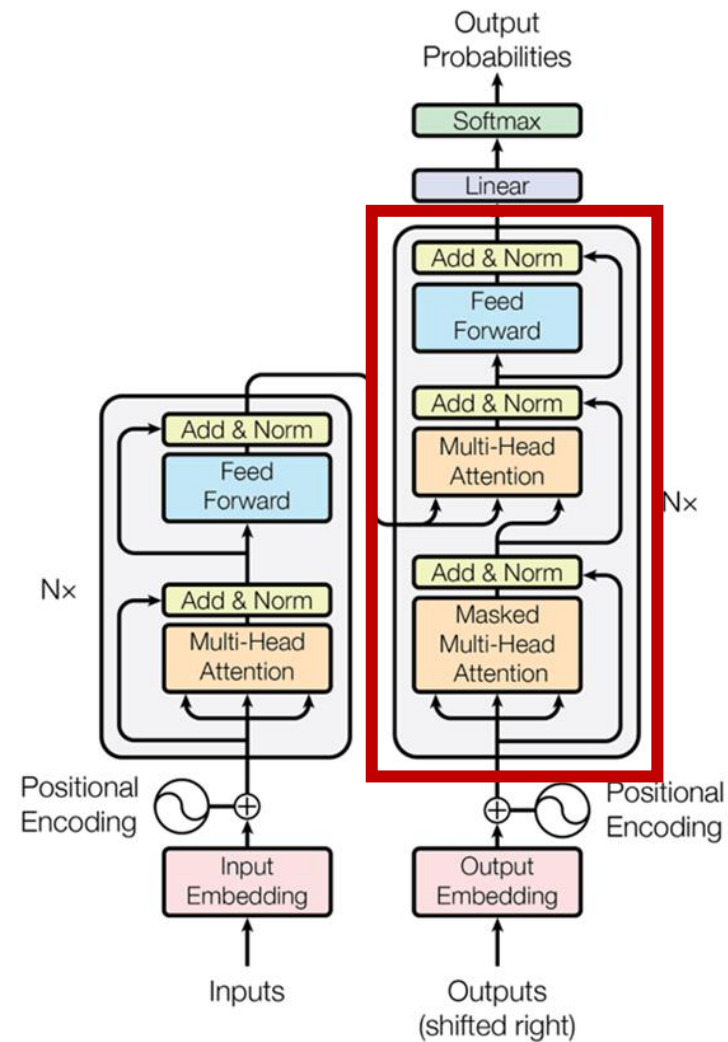
- 입력된 시퀀스를 처리하여 시퀀스의 문맥 정보를 추출한다.
- Positional Encoding된 시퀀스를 입력으로 받아, Multi-Head Attention을 거친 후, Feed Forward(정보가 입력->출력 한 방향으로만 진행되는 구조)를 거친다. Feed Forward를 거치며 비선형으로 변환되고, 복잡한 패턴을 학습할 수 있다.
- 이 때, ResNet의 Residual Block처럼 Residual Connection을 적용한다.
- 정규화 기법으로는 Layer Normalization을 사용한다.
- 트랜스포머의 인코더는 이 블록을 여러 개 쌓아 이루어지며, 논문에서는 Attention - Feed Forward 구조를 6번 쌓아 인코더를 정의한다.



Transformer의 Encoder

Decoder

- 트랜스포머의 디코더는 다른 구조의 디코더와 마찬가지로 입력된 시퀀스를 기반으로 현재 단어를 예측한다.
- 트랜스포머의 디코더는 Autoregressive한 구조를 가지며, 생성 작업을 수행한다.
- Positional Encoding된 시퀀스를 입력으로 받아 self attention 구조를 거친다. 이 때의 attention은 현재 위치 이전의 단어들만 참조하고, 미래의 단어들은 참조하지 못하도록 Look-Ahead Masking을 적용한다. 디코더는 미래의 단어를 예측하는 작업을 하므로, 미래의 단어를 참조해서는 안되기 때문이다.
- 이후, 이전 attention에서 출력된 결과를 Q로 받고, K, V는 인코더에서 입력받아 Attention을 수행한다. (따라서 self attention이 아니다.) 마지막으로, 인코더와 마찬가지로 Feed Forward를 통해 복잡한 패턴을 학습한다.
- 정규화는 모두 Layer Normalization을 사용하고, 논문에서는 인코더와 마찬가지로 이 구조를 6번 쌓아 디코더를 정의한다.



Transformer의 Encoder

Attention을 쓰는 이유

- 트랜스포머는 RNN이나 CNN을 쓰지 않고 오로지 Attention만 사용한다.
분명 성능이 더 좋으니까 쓸 텐데, 왜 Attention이 더 좋을까?

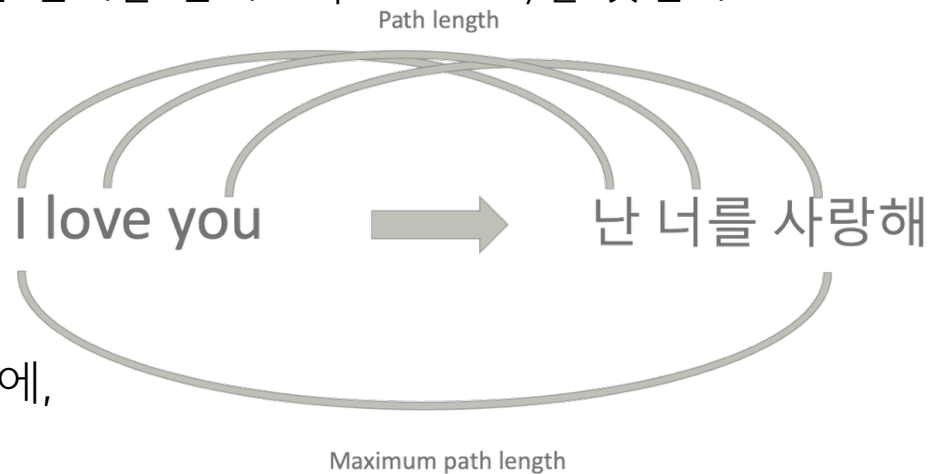
Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

Self-Attention이 RNN이나 CNN보다 좋은 이유

- 빅-오 표기법으로 표기된 레이어 당 복잡도를 보면, 시퀀스의 길이 n , Representation Dimensionality d 에 대해 $n < d$ 일 때만 Attention의 복잡도가 RNN보다 낮다고 한다. 그러나 보통 n 은 d 보다 작기 때문에, 대부분의 경우 Attention의 복잡도가 더 작다.
- restricted라고 표기된 Attention은 전체 K 에 대해 연산을 수행하지 않고, r 개의 이웃들만 연산을 수행한다. 따라서 복잡도는 더욱 감소한다.

Attention을 쓰는 이유

- Sequential Operations는 한 시퀀스를 처리하기 위해 필요한 시간복잡도이다.
- RNN의 경우는 순차적으로 진행되기 때문에 시퀀스의 길이만큼의 시간이 필요하지만, Attention의 경우 병렬적으로 처리가 가능하기 때문에 $O(1)$ 만큼의 복잡도만 필요하다.
- Long-range Dependencies라는 게 있는데, 이는 문장에서 멀리 떨어져 있는 단어들 간의 Dependency를 뜻한다. RNN에서는 이런 먼 거리의 관계를 학습하기 위해 여러 단계를 거쳐야 하며, 이는 네트워크 내부의 path length가 길어지는 결과를 초래한다. 우측의 이미지에서와 같이 각 토큰 사이의 거리가 Path Length이며, 가장 긴 거리가 Maximum Path Length인 것이다. 입력과 출력 시퀀스 사이에 path length가 짧을 수록 Long-range Dependencies를 더 잘 학습하는데, Attention은 각 토큰을 모든 토큰들과 참조하여 Correlation을 구하기 때문에, Attention은 $O(1)$ 만큼의 복잡도만 필요하며, 이 역시 RNN보다 우수하다.



실습 #1) Transformer 기반 텍스트 분류 (GPT로 생성된 예제)

- IMDb 영화 리뷰 데이터셋은 영화 리뷰와 함께 해당 리뷰가 긍정적인지 또는 부정적인지를 나타내는 라벨로 구성된 데이터셋입니다. 이 데이터셋을 사용하여 Transformer 기반의 텍스트 분류 모델을 설계하고, 주어진 리뷰가 긍정적인지 부정적인지를 분류하는 모델을 구현하시오.
- 세부 요구 사항:
 - 모델 설계
 - Transformer 구조의 주요 요소인 Multi-Head Attention, Feedforward Layer, Residual Connection 및 Layer Normalization을 포함한 Transformer 블록을 설계하시오.
 - Positional Encoding을 구현하여 시퀀스의 순서 정보를 모델에 포함하시오.
 - 데이터 전처리
 - 각 리뷰의 길이는 최대 512 단어로 제한하며, 부족한 부분은 패딩(padding)으로 채우시오.
 - 모델 학습
 - 설계한 Transformer 모델을 활용하여 IMDb 데이터셋을 학습시키고, 검증 데이터셋을 사용하여 모델의 성능(정확도 및 손실)을 평가하시오.
- 추가 조건:
 - 텐서플로우(TensorFlow) 및 케라스(Keras)를 사용하여 구현할 것.

IMDb Datasets

- 50000개의 영화 리뷰들로 구성됨
이 중 절반은 Training으로 사용됨
- 0과 1의 레이블로 분리되어 있고, 1이 Positive, 0이 Negative
- 각 리뷰는 단어를 숫자로 인코딩하여 제공됨.
데이터셋에는 리뷰에서 등장한 단어들이 빈도 순으로 순서가 매겨져 있음
가장 자주 등장한 단어는 인덱스 3번으로 배정됨
- 0~2번째 인덱스는 특수한 토큰으로,
각각 Padding, 시퀀스의 시작을 알리는 토큰, OOV를 나타냄
OOV는 Out Of Vocabulary의 약자로, 내장된 사전에 없는 단어를 뜻함



03 실습 #1 코드 분석

```
import tensorflow as tf
from tensorflow.keras import layers, Model
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.datasets import imdb
import matplotlib.pyplot as plt
import time as t
import numpy as np

# 하이퍼파라미터 설정
MAX_SEQ_LEN = 512
VOCAB_SIZE = 80000
EMBED_DIM = 512
NUM_HEADS = 8
FF_DIM = 2048
BATCH_SIZE = 64
EPOCHS = 10
```

필요한 라이브러리를 임포트한다. IMDb 데이터셋은 keras에 준비되어 있다.

하이퍼 파라미터를 설정하는데, 각각 아래와 같은 의미를 가진다.

MAX_SEQ_LEN: 입력 문장의 최대 길이로, 모델이 처리하기 쉽게 하는 역할을 맡는다.

VOCAB_SIZE: 내장된 사전의 크기이며, 빈도 순으로 상위 80000번째 단어까지 사용한다.

EMBED_DIM: 단어의 임베딩 차원

NUM_HEADS: Multi-Head Attention의 헤드(각 Attention)의 개수

FF_DIM: Feed Forward의 Hidden Layer의 차원

```
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=VOCAB_SIZE)
x_train = pad_sequences(x_train, maxlen=MAX_SEQ_LEN, padding='post')
x_test = pad_sequences(x_test, maxlen=MAX_SEQ_LEN, padding='post')
```

IMDb 데이터셋을 로드한다.

트랜스포머의 구조 상 시퀀스들의 길이가 동일해야 하므로, post 속성을 이용하여 뒤쪽에 zero padding을 추가한다.

03 실습 #1 코드 분석

```
class TransformerBlock(layers.Layer):
    def __init__(self, embed_dim, num_heads, ff_dim, rate=0.1):
        super(TransformerBlock, self).__init__()
        self.att = layers.MultiHeadAttention(num_heads=num_heads, key_dim=embed_dim)
        self.ffn = tf.keras.Sequential([
            layers.Dense(ff_dim, activation="relu"),
            layers.Dense(embed_dim),
        ])
        self.layernorm1 = layers.LayerNormalization(epsilon=1e-6)
        self.layernorm2 = layers.LayerNormalization(epsilon=1e-6)
        self.dropout1 = layers.Dropout(rate)
        self.dropout2 = layers.Dropout(rate)

    def call(self, inputs, training):
        attn_output = self.att(inputs, inputs)
        attn_output = self.dropout1(attn_output, training=training)
        out1 = self.layernorm1(inputs + attn_output)
        ffn_output = self.ffn(out1)
        ffn_output = self.dropout2(ffn_output, training=training)
        return self.layernorm2(out1 + ffn_output)
```

트랜스포머 블록을 정의한다. 기타 구조와는 달리 keras에서는 트랜스포머를 지원하지 않기 때문에 일일이 작성해야 한다.

Multi-Head Attention, Layer Normalization, Dropout 등등을 init에서 정의한다.

call()메서드는 실제 연산이 이루어지는 부분으로, 입력된 시퀀스는 Self-Attention과 Layer Normalization을 거쳐 출력된다.

03 실습 #1 코드 분석

```
class PositionalEncoding(layers.Layer):
    def __init__(self, maxlen, embed_dim):
        super(PositionalEncoding, self).__init__()
        self.positional_encoding = self.get_positional_encoding(maxlen, embed_dim)

    def get_positional_encoding(self, maxlen, embed_dim):
        angle_rads = self.get_angles(
            np.arange(maxlen)[:, np.newaxis],
            np.arange(embed_dim)[np.newaxis, :],
            embed_dim
        )
        angle_rads[:, 0::2] = np.sin(angle_rads[:, 0::2]) # 짝수: sin
        angle_rads[:, 1::2] = np.cos(angle_rads[:, 1::2]) # 홀수: cos
        return tf.convert_to_tensor(angle_rads[np.newaxis, ...], dtype=tf.float32)

    def get_angles(self, positions, i, d_model):
        return positions / np.power(10000, (2 * (i // 2)) / np.float32(d_model))

    def call(self, x):
        return x + self.positional_encoding[:, :tf.shape(x)[1], :]
```

Transformer는 입력된 시퀀스를 병렬로 한 번에 처리하는 특성 상 입력 시퀀스의 순서를 알지 못한다.
해당 코드는 단어의 위치 정보를 추가해주는 Positional Encoding을 구현한다.

get_positional_encoding() 메소드는 단어의 위치와 임베딩 차원에 따라 각 단어 위치의 고유한 값을 생성하는데,
sin과 cos를 이용하여 시퀀스의 순서 정보를 알 수 있게 한다.

03 실습 #1 코드 분석

```
def create_model():
    inputs = layers.Input(shape=(MAX_SEQ_LEN,))
    embedding_layer = layers.Embedding(VOCAB_SIZE, EMBED_DIM)(inputs) * tf.math.sqrt(tf.cast(EMBED_DIM, tf.float32))
    positional_encoding = PositionalEncoding(MAX_SEQ_LEN, EMBED_DIM)(embedding_layer)

    x = positional_encoding
    transformer_blocks = []
    for _ in range(6):
        transformer_block = TransformerBlock(EMBED_DIM, NUM_HEADS, FF_DIM)
        transformer_blocks.append(transformer_block)
    for block in transformer_blocks:
        x = block(x)

    x = transformer_block(positional_encoding)
    x = layers.GlobalAveragePooling1D()(x)
    x = layers.Dropout(0.1)(x)
    x = layers.Dense(64, activation="relu")(x)
    x = layers.Dropout(0.1)(x)
    outputs = layers.Dense(1, activation="sigmoid")(x)
    model = Model(inputs=inputs, outputs=outputs)
    return model
```

앞서 정의된 것들을 이용하여 실제 트랜스포머 구조를 생성하는 코드이다.

입력된 시퀀스를 임베딩한 후, 스케일링을 통하여 벡터 값의 크기를 조정한다.
이렇게 임베딩된 벡터에 Positional Encoding을 수행하고, 이를 인코더의 입력으로 한다.
이후, 논문과 동일하게 6개의 인코더 블록들을 중첩시킨다.

03 실습 #1 코드 분석

```
model = create_model()
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])

start_time = t.time()
history = model.fit(x_train, y_train, batch_size=BATCH_SIZE, epochs=EPOCHS, validation_split=0.2)
end_time = t.time()
print(f'Total Training Time: {(end_time - start_time):.4f}seconds')

start_time = t.time()
loss, accuracy = model.evaluate(x_test, y_test, batch_size=BATCH_SIZE)
print(f"Test Loss: {loss:.4f}\nTest Accuracy: {accuracy:.4f}")
end_time = t.time()
print(f'Total Test Time: {(end_time - start_time):.4f}seconds')
```

모델을 컴파일, 학습, 평가하는 코드이다.

time을 이용하여 각 과정의 실행 시간을 측정할 수 있게 하였다.

원래 논문에서는 Multi-Class Cross Entropy를 이용하지만, 현재 문제는 IMDb 데이터를 이진 분류하는 문제이므로 loss로 이진 CEE를 사용하였다. 옵티마이저로는 Adam을 사용하고, Training Set의 20%를 Validation Set으로 하였다.

이후 Test Set으로 모델을 평가하여 loss와 accuracy를 출력한다.

03 실습 #1 코드 분석

```
word_index = imdb.get_word_index()
reverse_word_index = {value: key for key, value in word_index.items()}

def decode_review(encoded_review):
    # OOV 변환
    review = " ".join([reverse_word_index.get(i - 3, "") for i in encoded_review])
    return review.replace(' br ', '')
```

이후 리뷰를 실제로 나타내기 위한 코드이다.

현재 리뷰는 인코딩 되어있기 때문에 사람이 이해하지 못하는 숫자로 표현되어 있을 것이다.

이것을 다시 디코딩하여 사람이 읽을 수 있는 텍스트로 다시 변환한다.

이 때, 사전에 없는 단어를 삭제하고, 이 데이터들은 리뷰를 html로 굵어온 것이라
 (줄바꿈 문자. \n과 같은 뜻이다.)를 제거해준다.

인덱스에서 3을 빼는 것을 볼 수 있는데, 이는 처음 3개의 인덱스들은 특수한 토큰을 나타내기 때문이다.

```
sample_review = x_test[:5]
sample_labels = y_test[:5]
sample_predictions = model.predict(sample_review)
for i in range(5):
    decoded_review = decode_review(x_test[i])
    prediction = "Positive" if sample_predictions[i] > 0.5 else "Negative"
    actual = "Positive" if sample_labels[i] == 1 else "Negative"
    correct = "Correct" if (sample_predictions[i] > 0.5) == (sample_labels[i] == 1) else "Incorrect"
    print(f"Review {i + 1}: {decoded_review}\nActual: {actual}, Predicted: {prediction} ({correct})\n")
correct_predictions = sum((sample_predictions.flatten() > 0.5) == sample_labels)
accuracy_print = correct_predictions / len(sample_labels)
print(f"Total Confidence on Predict: {(accuracy_print)*100:.2f}%")
```

리뷰의 분류가 잘 되었는지 맛보기로 보여주는 코드이다.

Test Set에서 5개의 리뷰를 샘플로 가져와 예측을 한 다음, 실제 레이블과 비교하고 5개 샘플에 대한 정확도를 출력한다.

03 실습 #1 코드 분석

```
plt.figure(figsize=(14, 6))

plt.subplot(1, 2, 1)
plt.plot(epochs, train_loss, label="Training Loss", marker="o")
plt.plot(epochs, val_loss, label="Validation Loss", marker="o")
plt.title("Training and Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(epochs, train_accuracy, label="Training Accuracy", marker="o")
plt.plot(epochs, val_accuracy, label="Validation Accuracy", marker="o")
plt.title("Training and Validation Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

plt.tight_layout()

# 테스트 데이터 성능 시각화
plt.figure(figsize=(8, 6))

# 테스트 손실 및 정확도
categories = ["Loss", "Accuracy"]
values = [loss, accuracy]

bars = plt.bar(categories, values, color=["red", "green"])
plt.title("Test Data Performance")
plt.ylabel("Value")
plt.ylim(0, 1) # 값의 범위를 0에서 1로 제한

# 값 표시
for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width() / 2, height, f"{height:.2f}", ha="center", va="bottom")

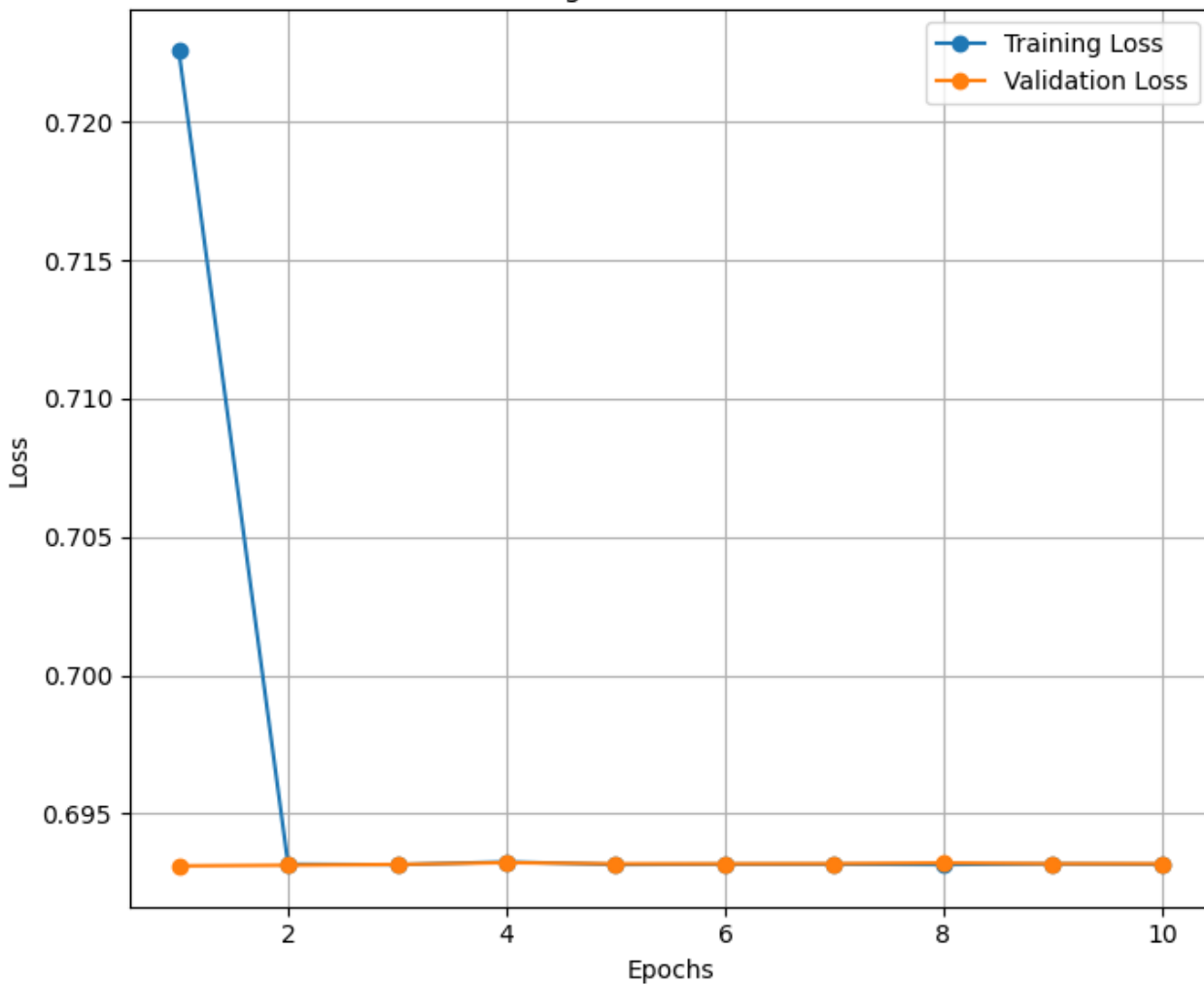
plt.grid(True, axis="y", linestyle="--", alpha=0.7)
plt.show()
```

그래프를 그려 시각화하는 부분이다.

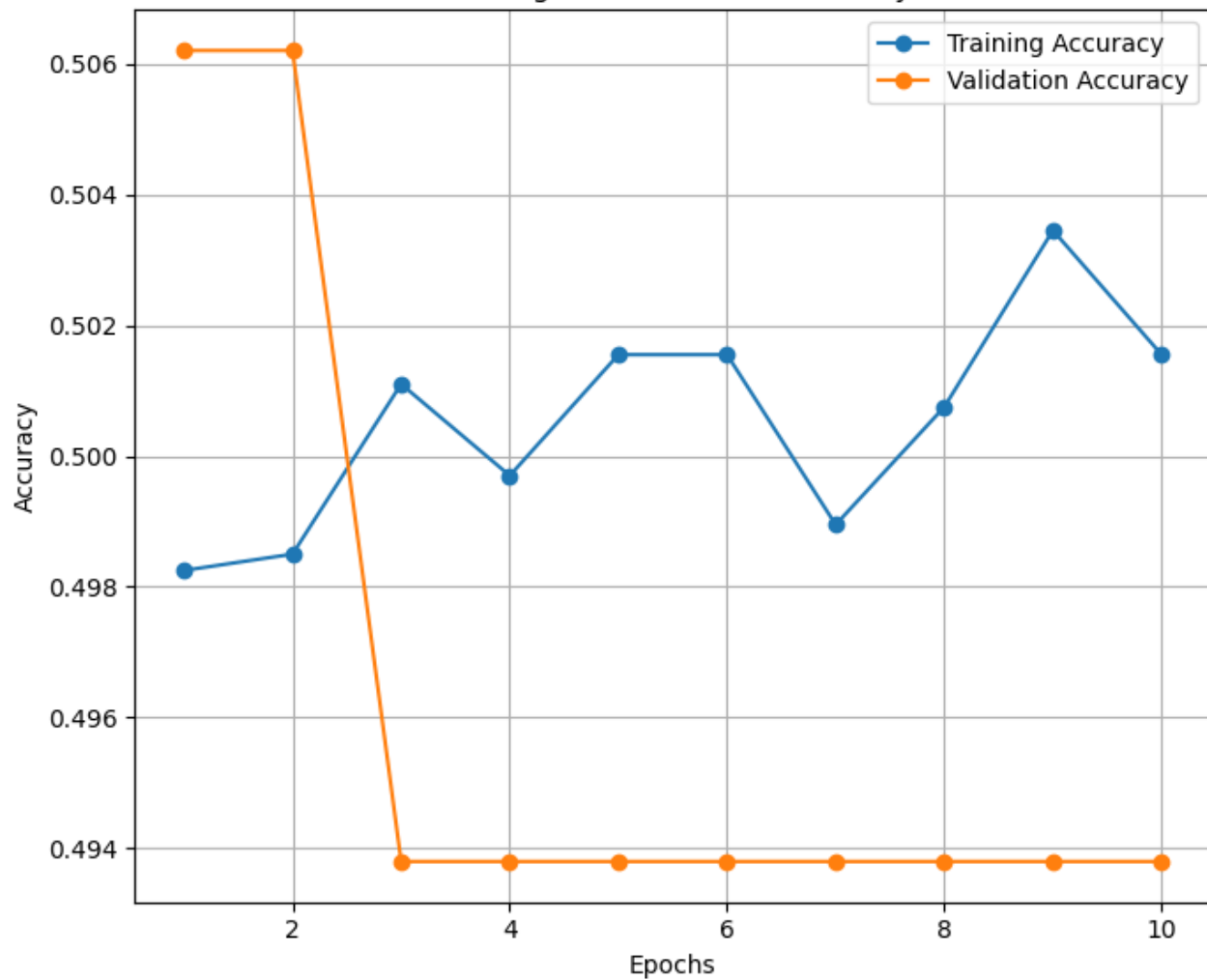
Training loss, accuracy, Test Set 등등을 시각화한다.

03 실습 #1 실행 결과

Training and Validation Loss



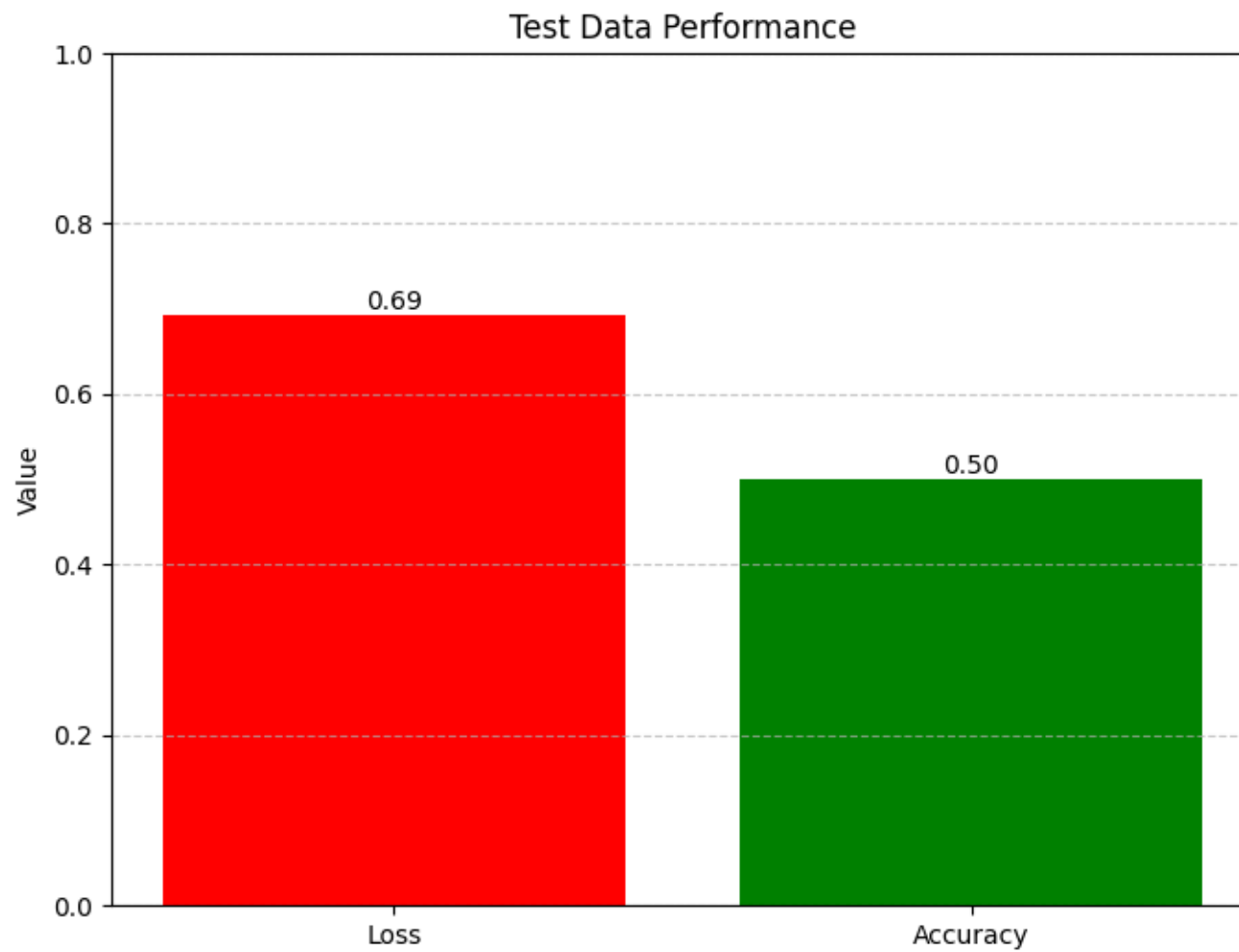
Training and Validation Accuracy



Total Training Time: 551.3343seconds

Total Test Time: 22.1179seconds

03 실습 #1 실행 결과



03 실습 #1 실행 결과

Review 1: please give this one a missbr kristy swanson and the rest of the cast rendered terrible performances the show is flat flat flatbr i don't know how michael madison could have allowed this one on his plate he almost seemed to know this wasn't going to work out and his performance was quite lacklustre so all you madison fans give this a miss

Actual: Negative, Predicted: Positive (**Incorrect**)

03 실습 #1 실행 결과

Review 2: this film requires a lot of patience because it focuses on mood and character development the plot is very simple and many of the scenes take place on the same set in frances austen's the sandy dennis character apartment but the film builds to a disturbing climaxbr the characters create an atmosphere rife with sexual tension and psychological trickery it's very interesting that robert altman directed this considering the style and structure of his other films still the trademark altman audio style is evident here and there i think what really makes this film work is the brilliant performance by sandy dennis it's definitely one of her darker characters but she plays it so perfectly and convincingly that it's scary michael burns does a good job as the mute young man regular altman player michael murphy has a small part the solemn moody set fits the content of the story very well in short this movie is a powerful study of loneliness sexual repression and desperation be patient soak up the atmosphere and pay attention to the wonderfully written scriptbr i praise robert altman this is one of his many films that deals with unconventional fascinating subject matter this film is disturbing but it's sincere and it's sure to elicit a strong emotional response from the viewer if you want to see an unusual film some might even say bizarre this is worth the timebr unfortunately it's very difficult to find in video stores you may have to buy it off the internet

Actual: Positive, Predicted: Positive (**Correct**)

03 실습 #1 실행 결과

Review 3: classic short the cameraman's revenge which he made in russia in 1912 at a time when motion picture animation of all sorts was in its infancybr the political tumult of the russian revolution caused starewicz to move to paris where one of his first productions coincidentally was a dark political satire variously known as frogland or the frogs who wanted a king a strain of black comedy can be found in almost all of films but here it is very dark indeed aimed more at grown ups who can appreciate the satirical aspects than children who would most likely find the climax upsetting i'm middle aged and found it pretty upsetting myself and indeed prints of the film intended for english speaking viewers of the 1920s were given title cards filled with puns and quips in order to help soften the sharp sting of the finalebr our tale is set in a swamp the frogland commonwealth where the citizens are unhappy with their government and have called a special session to see what they can do to improve matters they decide to beseech jupiter for a king the crowds are impressively animated in this opening sequence it couldn't have been easy to make so many frog puppets look alive simultaneously while jupiter for his part is depicted as a droll white bearded guy in the clouds who looks like he'd rather be taking a nap when jupiter sends them a tree like god who regards them the frogs decide that this is no improvement and demand a different king irritated jupiter sends them a storkbr delighted with this formidable looking new king who towers above them the frogs welcome him with a delegation of formally dressed dignitaries the mayor steps forward to hand him the key to the commonwealth as newsreel cameras record the event to everyone's horror the stork promptly eats the mayor and then goes on a merry rampage swallowing citizens at random a title card dryly reads news of the king's appetite throughout the kingdom when the now terrified frogs once more beseech jupiter for help he loses his temper and showers their community with lightning bolts the moral of our story delivered by a hapless frog just before he is eaten is let well enough alonebr considering the time period when this startling little film was made and considering the fact that it was made by a russian émigré at the height of that beleaguered country's civil war it would be easy to see this as a parable about those events starewicz may or may not have had russia's turmoil in mind when he made frogland but whatever prompted his choice of material the film stands as a cautionary tale of universal application frogland could be the soviet union italy germany or japan in the 1930s or any country of any era that lets its guard down and is overwhelmed by tyranny it's a fascinating film even a charming one in its macabre way but its message is no joke
Actual: Positive, Predicted: Positive (Correct)

03 실습 #1 실행 결과

Review 4: i generally love this type of movie however this time i found myself wanting to kick the screen since i can't do that i will just complain about it this was absolutely idiotic the things that happen with the dead kids are very cool but the alive people are absolute idiots i am a grown man pretty big and i can defend myself well however i would not do half the stuff the little girl does in this movie also the mother in this movie is reckless with her children to the point of neglect i wish i wasn't so angry about her and her actions because i would have otherwise enjoyed the flick what a number she was take my advise and fast forward through everything you see her do until the end also is anyone else getting sick of watching movies that are filmed so dark anymore one can hardly see what is being filmed as an audience we are impossibly involved with the actions on the screen so then why the hell can't we have night vision

Actual: Negative, Predicted: Positive (**Incorrect**)

03 실습 #1 실행 결과

Review 5: like some other people wrote i'm a die hard mario fan and i loved this gamebr this game starts slightly boring but trust me it's worth it as soon as you start your hooked the levels are fun and exiting they will hook you 'till your mind turns to mush i'm not kidding this game is also orchestrated and is beautifully donebr to keep this spoiler free i have to keep my mouth shut about details but please try this game it'll be worth itbr story 9 9 action 10 1 it's that good hardness 10 attention grabber 10 average 10

Actual: Positive, Predicted: Positive (**Correct**)

참고 자료

References

- [KLD & JSD](#)
- [LSGAN](#)
- [Lipschitz](#)
- [WGAN](#)
- [Seq2Seq](#)
- [Attention](#)
- [Attention Is All You Need](#)
- [Positional Encoding](#)

고급머신러닝 보고서
감사합니다